

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : INFORMATION TECHNOLOGY

Development of a Remote Testing System and AI-Powered Defect Analysis Chatbot for the Automotive Sector

By: *Houssem Eddine Saada*

Academic supervisor: *Mr. Foued Oueslati*

Corporate Internship Supervisor: *Ms. Lamia Slimane*

Capgemini



I validate the submission of the student's report:

- Name & Surname : Housseem Eddine Saada

Business site Supervisor

- Name & Surname : Lamia Slimane

Stamp & Signature


Altran Telnét Consulting
Parc Technologique El Chazoua
Bloc E3 2083 Ariana
Tél 31 37 88 00

Academic Supervisor

- Name & Surname : Foued Oueslati

Signature

Tunis le 17-06-2026



Dedication

*I dedicate this work
to my parents, Helmi and Hela,
the source of happiness, nobility, and affection in my life.
Thank you for your constant support, encouragement,
and your precious help, which gave me the strength
to continue until the very end.
To my sister, Hadir,
for always being by my side and for being the best companion I could ask for.
To all my family members,
for their love and support.
To all my friends,
for the love, support, beautiful memories,
and unforgettable moments we shared together.
Thank you.*

Housseem Eddine Saada

Acknowledgment

I would like to express my deepest gratitude to all those who supported me throughout the journey of completing this end-of-study project.

First and foremost, I would like to sincerely thank my industrial supervisor, Ms. Lamia Slimane, for agreeing to supervise me, as well as for her continuous support, valuable guidance, and constructive feedback.

I also extend my sincere gratitude to my educational supervisor, Mr. Foued Oueslati, for his professionalism and kindness, as well as for his supervision, guidance, constructive remarks, and availability. I am also grateful to Capgemini for offering me the opportunity to undertake this project and for providing a stimulating and supportive working environment.

I would like to thank the faculty and staff of ESPRIT, whose teaching and mentorship have equipped me with the knowledge and skills necessary to carry out this project.

Finally, I would like to express my appreciation to the members of the jury for honoring me by evaluating my work.

Contents

List of Figures	x
List of Tables	xii
General Introduction	1
1 Business Understanding	2
1.1 Introduction	2
1.2 Project Context	2
1.3 Host Company Presentation	3
1.3.1 Capgemini Group	3
1.3.2 Capgemini Engineering	4
1.3.3 Project Origin	4
1.3.4 The EVV Team	4
1.4 Problem Definition	5
1.5 Proposed Solution	6
1.6 Objectives and Expected Results	8
1.6.1 Objectives	8
1.6.2 Expected Results	9
1.7 Project Timeline	10
1.8 The CRISP-DM Methodology	11
1.9 Conclusion	13
2 Data Understanding & Data preparation	14
2.1 Introduction	14
2.2 The Octane ALM Platform	14
2.3 Database Connection Architecture	15
2.4 Complete Field Inventory of the Defect Collection	15
2.4.1 Identification and Core Content Fields	16
2.4.2 Status and Lifecycle Management Fields	17
2.4.3 Timestamp Fields	17
2.4.4 Detection Context Fields	18
2.4.5 ECU and Module Fields	18
2.4.6 Vehicle and Car Model Fields	19
2.4.7 Software Version and Integration Step Fields	19
2.4.8 Severity, Priority, and Classification Fields	20
2.4.9 Ownership, Assignment, and Team Fields	21

2.4.10	Relationship and Dependency Fields	22
2.4.11	Release, Sprint, and Planning Fields	22
2.4.12	Attachments, Comments, and Content Fields	22
2.4.13	Location, Visibility, and Custom Fields	23
2.5	Data Exploration and Characteristics	23
2.5.1	Data Volume and Scope	23
2.5.2	Bilingual Nature of the Data	25
2.5.3	Text Length Variability	26
2.5.4	Software Version Format Variability	27
2.5.5	Data Quality Observations	27
2.6	Variable Selection for Semantic Embeddings	28
2.6.1	Semantic Embedding Strategy and Feature Selection	29
2.7	Metadata Selection for the Vector Database	30
2.7.1	Complete Metadata Schema	31
2.7.2	Metadata Boosting Rationale	31
2.8	Data Retrieval and Filtering Strategy	32
2.8.1	Default Filters	32
2.8.2	MongoDB Query Construction	33
2.8.3	Field Projection	33
2.9	Conclusion	33
3	Chatbot Modeling	34
3.1	Introduction	34
3.2	Capgemini Generative Engine API	34
3.3	Multi-Agent Architecture Overview	34
3.4	Agent 1: Conversational AI — GPT-4o	36
3.4.1	Agent Overview and Model Selection	36
3.4.2	Core Conversational Use Cases	37
3.5	Agent 2: Embedding Agent — Amazon Titan Embed Text v1	38
3.5.1	Agent Overview and Model Selection	38
3.5.2	Embedding Generation Process	39
3.6	Agent 3: Task Agent — Amazon Nova Lite v1:0	40
3.6.1	Agent Overview and Model Selection	40
3.6.2	Core Task Use Cases	41
3.7	ChromaDB Vector Database	42
3.8	Data Ingestion Pipeline	43
3.9	User Query Pipeline	44
3.9.1	Step 1: Language Detection	45
3.9.2	Step 2: Intent Classification	45
3.9.3	Step 3: Context Extraction	45
3.9.4	Step 4: Keyword Extraction	46
3.9.5	Step 5: Two-Stage Semantic Search	46
3.9.6	Step 6: Metadata Boosting	47
3.9.7	Step 7: LLM Reranking	47
3.9.8	Step 8: Response Generation	48
3.10	Pending Defect Flow	49
3.11	Session and Conversation Management	50

3.12	Conclusion	50
4	Evaluation	51
4.1	Introduction	51
4.2	Remote Testing Platform Validation	51
4.2.1	Sound Cable and Voice Recognition Testing	51
4.2.2	WebScrcpy Latency Testing	52
4.2.3	User Acceptance Testing	52
4.2.4	Current Status: Manual Team Testing	53
4.3	Vector Database Build Results	53
4.3.1	Initial Test Build (100 Defects)	53
4.3.2	Root Cause Analysis: 403 Embedding Errors	54
4.3.3	Fix Implementation: Text Truncation and Retry Logic	54
4.3.4	Production Build Results (10,000+ Defects)	54
4.4	Search Quality Assessment	55
4.4.1	Similarity Score Distribution	55
4.4.2	Keyword Boost Effect	56
4.4.3	Metadata Boost Effect	56
4.4.4	Two-Stage Search vs. Single-Stage Search	56
4.5	LLM Reranking Effectiveness	56
4.5.1	Before vs. After Reranking	56
4.5.2	Score Blending Rationale (75/25 Split)	57
4.5.3	Filtering Threshold	58
4.6	Chatbot Functional Testing	58
4.6.1	Intent Detection Accuracy	58
4.6.2	Keyword Extraction Quality	59
4.6.3	Pending Defect Flow Testing	59
4.6.4	Bilingual Response Testing	59
4.6.5	Conversation History Testing	60
4.7	User Feedback Collection and Analysis	61
4.7.1	Feedback Data Model	61
4.7.2	Feedback Collection Flow	61
4.7.3	Admin Feedback Dashboard	61
4.7.4	Evaluation Insights from Feedback	62
4.8	Limitations and Challenges	63
4.8.1	Network Dependency and BMW Proxy Constraints	63
4.8.2	Embedding API Rate Limits and 403 Errors	63
4.8.3	Data Privacy Restrictions	63
4.8.4	Embedding Model Limitations	64
4.8.5	LLM Reranking Latency	64
4.8.6	Vector Database Staleness	64
4.9	Conclusion	64
5	Remote Testing Platform Development	66
5.1	Introduction	66
5.2	Physical Test Setup (Hardware)	66
5.2.1	Head Unit (HU)	66

5.2.2	Test Bench Components	67
5.2.3	Sound Cable Construction	68
5.2.4	Network Architecture	69
5.3	SSH Communication Protocol	70
5.3.1	Definition	70
5.3.2	SSH Protocol Layers	70
5.3.3	SSH Tunnel Implementation	70
5.3.4	SSH Scripts	71
5.4	Android Debug Bridge (ADB)	72
5.4.1	Definition	72
5.4.2	ADB over SSH Tunnel	72
5.4.3	ADB Key Authentication	72
5.5	WebScrepy — Browser-Based Screen Mirroring	73
5.5.1	Definition	73
5.5.2	Key Features	73
5.5.3	Video Decoding	73
5.5.4	Screpy Command Parameters	74
5.6	Voice Testing System	74
5.6.1	Live Speech Mode	75
5.6.2	Dynamic Speech Mode	75
5.7	Django Web Application	76
5.7.1	Framework Choice	76
5.7.2	Django Configuration	76
5.7.3	Database Schema (SQLite)	76
5.7.4	MVT Architecture Pattern	78
5.8	Website Features — Page-by-Page Description	79
5.8.1	Landing Page (index.html)	79
5.8.2	Login Page (login.html)	79
5.8.3	Navigation Bar (navbar.html)	80
5.8.4	Rack Selection Page (racks.html)	81
5.8.5	Calendar Page (calendar.html)	81
5.8.6	Remote Testing Page (screpy.html)	82
5.8.7	User Settings Page (settings.html)	84
5.8.8	User Management (user_management.html)	84
5.8.9	Rack Management (list_racks.html + edit_rack.html)	86
5.8.10	Chatbot Feedback Management (chatbot_feedback.html)	86
5.9	AI Chatbot Widget Integration	87
5.9.1	Widget Design	87
5.9.2	REST API Endpoints	88
5.9.3	Session Management	88
5.9.4	CSRF Protection	88
5.9.5	User Feedback System	89
5.10	Security Considerations	90
5.10.1	Communication Security	90
5.10.2	Django Security Middleware	90
5.10.3	Authentication and Authorization	91
5.10.4	Data Privacy	91

5.11 Version Control	91
5.12 Hosting and Future Production Deployment	92
5.12.1 Current Deployment (Development Environment)	92
5.12.2 Production Migration Path	92
5.13 Conclusion	93
General Conclusion	94
Bibliography	96

List of Figures

1.1	Capgemini Group — Global Presence Map	3
1.2	Capgemini Engineering Logo	4
1.3	Proposed Solution — High-Level User Interaction Flowchart	7
1.4	Project Timeline — Gantt Chart	11
1.5	CRISP-DM Methodology Phases	12
2.1	Micro Focus ALM Octane — Platform Overview	15
2.2	Defect Document Structure — Field Categories Overview	16
2.3	Severity Distribution and Error Occurrence Frequency	21
2.4	Defect Volume by ECU Module — Bar Chart and Pie Chart	24
2.5	Defect Creation Timeline — Stacked Area Chart by Severity	25
2.6	Language Distribution and Description Length by Language	26
2.7	Text Length Variability — Violin + Box Plots for name and description Fields	27
2.8	Software Version Format Distribution and Data Quality — Field Completeness Rates	28
2.9	Variable Selection	29
3.1	Multi-Agent Architecture - BMW Defect Analysis Chatbot	35
3.2	Data Ingestion Pipeline — From MongoDB to ChromaDB	43
3.3	User Query Pipeline — 8-Step Processing Flow	45
3.4	Pending Defect Flow — Context Collection Sequence	49
4.1	Vector Database Build Results — Before and After Fix	54
4.2	L2 Distance Distribution — 148 Defects Similarity Histogram	55
4.3	LLM Reranking — Before vs. After Comparison	57
4.4	Chatbot Conversation — Bilingual Response Example	60
4.5	Admin Feedback Management Dashboard — Thumbs-Up/Down Review	62
5.1	Head Unit (HU) — BMW Infotainment Central Display	67
5.2	Complete Test Bench (Rack) with all components	68
5.3	Sound Cable — 3.5mm Jack to BMW Microphone Plug	69
5.4	Network Architecture — Remote Tester to Test Bench	69
5.5	SSH Protocol Layers	70
5.6	ADB Architecture — Client, Server, and Daemon	72
5.7	Live Speech Mode — Signal Path from Microphone to Head Unit	75
5.8	Database Schema — Entity-Relationship Diagram	78
5.9	Landing Page — Welcome Screen	79
5.10	Login Page — QX Number + Password Authentication Form	80
5.11	Sidebar Navigation Bar	80

5.12	Rack Selection Page — Available Test Benches	81
5.13	Calendar Page — Reservation System with FullCalendar	82
5.14	Remote Testing Page — Voice Testing + Command Execution	83
5.15	Remote Testing Page — WebScrcpy	83
5.16	User Settings Page — Profile Picture and Password Change	84
5.17	User Management Page — User List with Actions	85
5.18	Add User Form	85
5.19	Rack Management Page — Test Bench Configuration	86
5.20	Chatbot Feedback Page — Interaction List and Detailed View	87
5.21	AI Chatbot Widget	87
5.22	Chatbot Feedback Buttons — Thumbs-Up / Thumbs-Down in Widget	89
5.23	GitLab Repository	92

List of Tables

1.1	Capgemini Engineering Overview	5
1.2	Project Development Phases and Key Activities	10
2.1	MongoDB Defect Collection — Field Schema	16
2.2	MongoDB Defect Collection — Phase & Lifecycle Fields	17
2.3	MongoDB Defect Collection — Timestamp & Date Fields	18
2.4	MongoDB Defect Collection — Detection & Origin Fields	18
2.5	MongoDB Defect Collection — ECU & Module Fields	19
2.6	MongoDB Defect Collection — Vehicle & Model Fields	19
2.7	MongoDB Defect Collection — Software Version & Integration Fields	20
2.8	MongoDB Defect Collection — Severity & Classification Fields	20
2.9	MongoDB Defect Collection — Ownership & Responsibility Fields	21
2.10	MongoDB Defect Collection — Hierarchy & Dependency Fields	22
2.11	MongoDB Defect Collection — Release & Program Fields	22
2.12	MongoDB Defect Collection — Attachments & Comments	23
2.13	MongoDB Defect Collection — Additional Metadata Fields	23
2.14	ECU Defect Monitoring — Configuration & Operational Characteristics	24
2.15	Example Defect Names by Language	25
2.16	Content Length Guidelines for Defect Fields	26
2.17	Versioning Formats for Build Identification	27
2.18	Field Categories Excluded from Embedding and Metadata Usage	30
2.19	Metadata Configuration for Defect Display, Filtering, and Boosting	31
2.20	Similarity Boost Conditions for Defect Matching	32
2.21	Default Filter Settings for Defect Ingestion	32
3.1	Multi-Agent Architecture — Model Roles and Responsibilities	35
3.2	Conversational Model Selection Comparison	36
3.3	Intent Categories and Response Strategy	37
3.4	Regex Patterns for Context Extraction	38
3.5	Embedding Model Evaluation and Selection	39
3.6	Task Agent Model Comparison	40
3.7	Keyword Extraction Priority Order	41
3.8	Intent Classification Decision Flow	42
3.9	Vector Database Selection Comparison	43
3.10	Ingestion Pipeline Command-Line Flags	44
3.11	L2 Distance to Similarity Mapping	46
3.12	Keyword-Based Similarity Boost Categories	46

3.13	Metadata Boosting Rules	47
3.14	LLM Reranking Relevance Scale	48
3.15	Session Variables for Conversation Management	50
4.1	Speech Testing Modes	51
4.2	Network Latency Test Scenarios	52
4.3	Embedding Process Metrics	53
4.4	Blending Weight Configuration Comparison	57
4.5	Intent Detection Test Results	58
4.6	Missing Metadata Handling Test Scenarios	59
4.7	Feedback Model Fields and Descriptions	61
5.1	Hardware Components and Their Role in Testing	67
5.2	Custom Audio Cable Materials and Specifications	68
5.3	SSH Tunnel Configuration Parameters	71
5.4	SSH Script Overview and Key Libraries	71
5.5	Screpy Web Features and Technologies	73
5.6	Screpy Web Decoder Options Comparison	74
5.7	Screpy Launch Parameters and Their Purpose	74
5.8	Django Project Configuration Settings	76
5.9	TestBench Model Fields and Constraints	77
5.10	Reservation Model Fields and Constraints	77
5.11	Django MVT Architecture Components	78
5.12	Chatbot API Endpoints	88
5.13	Session Variables for Conversation Management	88
5.14	Security Layers and Their Protection Mechanisms	90
5.15	Django Security Middleware and Their Protections	90
5.16	Development vs. Production Configuration Changes	92

General Introduction

In recent years, the automotive industry has changed a lot. Modern vehicles now depend heavily on software, not only for driving functions but also for features such as infotainment, connectivity, and voice interaction. As these systems become more advanced, testing them becomes more difficult and requires better tools, faster workflows, and easier collaboration between teams.

At the same time, the way engineers work has also changed. Hybrid and remote work have become more common, which has increased the need for tools that allow engineers to perform technical tasks from a distance. In automotive testing, this creates an important challenge: how can engineers access and test real vehicle systems remotely in a secure and efficient way? Another challenge is the large number of defect reports produced during testing. Finding similar past defects among thousands of records can take a lot of time, especially when the same problem is described in different ways or in different languages.

This project was carried out at **Capgemini Engineering Tunisia** for the **BMW Group**, within the **Embedded Verification and Validation (EVV)** team, to respond to these needs. Its main goal was to develop an integrated solution that improves both remote testing and defect analysis in the automotive field. The project is based on two main parts: a **remote testing web platform** for vehicle infotainment systems, and an **AI-powered chatbot** to help test engineers find and analyze similar historical defects.

The first part of the project focuses on remote access to physical test benches through a secure and easy-to-use web platform. Using technologies such as Django, SSH tunneling, ADB, and WebScrepy, the platform allows engineers to control infotainment systems remotely, reserve test benches, and perform voice recognition tests. The second part adds artificial intelligence to the testing process through a chatbot that understands defect descriptions written in natural language, searches through a large database of previous issues, and returns the most relevant results with helpful analysis.

More than just an academic project, this work answers real industrial problems in a context where efficiency, flexibility, and innovation are becoming more and more important. It shows how digital tools and artificial intelligence can improve traditional validation methods and support a more modern way of working in the automotive sector.

This report presents the different stages of the project, from understanding the context and identifying the needs, to designing, developing, and evaluating the final solution. Its purpose is to show how remote testing technologies and AI can help make automotive software validation faster, smarter, and more effective.

Chapter 1

Business Understanding

1.1 Introduction

This chapter presents the context and motivation behind the project, introduces the host company , defines the problem to be addressed, presents the proposed solution, establishes the project objectives and expected results, and outlines the project timeline. The goal is to demonstrate the importance of remote testing systems and AI-powered defect analysis in the automotive sector, and to justify the relevance of the developed solution.

1.2 Project Context

In the current context, characterized by the need for constant adaptation to new technological challenges, the development of remote testing tools emerges as a strategic decision to adapt to the current needs of the market. This approach brings notable benefits, such as the reduction of operational costs, greater agility, flexibility and precision in test execution. The adaptation of the labor market to hybrid and remote regimes is a growing need, especially after the pandemic, which accelerated digital transformation and reshaped the global work environment. The vast majority of companies and workers have perceived both the benefits and challenges of these new models, and adaptation is crucial to ensure productivity, satisfaction and competitiveness in the market.

The pandemic caused a significant transformation in the work models adopted by many companies. According to researchers, approximately 70% of professionals were expected to continue working remotely, even after the end of pandemic-related restrictions [Bloom, 2012]. Thus, digital transformation and the growing demand for innovative technological solutions have driven the automotive industry to adopt advanced tools with the aim of optimizing processes and ensuring operational efficiency. On the other hand, the development of a tool of this nature enables remote work performance, which in turn can improve inclusion, allowing people from diverse regions and cultures (or with specific work backgrounds) to amplify and share their knowledge to enrich their functions. A Gallup study shows that distributed teams allow for greater talent diversity, enriching the corporate environment and expanding innovation[Gallup, 2023].

As a response to the factors referenced above, automotive sector consulting is characterized as a highly dynamic and multifaceted line of business, requiring a deep understanding of the evolution and transformation in this sector, in order to increase the capacity to implement innovative and effective solutions. In this sense, the need arose for this project to be developed by Capgemini Engineering Tunisia, a multidisciplinary company where the synergy between diverse cultures is essential. This

project brings to its client (BMW) an innovative remote testing system that will allow its workers, spread geographically, not to have to travel to their workplaces, being able to perform their functions remotely from their Home-Office, thus reducing their response time to possible problems that may arise.

Beyond the remote testing infrastructure, the increasing volume of defect reports in automotive software projects creates an additional challenge. BMW’s Octane platform stores thousands of defect records from infotainment system testing. When a tester encounters an issue, they need to determine whether a similar defect has already been reported — but manually searching through thousands of records, written in both German and English with varying levels of technical detail, is time-consuming and error-prone. Traditional keyword search fails because different testers describe the same problem using different words. This challenge motivated the development of an AI-powered defect analysis chatbot as an integral component of the remote testing platform — leveraging semantic search and large language models to understand the meaning behind defect descriptions rather than relying on exact keyword matches.

1.3 Host Company Presentation

1.3.1 Capgemini Group

Figure 1.1 below illustrates the worldwide presence of the Capgemini Group across 44 countries.



Figure 1.1: Capgemini Group — Global Presence Map

Capgemini is a multinational founded in 1967 in France by Serge Kampf that is among the largest providers of consulting, technology and outsourcing services in the world. The Capgemini Group has

more than 300,000 professionals in 44 countries and reported a global revenue of 22,096 million euros in 2024 [Capgemini, 2024].

1.3.2 Capgemini Engineering



Figure 1.2: Capgemini Engineering Logo

Capgemini Engineering is the engineering and Research and Development (R&D) hub of the Capgemini Group, focused on helping clients develop intelligent products and services, operations and services at scale to achieve smarter industry. With deep industry knowledge and cutting-edge digital and software technologies, combined with an agile mindset, this division of the Group has become the global leader in Engineering Research and Development (ERD). The company holds several reference certifications, most notably the ISO 9001 standard — the most widely used management system worldwide and the international reference for Quality Management System Certification [IOFS, 2015].

The business sector of Capgemini Engineering can be described as a combination of business consulting, information technology services, and business process outsourcing. The company operates in diverse sectors, offering solutions and services for various industries. The main mission of Capgemini Engineering is to help organizations achieve their objectives through digital transformation, offering innovative and practical solutions to drive operational efficiency, productivity, and market growth. The company combines its consulting and technology experience to provide customized solutions that meet the specific needs of its clients, creating innovative products, solutions, operations, and services.

This organization is recognized for its collaborative approach and partnership with its clients. It works in collaboration with its teams to understand their needs, challenges, and objectives, with the purpose of providing personalized and effective solutions. Capgemini Engineering seeks to promote innovation allied with sustainability, using emerging technologies and responsible practices in its projects.

1.3.3 Project Origin

This project was developed within Capgemini Engineering Tunisia, working for BMW — a renowned company in the automotive sector. The development of the project was proposed by a PPL (Pay-Per-Lead) of Capgemini Engineering Germany, who, allied with Capgemini Engineering Tunisia, had the objective of presenting a solution to respond to this new challenge — Development of a Remote Testing Tool for the Automotive Sector — adapting to this new VUCA (Volatility, Uncertainty, Complexity, Ambiguity) world.

1.3.4 The EVV Team

The project fits within the Embedded Verification and Validation (EVV) team. This team's mission is to reduce the risk of software failure through the use of more efficient testing techniques in test case

design. The vision of the testing team is the starting point for software quality, increasing exposure to defects and providing support in debugging activities. The EVV team operates across multiple BMW infotainment projects, validating software releases on physical test setups (Head Units, audio modules, and other ECUs) before they reach production vehicles.

Attribute	Details
Full Name	Capgemini Engineering (Capgemini Group).
Founded	1967, Grenoble, France (by Serge Kampf).
Global Presence	44 countries, 300,000+ employees.
Revenue (2024)	£22,096 million.
Key Certification	ISO 9001 — Quality Management System.
Project Location	Capgemini Engineering Tunisia.
Client	BMW Group — Automotive Sector.
Team	Embedded Verification and Validation (EVV).
Team Mission	Reduce software failure risk via efficient test case design.

Table 1.1: Capgemini Engineering Overview

Table 1.1 above summarizes the key attributes of the host company, including its global presence, the project location in Tunisia, and the EVV team responsible for this project.

1.4 Problem Definition

The Embedded Verification and Validation (EVV) team is responsible for testing BMW infotainment systems on physical test benches (racks) located in BMW’s offices in Munich, Germany and Tunisia. These test benches contain Head Units (HUs) — embedded computers running the vehicle’s infotainment software — along with associated hardware modules (rear-seat entertainment, wireless charging, antenna systems, and body controllers). Until the development of this project, all testing was performed exclusively on-site, requiring testers to be physically present at the test benches. This constraint created several significant challenges:

Geographic Dependency: Testers could not perform tests without traveling to Capgemini/BMW’s offices. This resulted in high travel costs, time delays, and limited testing hours — testers could only work during their on-site visits, leading to underutilization of the test hardware during off-hours and weekends.

Voice Testing Impossibility: Testing the vehicle’s voice recognition system — a critical infotainment feature — required a person to physically speak into a microphone in front of the Head Unit. There was no mechanism to send voice commands remotely, making voice recognition testing entirely dependent on physical presence.

Test Bench Scheduling Conflicts: Multiple testers from different teams needed access to the

same physical test benches. Without a centralized reservation system, scheduling was managed informally through emails and spreadsheets, leading to conflicts, double-bookings, and lost testing time when a tester arrived to find the bench occupied by another team.

No Remote Screen Access: Testers had no way to see or interact with the Head Unit’s display remotely. Even for simple verification tasks — such as confirming a software update was applied correctly or checking a visual change on the display — a physical visit to the test bench was required.

Defect Duplication Problem: BMW’s Octane ALM platform stores tens of thousands historical defect records from infotainment system testing. When a tester encounters an issue, they need to determine whether a similar defect has already been reported. However, searching through thousands of records — written in both German and English, with varying levels of technical detail and different terminology for the same problems — is extremely time-consuming and error-prone. Traditional keyword search fails because different testers describe the same problem using different words (e.g., "screen freezes" vs. "display becomes unresponsive" vs. "Bildschirm friert ein"). This leads to duplicate defect reports, wasted investigation effort, and difficulty tracking patterns across related issues.

These challenges collectively represented a significant bottleneck in the testing workflow, reducing team productivity, increasing operational costs, and limiting the speed at which software defects could be identified, reported, and resolved. The need for a comprehensive solution that would address all of these pain points simultaneously motivated the development of this project.

1.5 Proposed Solution

To address the challenges described in Section 1.4, this project proposes the development of the BMW Remote Testing Platform — an integrated web-based system that combines remote hardware access, voice testing capabilities, test bench management, and AI-powered defect analysis into a single, unified solution. The platform is designed around two interconnected pillars:

Remote Testing Infrastructure: A web application built with Django that enables testers to access physical test benches remotely from any location with network access. The platform provides real-time screen mirroring and control of the Head Unit through WebScrcpy (displayed directly in the browser), secure communication via SSH tunneling and ADB (Android Debug Bridge), and — most innovatively — remote voice recognition testing through a custom-built sound cable and text-to-speech system. Testers can send voice commands either by speaking into their microphone (Live Speech mode) or by typing text that is converted to speech on the remote PC (Dynamic Speech mode). A calendar-based reservation system with automatic conflict detection ensures organized, fair access to shared test benches.

AI-Powered Defect Analysis Chatbot: An intelligent assistant integrated as a floating widget in the web application that helps testers analyze defect descriptions using natural language. When a tester describes a problem they have encountered, the chatbot uses semantic search — powered by Amazon Titan vector embeddings and a ChromaDB vector database — to find the most similar historical defects from BMW’s Octane platform. A multi-agent AI architecture combines three specialized models: GPT-4o for conversation management, Amazon Titan Embed for semantic similarity, and Amazon Nova Lite for intelligent result reranking. The system supports bilingual operation (German and English) with automatic language detection, enabling testers to work in their preferred language.

Figure 1.3 below presents a high-level flowchart illustrating the typical user interactions with the BMW Remote Testing Platform, showing how the two pillars of the solution work together to support the testing workflow.

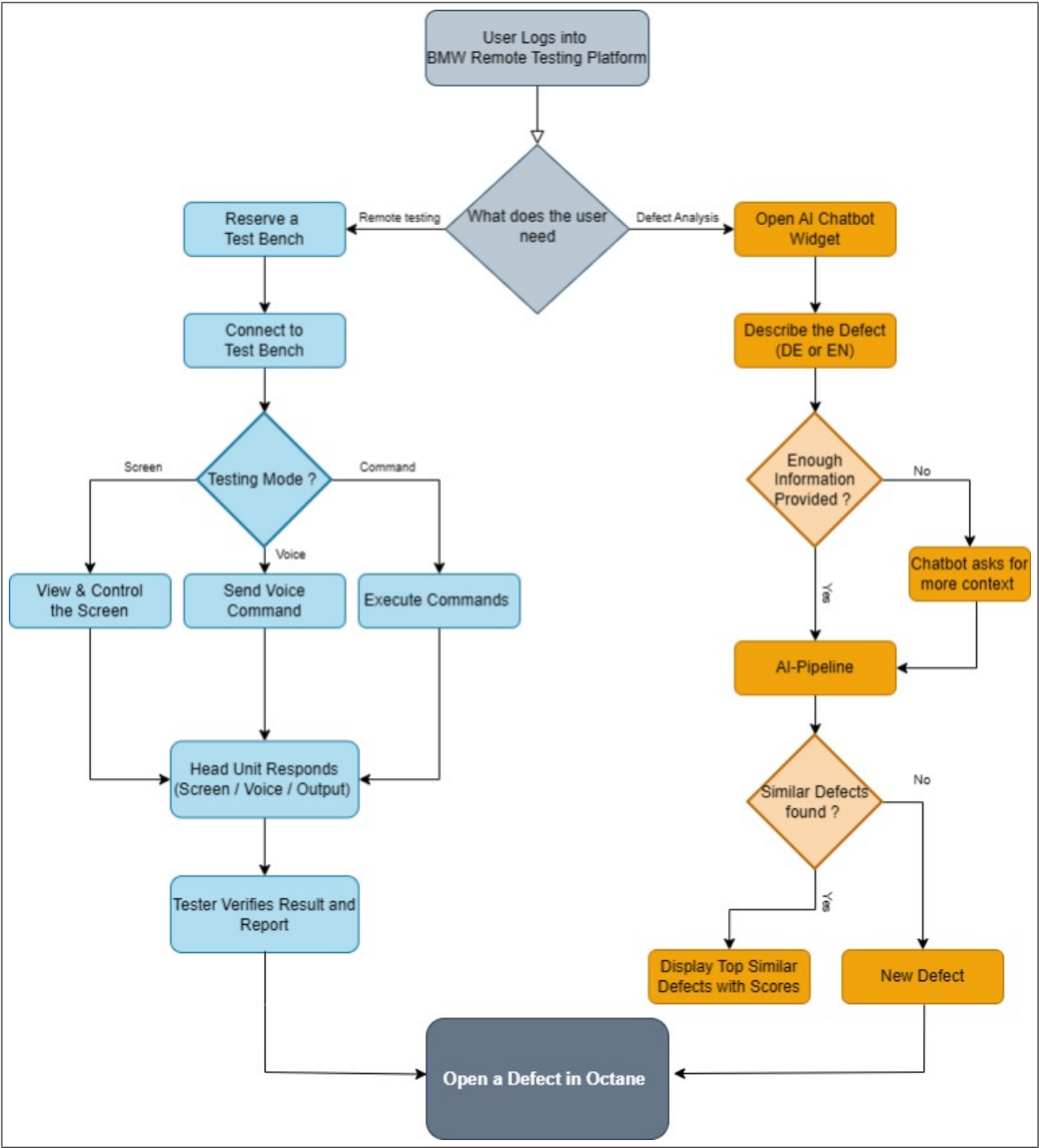


Figure 1.3: Proposed Solution — High-Level User Interaction Flowchart

By combining these two pillars into a single platform, the proposed solution eliminates the geographic dependency of the testing process, enables voice testing from any location, provides structured test bench scheduling, and dramatically reduces the time needed to identify duplicate or similar defects — transforming what was previously a manual, on-site-only workflow into a fully remote, AI-assisted testing experience.

1.6 Objectives and Expected Results

1.6.1 Objectives

The main objective of this project consisted in the development of a remote testing system using a web interface that would allow the testing of vehicle functionalities in real time. The use of this interface ensures greater security for the system, requiring users to access the platform using specific credentials.

With the objective of structuring and efficiently organizing a work plan, the various stages of the project were segmented, ensuring a clear and systematic approach. This made it possible to facilitate the management and monitoring of tasks throughout the development of the project, resulting in the definition of the following phases:

- Understand and evaluate the operation of remote tests carried out at BMW, as well as the methods used in their execution
- Highlight the benefits and practical applications of the project in the context of the company's technological evolution
- Elaborate a detailed strategic plan to structure and guide the development of the project
- Design and implement a remote testing system accessible globally, regardless of geographic location
- Develop an intuitive and functional web interface to allow access to the testing system efficiently
- Integrate SSH tunneling for secure communication between the web platform and remote test setups
- Enable real-time screen mirroring and control of the Head Unit via WebScrcpy
- Implement voice recognition testing capabilities — both Live Speech (microphone) and Dynamic Speech (text-to-speech) — remotely
- Create a calendar-based reservation system for testbench
- Build an admin panel for user and testbench management with role-based access control
- Develop an AI-powered defect analysis chatbot integrated into the web platform
- Connect to BMW's Octane database to retrieve and index historical defect records
- Build a vector database with semantic embeddings
- Implement a multi-agent AI architecture with three specialized models for conversations, task operations, and embeddings respectively
- Create a multi-stage search pipeline combining semantic search, keyword boosting, metadata boosting, and LLM reranking for finding similar defects
- Support bilingual operation (German and English) with automatic language detection

- Produce comprehensive technical documentation about the developed project, ensuring its continuity and maintenance

In addition to these specific objectives, the project also aimed at developing comprehensive and in-depth competencies in programming, focusing on the various languages mentioned throughout this report. It was also expected that the work would foster practical capabilities in handling and operating the software used within the scope of the project, promoting a solid familiarity with its functionalities. In parallel, the aim was to strengthen knowledge in the area of data communication, namely the SSH protocol and the tunneling process, as well as in artificial intelligence technologies including vector embeddings, semantic search, and large language models.

1.6.2 Expected Results

Upon completion of the project, it is expected that all users with access to the network where the test benches are hosted and with the necessary credentials can access the website and perform, in real time, the remote testing of the available functionalities. It is also intended that the web interface be simple and intuitive, ensuring an accessible user experience, allowing users to easily understand how to explore its different functionalities. Furthermore, the interface should be designed flexibly, enabling the integration of future functionalities as needs evolve.

Remote Testing Platform Results

- All users with network access and valid credentials can log into the website and perform real-time remote testing on the available test setups
- The web interface is simple and intuitive, allowing users to easily explore its different functionalities
- The calendar system effectively prevents scheduling conflicts for testbench reservations
- Voice recognition testing (Live Speech and Dynamic Speech) works reliably over SSH connections at varying distances
- The interface supports dynamic addition of new test setups by administrators without code changes

AI Chatbot Results

- The chatbot accurately classifies user intent (defect description, question, or greeting) and responds appropriately in the user's detected language
- When a tester describes a problem, the system finds the most semantically similar defects from the historical database with relevant similarity scores
- Search results are enriched with metadata (car model, software version, ECU, dates) and boosted based on the tester's vehicle context
- The LLM reranking step significantly improves the relevance of top results compared to pure vector similarity search

- The vector database can be rebuilt and updated through command-line tools with configurable filters (ECU, phase, date range, limit)
- The chatbot asks for missing context (car model, software version) before analysis to improve search accuracy, with an option to skip

1.7 Project Timeline

With the aim of organizing tasks in a clear and efficient manner, the project was divided into structured phases. This organization allows for a practical visualization of the tasks to be carried out, the start and end dates of each task, as well as their duration, facilitating the monitoring of progress and the identification of possible dependencies between activities. This ensures more effective time management, contributing to the success of the project.

Table 1.2 below details the seven phases into which the project was segmented, along with the key activities carried out in each phase.

Phase	Activities
Phase 1: Study & Analysis	Analysis of existing BMW testing tools and workflows, study of SSH/ADB protocols, evaluation of web frameworks (Django vs Flask), study of AI/ML technologies for semantic search, requirements gathering with BMW client.
Phase 2: Infrastructure Setup	Sound cable construction (jack + BMW Mic Plug), SSH server configuration (3 layers: transport, authentication, session), ADB tunneling setup, WebScrepy integration, MongoDB connection to Octane platform.
Phase 3: Web Development	Django project setup with SQLite database, frontend development (HTML/CSS/JavaScript), login system with role-based access, calendar with conflict detection, admin panel for user and setup management, sidebar navigation.
Phase 4: Remote Testing Features	Live Speech integration (microphone → SSH → Head Unit), Dynamic Speech (text-to- speech → SSH → Head Unit), WebScrepy embedding in website, command execution panel, output display area
Phase 5: AI Chatbot Development	Capgemini Generative Engine API integration, MongoDB Octane data retrieval, ChromaDB vector database setup, multi-agent architecture , semantic search pipeline, LLM reranking, bilingual support (DE/EN), pending defect flow for context collection.
Phase 6: Testing & Validation	Sound cable testing, remote latency testing (office + home-office), user acceptance testing with Capgemini collaborators worldwide, vector database rebuild testing , chatbot accuracy evaluation, feedback collection.
Phase 7: Documentation	Technical documentation, project handovermaterials, preparation for future production deployment.

Table 1.2: Project Development Phases and Key Activities

Figure 1.4 below illustrates the complete project timeline as a Gantt chart, spanning from October 2025 to May 2026, with the seven phases distributed across the project duration.

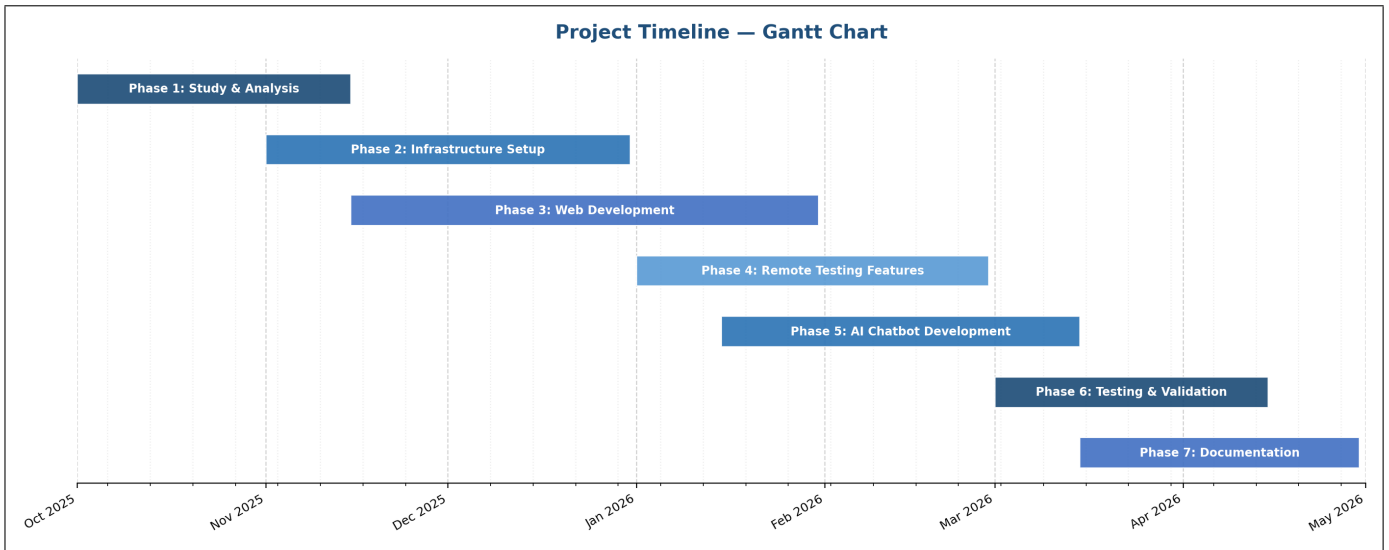


Figure 1.4: Project Timeline — Gantt Chart

The project was structured following the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, which provided a systematic framework for organizing the work into well-defined phases. GitLab was used for version control with a feature-branch workflow. Regular demonstration sessions and follow-up meetings were held with the BMW client. These sessions were fundamental to ensure process transparency, collect real-time feedback, and align the developed functionalities with the client's expectations and needs. The interest and enthusiasm demonstrated by the client in each of these meetings were clear indicators of the added value the project was bringing.

1.8 The CRISP-DM Methodology

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a widely adopted methodology for structuring data science and artificial intelligence projects. Originally developed in 1996 by a consortium including SPSS, Teradata, Daimler AG, NCR Corporation, and OHRA [Chapman, 2000], it has since become the de facto standard for organizing analytical and AI-driven projects across industries. According to multiple industry surveys, CRISP-DM remains the most commonly used methodology for data mining and machine learning projects worldwide [KDnuggets, 2014]. The methodology defines six phases that form an iterative lifecycle:

1. **Business Understanding:** This initial phase focuses on understanding the project objectives and requirements from a business perspective. It involves defining the problem, identifying stakeholders, and translating business goals into technical objectives. In this project, it corresponds to analyzing BMW's remote testing needs and defining the scope of both the web platform and the AI chatbot.
2. **Data Understanding:** This phase involves collecting, exploring, and assessing the quality of available data. It includes identifying data sources, understanding data structures, and performing initial data exploration. For this project, it involves the in-depth analysis of the Octane ALM defect database with its 270+ fields and 10,000+ records.
3. **Data Preparation:** In a traditional CRISP-DM project, this phase covers data cleaning, transformation, and feature engineering. In this project, data preparation is covered in the

same chapter as Data Understanding (Chapter 2), since the preparation steps—field selection, text concatenation, truncation, and embedding generation—are a direct continuation of the data exploration and analysis work.

4. **Modeling:** This phase involves selecting and applying appropriate modeling techniques. It includes model selection, parameter tuning, and architecture design. In this project, it encompasses the design of the multi-agent AI architecture with GPT-4o, Amazon Titan Embed, and Amazon Nova Lite, as well as the vector database and search pipeline design.
5. **Evaluation:** The evaluation phase assesses how well the model meets the business objectives defined in the first phase. It involves testing, validation, and determining whether the results are satisfactory. This project evaluates both the remote testing platform functionality and the AI chatbot’s search quality and accuracy.
6. **Deployment:** The final phase covers putting the results into production use. It includes planning deployment, monitoring, maintenance, and producing final documentation. In this project, deployment covers the complete system from hardware setup to software installation, security configuration, and production readiness.

A key strength of CRISP-DM is its iterative nature: the arrows between phases indicate that the process is not strictly linear. Findings from later phases frequently lead to revisiting earlier ones. For example, evaluation results may reveal the need for additional data understanding or modeling adjustments. This iterative characteristic was essential in this project, where search quality evaluation led to refinements in the embedding pipeline, the addition of LLM reranking, and metadata boosting strategies.

For the purposes of this report, the Data Preparation phase has been merged into the Data Understanding chapter (Chapter 2), as the data transformation steps (field selection, text concatenation, truncation, and embedding generation) are a natural extension of the data exploration work. This results in a five-chapter structure that closely follows the remaining CRISP-DM phases.

Figure 1.5 below illustrates the six interconnected phases of the CRISP-DM methodology and their iterative relationships.

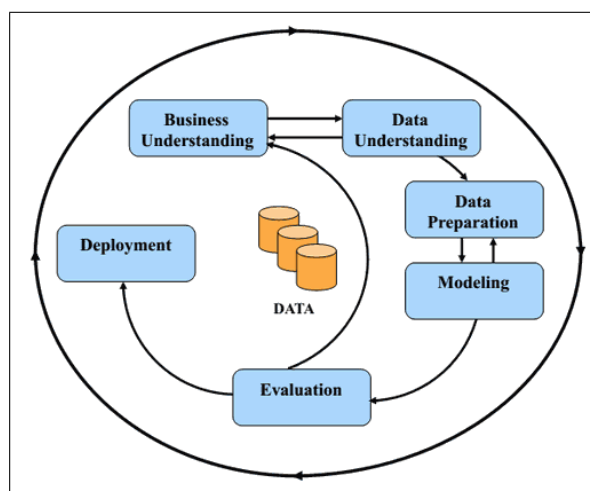


Figure 1.5: CRISP-DM Methodology Phases

1.9 Conclusion

This chapter presented the context of the project, the host company (Capgemini Engineering Tunisia), the problem definition highlighting the challenges of on-site-only testing and manual defect searching, the proposed solution combining remote testing infrastructure with an AI-powered chatbot, the project objectives and expected results, as well as the project timeline . With this foundation, the following chapters will detail the data understanding and data preparation , the modeling approach, the evaluation results, and the deployment architecture, contributing to the technological and operational advancement of remote testing in the automotive sector.

Chapter 2

Data Understanding & Data preparation

2.1 Introduction

This chapter provides a comprehensive analysis of the data source that underpins the AI-powered defect analysis chatbot: the BMW Octane platform. The chapter begins by introducing the Octane Application Lifecycle Management (ALM) platform and its role within BMW’s infotainment software development process. It then details the database connection architecture, presents a complete inventory of all available fields in the defect collection — organized by functional category — and concludes with a thorough justification of why specific fields were selected for semantic embeddings and metadata storage in the vector database. Understanding the data is a critical step in the CRISP-DM methodology, as the quality, structure, and characteristics of the data directly influence the performance of the AI model.

2.2 The Octane ALM Platform

Micro Focus ALM Octane (formerly HPE ALM Octane) is an enterprise-grade Application Lifecycle Management platform used by BMW to manage the entire software development lifecycle for its automotive infotainment systems. Octane serves as the central repository for requirements, user stories, defects, tests, and releases across multiple vehicle programs and ECU (Electronic Control Unit) modules[Focus, 2024]. Within the EVV (Embedded Verification and Validation) team, Octane is the primary tool where test engineers report, track, and manage defects found during integration testing of the infotainment Head Units. Every time a tester discovers a bug they create a defect record in Octane with a structured set of fields describing the problem, its severity, the affected ECU module, the vehicle model, and the software version under test. Over time, this process generates thousands of defect records. In a typical 3-month period, approximately 10,000 defects are created for the Central Display (CD) ECU alone during the Integration testing phase. These records are written in both German and English, depending on the tester’s language preference, and vary significantly in length and level of detail from single-sentence descriptions to multi-paragraph technical analyses with reproduction steps, log references, and expected vs. actual behavior comparisons. The Octane platform exposes its data through a MongoDB back-end hosted on MongoDB Atlas [MongoDB, 2024], a cloud-managed database service. This allows external tools — such as the AI chatbot developed in this project — to connect to the database and retrieve defect records programmatically using the PyMongo driver.

Figure 2.1 below provides an overview of the Micro Focus ALM Octane platform interface.

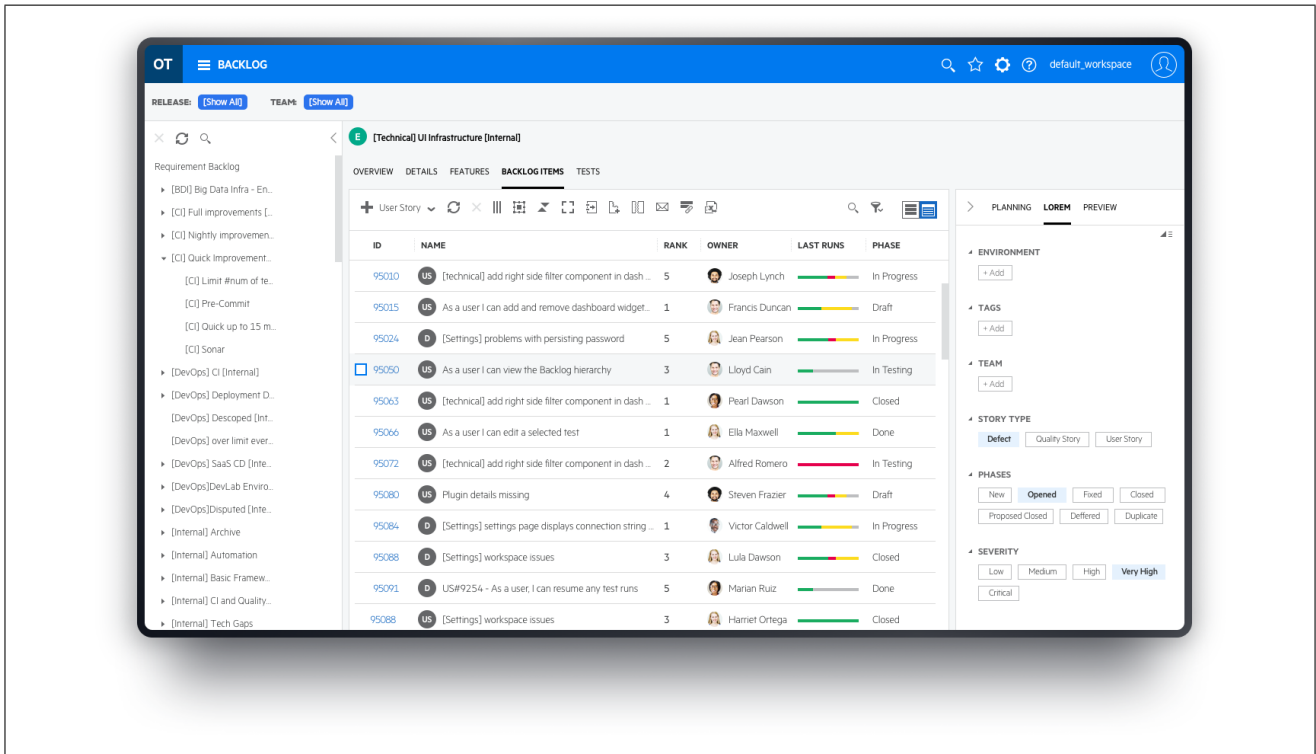


Figure 2.1: Micro Focus ALM Octane—Platform Overview

2.3 Database Connection Architecture

The connection to the Octane defect database is established through a MongoDB Atlas SRV connection string, which provides automatic DNS-based service discovery, TLS encryption, and replica set configuration. The connection parameters are stored in a separate JSON configuration file (`mongodb_config.json`) to keep credentials out of the application source code. The database user is a read-only viewer account, which means the AI chatbot can retrieve defect records for analysis but cannot modify or delete any data. This is an important security measure, ensuring that the AI system operates in a strictly read-only capacity and cannot alter production defect data. The PyMongo driver establishes a connection pool and manages retries transparently. Queries to the defect collection support MongoDB's rich query language, including field-level filtering, date range queries and array element matching. The system applies filters at the database level — for example, filtering by ECU module (`ecu_to_modul.name = "CD"`) and detection phase (`detected_on.name = "02-Integration"`) — before transferring data to the application layer, minimizing bandwidth usage and processing time.

2.4 Complete Field Inventory of the Defect Collection

Each defect document in the Octane MongoDB collection contains an extensive set of fields that capture every aspect of the defect's lifecycle — from its initial detection to its resolution and closure. The complete defect schema comprises over 270 raw field columns. Many of these fields are complex objects with sub-properties (`type`, `id`, `user_activation_status`, `name`), meaning the actual number of logical entities is approximately 130. This section presents the complete inventory of all fields, organized into functional categories, to provide a thorough understanding of the data available for the

AI model. Understanding the full scope of available data is essential for making informed decisions about which fields to include in the embedding pipeline and which to store as metadata for filtering and boosting. The following subsections describe each category in detail.

Figure 2.2 below provides an overview of the different fields of a defect in BMW.

Figure 2.2: Defect Document Structure — Field Categories Overview

2.4.1 Identification and Core Content Fields

Table 2.1 below lists the fundamental fields that uniquely identify each defect and contain the primary textual content describing the problem. The name and description fields are the most semantically rich fields in the entire document and form the basis of the embedding pipeline.

Field Name	Type	Description / Example Values
id	Integer	Unique defect identifier (e.g., 2548182). Auto-generated by Octane. Used for deduplication and display.
type	String	Entity type, always “defect” for this collection.
name	String	Short defect title (10–100 chars). Bilingual (DE/EN).
description	String	Full defect description (1 sentence to 5+ paragraphs). Contains problem details, reproduction steps, expected vs. actual behavior. Bilingual (DE/EN).
uuid	String	Universal unique identifier (UUID) for cross-system reference.
res_descr	String	Resolution description — how the defect was fixed (if applicable). Often empty for open defects.

Table 2.1: MongoDB Defect Collection — Field Schema

2.4.2 Status and Lifecycle Management Fields

Table 2.2 below presents the fields that track the defect’s progression through its lifecycle, from initial creation to resolution. The phase field is particularly important as it indicates whether a defect is still active, under analysis, or has been concluded.

Field Name	Type	Description / Example Values
phase	Object	Current lifecycle phase. Values: 01-New, 02-In Pre-Analysis, 03-Assigned, 04-In Analysis, 05-Solution Proposed, 06-Concluded, 07-In Implementation, 08-In Verification, 09-Concluded without action, 10-On Hold.
phase.metaphase	Object	Higher-level phase grouping.
metaphase	Object	Duplicate reference to metaphase at document root level.
substatus	Object	Sub-status within the current phase for finer tracking.
rejected_status	Object	Rejection status if defect was rejected.
parent_phase	Object	Phase of the parent defect (if this is a child/duplicate).
phase_age	Integer	Number of days the defect has been in the current phase.
cycle_time	Object	Total cycle time from open to close.
cycle_time_value	Float	Numeric cycle time value in days.
timeInCurrentPhase	Integer	Days spent in the current lifecycle phase.
blocked	Boolean	Whether the defect is currently blocked.
blocked_reason	String	Reason why the defect is blocked (free text).
blocking_reason_det	Object	Structured blocking reason.
reopens	Integer	Number of times the defect has been reopened after closure.

Table 2.2: MongoDB Defect Collection — Phase & Lifecycle Fields

2.4.3 Timestamp Fields

Table 2.3 below lists the temporal fields, which are crucial for understanding when defects were created, last modified, and resolved. The creation_time and last_modified fields are used as meta-data in the vector database to enable time-based filtering and to show recency in search results.

Field Name	Type	Description / Example Values
creation_time	DateTime	When the defect was first created (ISO 8601 format, e.g., 2026-01-19T10:30:00Z).
last_modified	DateTime	Last modification timestamp. Updated whenever any field changes.
done_date	DateTime	Date when the defect was marked as done/concluded.
version_stamp	Integer	Internal version counter, incremented on each update.
client_lock_stamp	Integer	Client-side lock timestamp for concurrent editing.
updatedByLoaderAt	DateTime	Timestamp of last automated data loader update.
transfer_date	String	Date of Change of Content (CoC) transfer.
com_sol_avail_date	String	Date when a common solution became available.
req_release_date	String	Requested release date for the fix.
error_detection_time	String	Specific time when the error was detected during testing.

Table 2.3: MongoDB Defect Collection — Timestamp & Date Fields

2.4.4 Detection Context Fields

Table 2.4 below details the fields that capture where and how the defect was discovered. The detected_on field is used as a primary filter when building the vector database, as it determines the testing phase (e.g., Integration, System Test, Vehicle Test) and directly impacts the relevance of defects for the target user group.

Field Name	Type	Description / Example Values
detected_by	Object	The person who detected the defect.
detected_on	Object	Testing phase where the defect was found. Values: 01-CoC, 02-Integration, 03-Function Test, 04-System Test, 05-Vehicle Test, 06-Field.
detected_in_release	Object	The software release in which the defect was detected.
initially_found_at	Object	Location/site where the defect was first found.

Table 2.4: MongoDB Defect Collection — Detection & Origin Fields

2.4.5 ECU and Module Fields

Table 2.5 below presents the Electronic Control Unit (ECU) fields that identify which hardware/software module is affected by the defect. The ecu_to_modul field is the primary ECU filter used when build-

ing the vector database, as defects are typically analyzed per ECU. The Central Display (CD) is the default ECU for infotainment defect analysis.

Field Name	Type	Description / Example Values
ecu_to_modul	Object	Primary ECU/module responsible. Values: CD (Central Display), HU (Head Unit), MGU (Media Graphics Unit)..
assigned_ecu	Object	ECU assigned for resolution (may differ from detection ECU).
ecu_variant	Object	Specific ECU hardware variant.
ecu_hw_or_app_sw	String	Whether the issue is in ECU hardware or application software.
ecu_no_of_changes	String	Number of ECU changes related to this defect.
sgbd	Object	SGBD (Standard Core Data) file classification.

Table 2.5: MongoDB Defect Collection — ECU & Module Fields

2.4.6 Vehicle and Car Model Fields

Table 2.6 below lists the fields that identify which BMW vehicle model(s) are affected by the defect. The lead_model and model_series fields are critical for the metadata boosting algorithm in the search pipeline, as testers typically work on a specific vehicle model and want to see defects from the same model prioritized.

Field Name	Type	Description / Example Values
lead_model	Object	Primary car model for this defect. Values: NA5, G45, G70, G30, I20, etc.
model_series	Array	All related car models sharing the same platform (array of objects). Example: [{name: "G70"}, {name: "NA5"}, {name: "NA6"}].
legacy_model	Object	Legacy model designation (if applicable).
drive_type	Object	Drive type of the affected vehicle (e.g., BEV, PHEV, ICE).
vin	String	Vehicle Identification Number (VIN) of the specific test vehicle (if applicable).

Table 2.6: MongoDB Defect Collection — Vehicle & Model Fields

2.4.7 Software Version and Integration Step Fields

Table 2.7 below presents the software version fields that track which build or integration step revealed the defect. The software_version field is used in the metadata boosting algorithm to match defects

with the tester’s current software version. These fields exhibit high format variability, which presents a challenge for automated matching.

Field Name	Type	Description / Example Values
software_version	String	Software version identifier. Formats vary: “NA05-25-03-900”, “BMWIDCEVO;2511_i900-25w32.7-1”, “23-11”, “2607”.
involved_i_step	Object	Integration step where defect was found.
target_i_step	Object	Target integration step for fix delivery.
legacy_i_step	String	Legacy integration step reference.
legacy_target_i_step	String	Legacy target integration step.
closed_in_ver	String	Software version in which the defect was fixed and closed.
mapdb_vers	String	Navigation map database version (for map-related defects).

Table 2.7: MongoDB Defect Collection — Software Version & Integration Fields

2.4.8 Severity, Priority, and Classification Fields

Table 2.8 below lists the classification fields that allow defects to be categorized by impact, urgency, and type. While severity and priority are commonly used in defect triage, they were not selected for embedding because they describe the business impact rather than the technical nature of the problem, which is what the semantic search needs to match on.

Field Name	Type	Description / Example Values
severity	Object	Defect severity level. Values: Critical, High, Medium, Low.
priority	Object	Defect priority for resolution scheduling.
problem_severity	Object	BMW-specific problem severity rating.
defect_type	Object	Defect category type.
error_occurrence	Object	Error frequency. Values: 01-single event, 02-sporadic , 03-reproducible ,etc
error_classif	Object	Error classification.
maturity_grade	String	Maturity grade assessment of the defect.

Table 2.8: MongoDB Defect Collection — Severity & Classification Fields

Figure 2.3 below visualises the severity distribution (left, donut chart) and error occurrence frequency (right, lollipop chart) for the defect database. The severity distribution shows that Medium-severity

defects dominate the dataset, while the error occurrence chart reveals that Reproducible errors are the most common, with reproducible multi-lifecycle defects being relatively rare.

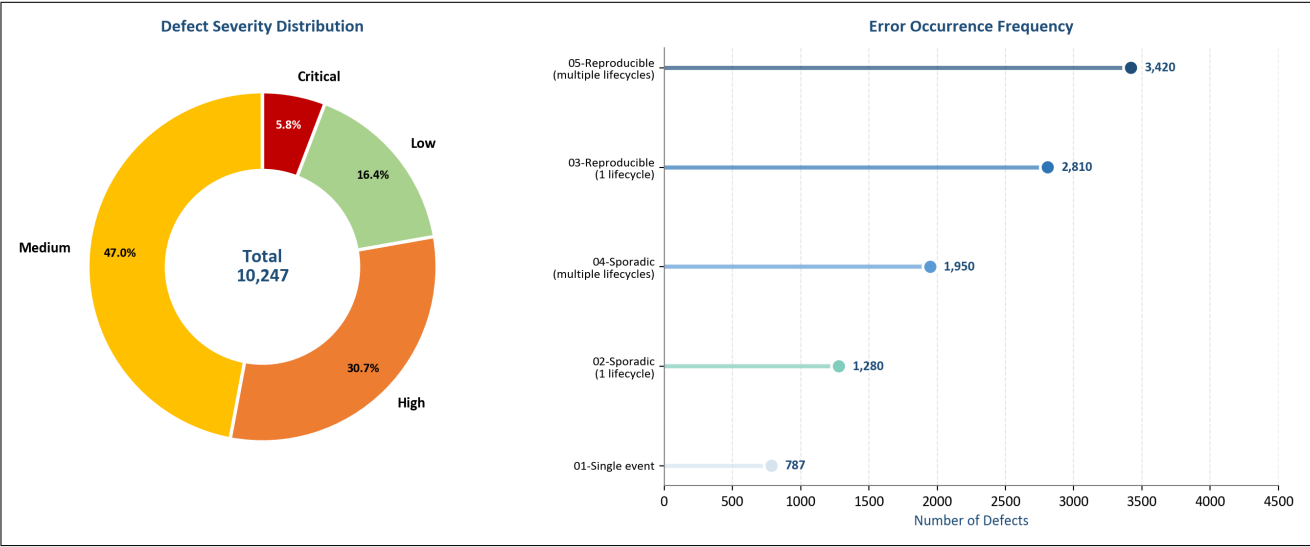


Figure 2.3: Severity Distribution and Error Occurrence Frequency

2.4.9 Ownership, Assignment, and Team Fields

Table 2.9 below presents the fields that track who is responsible for detecting, analyzing, and resolving the defect. They are primarily used for workflow management and team performance metrics. They were not included in the AI model because they describe organizational attribution rather than technical content.

Field Name	Type	Description / Example Values
owner	Object	Current owner of the defect.
author	Object	Person who created the defect report.
qa_owner	Object	QA engineer responsible for verification.
team	Object	Team responsible for the defect.
solution_responsible	Object	Person responsible for implementing the solution.
function_responsible	String	Function-level responsible person.
function_responsible	String	Secondary function-level responsible person.
follower	Object	Person following/watching the defect.

Table 2.9: MongoDB Defect Collection — Ownership & Responsibility Fields

2.4.10 Relationship and Dependency Fields

As shown in Table 2.10 below, Octane supports rich relationships between defects, including parent-child hierarchies (for duplicate clustering), dependencies (blocking/blocked-by), and cross-references. The `parent_child` field is particularly relevant for identifying confirmed duplicates, though these relationships are not used in the embedding model.

Field Name	Type	Description / Example Values
<code>parent_type</code>	String	Type of parent entity (if this defect has a parent).
<code>parent_id</code>	Integer	ID of the parent entity.
<code>parent_name</code>	String	Name of the parent entity.
<code>ancestors</code>	Array	Chain of ancestor entities in the hierarchy.
<code>parent_child</code>	Object	Parent/child relationship.
<code>relation_to</code>	String	Related defect ID (for cross-references).

Table 2.10: MongoDB Defect Collection — Hierarchy & Dependency Fields

2.4.11 Release, Sprint, and Planning Fields

Table 2.11 below lists the planning fields that support the defect resolution workflow, tracking target releases, sprints, and deadlines. These are used for project management purposes but do not contribute to the semantic understanding of what the defect describes.

Field Name	Type	Description / Example Values
<code>release</code>	Object	Target release for the fix.
<code>target_set</code>	Object	Target set/milestone.
<code>target_week</code>	Object	Target week for resolution.
<code>program</code>	Object	BMW program/project.

Table 2.11: MongoDB Defect Collection — Release & Program Fields

2.4.12 Attachments, Comments, and Content Fields

Table 2.12 below lists the fields that contain additional content such as file attachments (screenshots, logs, videos), comments from team members, and links to test cases. While comments and attachments can contain valuable information, they are not currently used in the embedding pipeline due to their unstructured nature and the complexity of extracting text from binary attachments.

Field Name	Type	Description / Example Values
attachments	Array	Attached files (screenshots, logs, videos, traces).
has_attachments	Boolean	Whether the defect has file attachments.
comments	Array	Discussion comments from team members. Array of text entries.
has_comments	Boolean	Whether the defect has comments.
requirements	Array	Linked requirements (if defect violates a requirement).
application_modules	Array	Application modules affected by the defect.

Table 2.12: MongoDB Defect Collection — Attachments & Comments

2.4.13 Location, Visibility, and Custom Fields

Table 2.13 below covers the remaining fields related to geographic context, visibility controls, and BMW-specific custom fields for internal workflow management.

Field Name	Type	Description / Example Values
country	Object	Country where the defect was found.
created_at	Object	Location/site where the defect was created.
visible_for	Object	Visibility restriction (who can see this defect).
function2modul	String	Function-to-module mapping.
tolerated_count	String	Number of times the defect has been tolerated.

Table 2.13: MongoDB Defect Collection — Additional Metadata Fields

2.5 Data Exploration and Characteristics

Before selecting fields for the AI model, an exploratory analysis of the defect data was conducted to understand its volume, distribution, linguistic characteristics, and quality. This exploration was essential for making informed decisions about the embedding strategy and for identifying potential challenges.

2.5.1 Data Volume and Scope

The Octane defect collection for the infotainment domain contains a substantial volume of records. Table 2.14 below summarizes the key characteristics of the data scope:

Characteristic	Value
Total defects (CD ECU, Integration)	≈10,000 per 3-month period
Default ECU filter	CD (Central Display)
Default detection phase	02-Integration
Default time window	Last 3 months (configurable)
Available ECU values	CD, HU,etc.
Available detection phases	01-CoC, 02-Integration,etc.
Update frequency	Continuous (defects created daily)

Table 2.14: ECU Defect Monitoring — Configuration & Operational Characteristics

Figure 2.4 below breaks down the defect volume by ECU module, showing both the absolute count per module (bar chart, left) and the relative share (pie chart, right). The CD (Central Display) module accounts for the majority of defects, which is expected given that it is the primary infotainment interface and the default ECU filter used in the data retrieval pipeline.

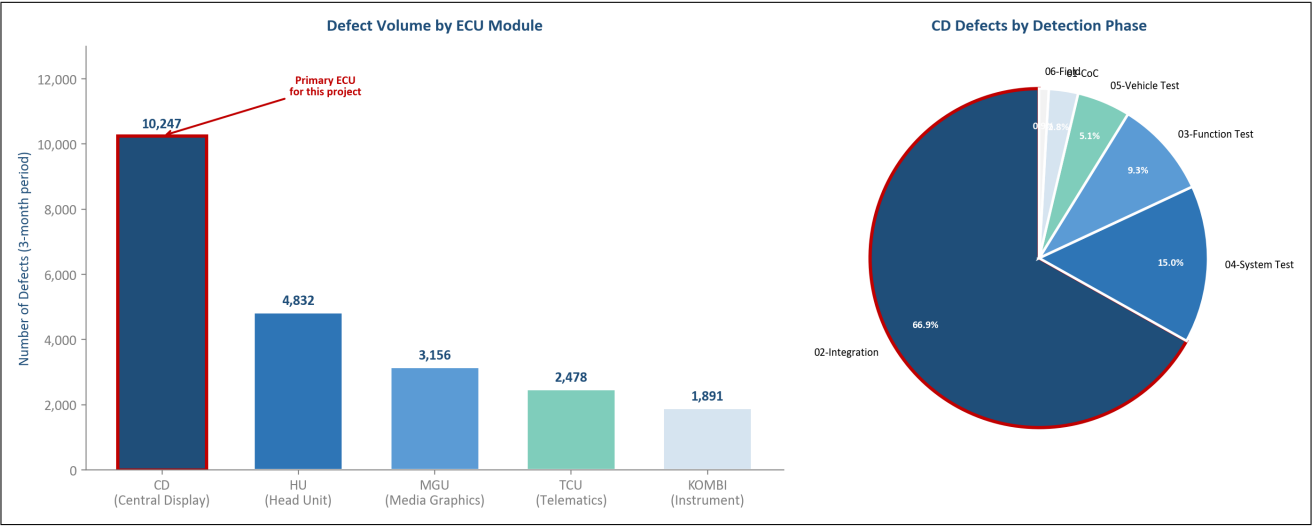


Figure 2.4: Defect Volume by ECU Module — Bar Chart and Pie Chart

Figure 2.5 below shows the defect creation timeline over a recent 12-week window, broken down by severity level in a stacked area chart. This view reveals the temporal patterns in defect reporting: peaks often correspond to integration test milestones or software release cycles where intensive testing takes place.

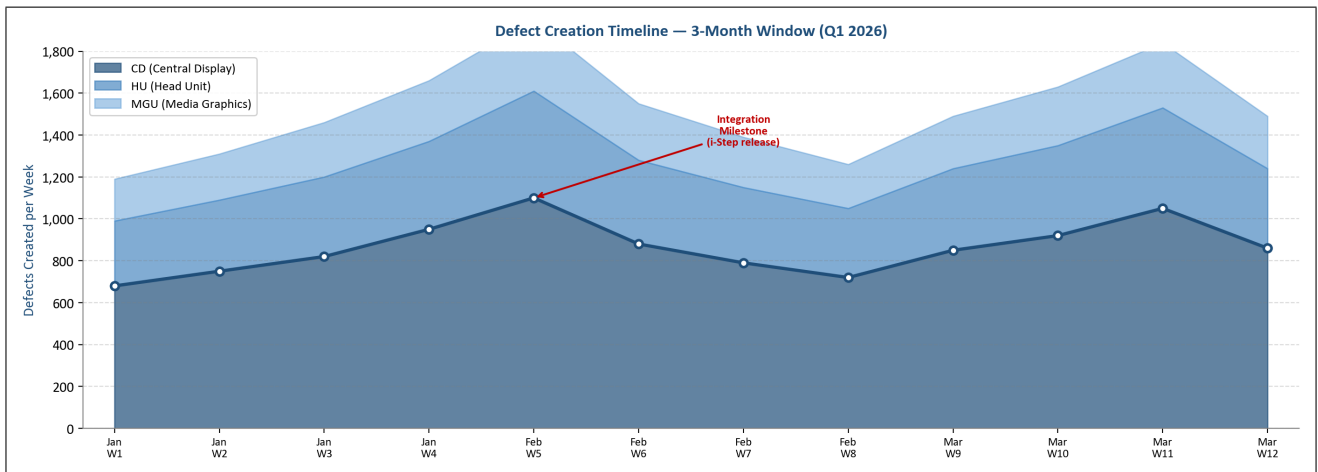


Figure 2.5: Defect Creation Timeline — Stacked Area Chart by Severity

2.5.2 Bilingual Nature of the Data

A distinctive characteristic of the Octane defect database is its bilingual nature. Defect names and descriptions are written in either German or English, depending on the tester’s language preference. This poses a significant challenge for traditional keyword-based search systems but is well-handled by the embedding model (Amazon Titan Embed), which supports multilingual text.

Table 2.15 below illustrates examples of bilingual defect descriptions found in the database:

Language	Example Defect Name
English	Spotify playback crash after opening
German	Sprachsteuerung reagiert nicht nach Neustart des Infotainments

Table 2.15: Example Defect Names by Language

The AI system handles this bilingualism through automatic language detection at the query level. When a user writes a defect description in German, the system detects the language by looking for German keywords (e.g., “defekt”, “fehler”, “problem”), articles (“der”, “die”, “das”), and umlauts (ä, ö, ü, ß). The embedding model generates language semantic vectors, meaning a German query can match an English defect (and vice versa) if they describe the same problem.

Figure 2.6 below illustrates the language distribution of defect descriptions in the database (left) and the description length distribution by language (right), showing that German descriptions tend to be slightly longer due to the compound word structure of the German language. The dashed red line marks the 2,500-character truncation limit applied before embedding generation.

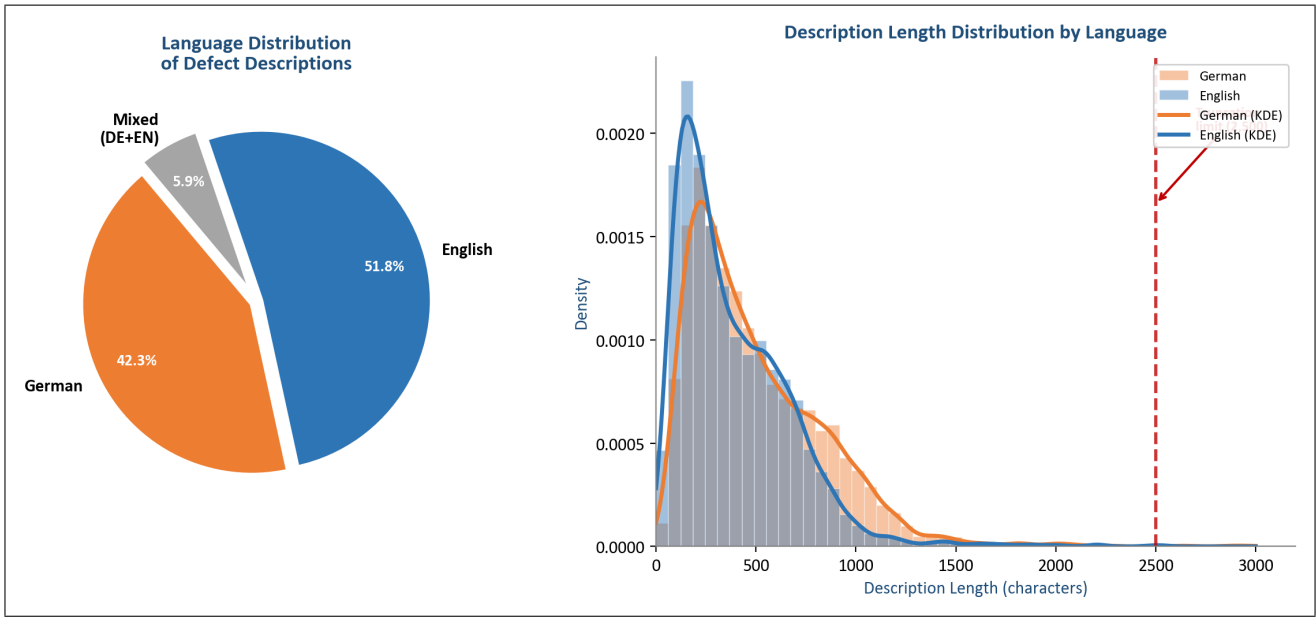


Figure 2.6: Language Distribution and Description Length by Language

2.5.3 Text Length Variability

The name and description fields can vary a lot in length and detail. Some defects have concise, one-line descriptions, while others contain multi-paragraph analyses with reproduction steps, logs, and expected vs. actual behavior comparisons.

Table 2.16 below summarizes the observed text length ranges and the truncation limits applied by the system.

Field	Minimum	Typical	Maximum	Truncation Limit
name	10 characters	30–80 characters	≈300 characters	300 characters
description	1 sentence	1–3 paragraphs	5+ paragraphs	2,500 characters

Table 2.16: Content Length Guidelines for Defect Fields

To handle this variability while staying within the Amazon Titan Embed API token limits, the system truncates the name to 300 characters and the description to 2,500 characters before generating the embedding. This truncation was carefully calibrated: reducing it further would lose important context from longer descriptions; increasing it would exceed the API’s input token limit and trigger 403 errors (a problem that was encountered and resolved during development). The combined embedding text (name + description) is therefore capped at approximately 2,800 characters, which fits comfortably within the Titan model’s input window.

Figure 2.7 below presents violin and box plots for the name (left) and description (right) fields, showing the full distribution shape, quartiles, mean, median, and the truncation limits applied by the system. The violin plots reveal that the name field is tightly concentrated around 50–80 characters, while the description field has a much wider spread with a long tail.

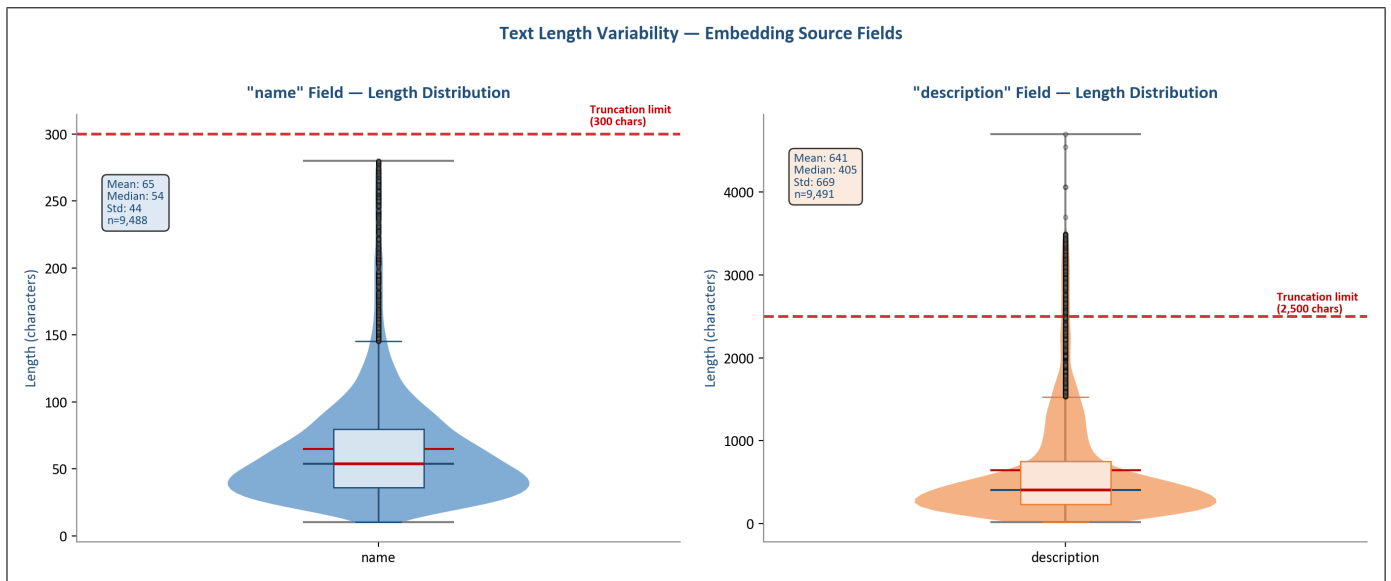


Figure 2.7: Text Length Variability — Violin + Box Plots for name and description Fields

2.5.4 Software Version Format Variability

The software `_version` field presents a particular challenge due to the wide variety of formats used by different teams and tools to identify software builds. This field accepts free-form text, resulting in multiple naming conventions coexisting within the same database.

Table 2.17 below presents the most common format types encountered:

Format Type	Example	Description
Structured (prefix + version)	NA05-26-03-900	Model prefix + year-month-build number
Full build identifier	BMWIDCEVO;2511_i900-25w32.7-1;jdcevo	Platform; build_integration-week.patch; variant
Build + integration	2607_i460-25w45.7-1	Version code + integration step reference
Week-based	25w32.7	Year (25) + calendar week (32) + patch (.7)
Month-year	23-11, 25-05, 26-03	Year-month shorthand

Table 2.17: Versioning Formats for Build Identification

This variability means that software version matching in the metadata boosting algorithm uses substring containment rather than exact matching: if the user's specified version appears anywhere within the stored `software_version` value, a +5% similarity boost is applied.

2.5.5 Data Quality Observations

During the data exploration phase, several quality characteristics were observed that influenced the design of the AI pipeline:

Missing Fields: Not all defects have values for every field. Fields like `software_version`, `lead_model`, and `model_series` may be empty or null for some records. The system handles this case by defaulting to "Unknown" for missing metadata values, ensuring that missing fields do not prevent a defect from being indexed or returned in search results.

Inconsistent Naming Conventions: Defect names often include internal identifiers that are meaningful within BMW’s workflow but do not contribute to semantic understanding. The embedding model naturally handles this by focusing on the semantic content of the text rather than these tokens.

Mixed Technical Detail Levels: Some defects are written by experienced engineers with detailed reproduction steps and log analysis, while others are brief one-liners. The AI system must handle both extremes, which is why semantic embedding is preferred over keyword matching — even brief descriptions contain enough semantic signal for the model to identify similar problems.

Figure 2.8 below provides a combined view of the software version format distribution and the field completeness rates across all key metadata fields, illustrating both the format variability challenge and the data quality landscape.

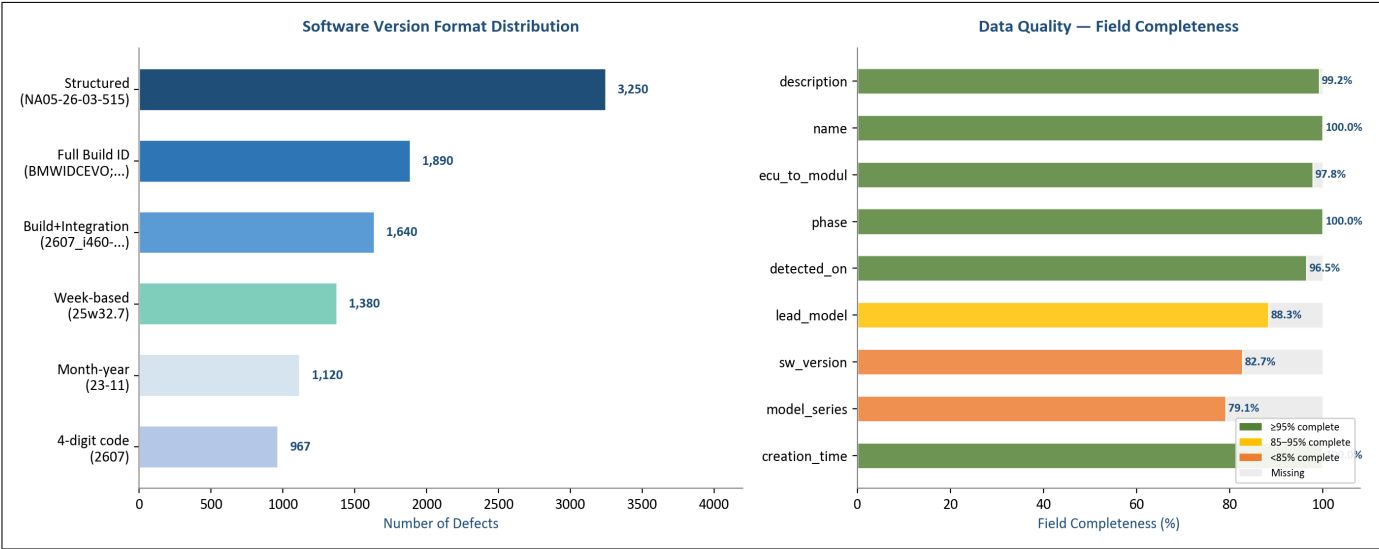


Figure 2.8: Software Version Format Distribution and Data Quality — Field Completeness Rates

2.6 Variable Selection for Semantic Embeddings

From the 270+ available field columns (approximately 130 logical entities), only two fields were selected for generating semantic embeddings: `name` and `description`. This section explains the rationale behind this decision, which is fundamental to the effectiveness of the AI model.

Figure 2.9 below visualizes the variable selection process.

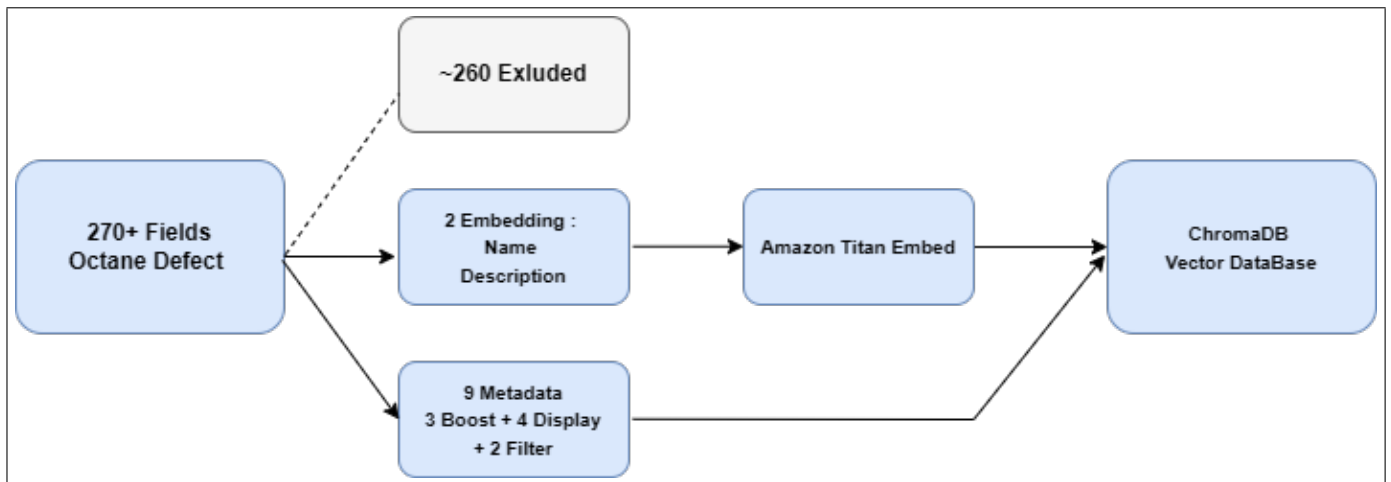


Figure 2.9: Variable Selection

2.6.1 Semantic Embedding Strategy and Feature Selection

The purpose of semantic embeddings is to capture the meaning of text in a high-dimensional vector space, enabling the system to find defects that describe similar problems even when they use completely different words. The critical insight is that the name and description fields are the only fields that contain free-form natural language describing what the problem actually is. All other fields are structured, categorical, or numerical — they describe the context around the problem (which car, which version, who found it, when) but not the problem itself.

Consider the following example: if two testers report the same bug — “the navigation map freezes when switching from 2D to 3D view” — they will write similar descriptions (name + description), but the car model, software version, owner, team, and severity may all be different. The embedding must match on what happened (the map freeze), not on the surrounding context. Including structured fields like severity, priority, or owner name in the embedding text would weaken the semantic signal with irrelevant Variables, reducing the model’s ability to find genuinely similar problems.

This separation of concerns — embeddings for semantic content, metadata for contextual filtering and boosting — is a recognized best practice in vector database design. It allows the system to first find semantically similar defects (through vector search) and then adjust the ranking based on contextual relevance (through metadata boosting), achieving better results than either approach alone.

Exclusion of Non-Semantic Fields from Embeddings

Table 2.18 below summarizes why other defect attributes were excluded from the embedding input and indicates whether they are instead utilized as metadata for filtering or ranking purposes.

Field Category	Reason for Exclusion from Embedding	Used as Metadata?
severity / priority	Describes business impact (High/Low), not the technical nature of the problem. Two identical bugs on different car models may have different severity.	No — not relevant for finding similar problems.
phase	Lifecycle status (New, In Analysis, Concluded) describes process state, not what the defect is. A fixed bug and an open bug can describe the same problem.	Yes — stored as phase_status for display in search results.
owner / author / team	People’s names add noise to embeddings. Different people may report the same bug. Including names would create false associations.	No — organizational info, not useful for similarity.
lead_model	Car model is structured (G70, NA5). Same bug occurs across models. Including model in embedding would prevent cross-model matching.	Yes — used for +7% metadata boost if exact match.
model_series	Related car models sharing platform. Same rationale as lead_model.	Yes — used for +5% metadata boost if platform match.
software_version	Highly variable format. Including versions in embeddings would create version-specific clusters instead of problem-specific ones.	Yes — used for +5% metadata boost if version match.
ecu_to_modul	ECU name (CD, HU) is a single token with no semantic content. All defects already share the same ECU (filtered at ingestion).	Yes — stored as ecu for display.
detected_on	Detection phase describes testing process, not the bug. Same bug can be found in different phases.	Yes — stored as detected_on for display.
creation_time / last_modified	Dates have no semantic meaning for embeddings. Temporal relevance handled through metadata filtering (last 3 months).	Yes — stored for time-based display and filtering.
error_occurrence	Occurrence frequency is useful for debugging but not for matching similar symptoms.	No — not enough impact on similarity.
attachments / comments	Binary files cannot be embedded. Comments often contain workflow discussions rather than problem descriptions.	No — too complex to extract and index.
user_tags	Internal classifications. Including tags would create tag-based clusters rather than semantic ones.	No — internal workflow tags.

Table 2.18: Field Categories Excluded from Embedding and Metadata Usage

The exclusion of these fields follows a consistent principle: only attributes that contribute directly to semantic understanding are included in the embedding, while contextual or structured attributes are handled separately through metadata. This design prevents noise injection into the embedding space and avoids unintended clustering based on non-semantic features such as organizational structure, lifecycle status, or versioning.

2.7 Metadata Selection for the Vector Database

While only name and description are used for generating embeddings, a carefully selected set of additional fields is stored as metadata alongside each vector in the ChromaDB vector database. These metadata fields serve three distinct purposes: display (showing relevant context to users in search results), filtering (narrowing the search space), and boosting (adjusting similarity scores based on contextual matches).

2.7.1 Complete Metadata Schema

Table 2.19 below presents the complete metadata schema, detailing each metadata key stored alongside the embedding vector in ChromaDB.

Metadata Key	Purpose
defect_id	Display: unique identifier shown in results. Deduplication key.
name	Display: defect title shown in search results for quick identification.
description	Display: full description shown when user requests details.
ecu	Display + Filter: shows affected ECU and enables ECU-based filtering at ingestion.
phase_status	Display: lifecycle state (e.g., 06-Concluded, 08-In Verification).
detected_on	Display + Filter: detection phase used during database construction.
lead_model	Boosting: +7% similarity boost when user's model matches defect's lead model.
model_series	Boosting: +5% similarity boost when user's model appears in related series.
software_version	Boosting: +5% similarity boost when user's version matches.
creation_time	Display: shows when defect was created for recency assessment.
last_modified	Display: shows last update date to indicate activity level.

Table 2.19: Metadata Configuration for Defect Display, Filtering, and Boosting

2.7.2 Metadata Boosting Rationale

The metadata boosting mechanism is a key innovation of the search pipeline. Rather than including car model and software version in the embedding text (which would compromise semantic matching), these fields are used after the initial semantic search to adjust scores based on contextual relevance. The boost values were determined experimentally using real defect data. A defect with a semantic similarity of 82% that also shares the same car model and software version would receive a final adjusted score of $82\% + 7\% + 5\% = 94\%$, placing it higher in the results — and rightfully so, as a bug from the same car model and software version is objectively more likely to be the exact same issue the tester is experiencing.

Table 2.20 below details the specific boost conditions and their values:

Boost Condition	Boost Value	Rationale
User's car model = defect's lead_model	+7%	Exact model match strongly indicates the defect is relevant to the user's vehicle (same HW/SW configuration).
User's car model $\in$ defect's model_series	+5%	Platform match: models sharing the same platform often share infotainment software and defects.
User's SW version $\in$ defect's software_version	+5%	Version match increases likelihood the defect exists in the user's build; substring match handles format variability.
No match (default)	+0%	Pure semantic similarity score is used without adjustment.

Table 2.20: Similarity Boost Conditions for Defect Matching

2.8 Data Retrieval and Filtering Strategy

The system retrieves defect data from the MongoDB Octane database using configurable filters that are applied at the database query level, before any data is transferred to the application. This server-side filtering is essential for performance and relevance, as the full defect collection contains defects across all ECUs, all testing phases, and all time periods — far too many records to index in a single vector database instance.

2.8.1 Default Filters

Table 2.21 below presents the default filters applied when building the vector database for the EVV team.

Filter	Default Value	Rationale
ECU	CD (Central Display)	The EVV team primarily tests infotainment functions on the Central Display. Other ECUs have their own teams and defect contexts.
Detection Phase	02-Integration	Integration testing is the primary testing phase for the EVV team.
Time Window	Last 3 months	Recent defects are most relevant. Older defects may reference deprecated software versions or already-fixed issues.

Table 2.21: Default Filter Settings for Defect Ingestion

2.8.2 MongoDB Query Construction

The system constructs MongoDB filter dictionaries dynamically based on the command-line parameters. It creates a query that filters by ECU, detection phase, and date range—all combined using a logical AND :

```
filter = {
  "$and": [
    {"ecu_to_modul.name": "CD"},
    {"detected_on.name": {"$in": ["02-Integration"]}},
    {"creation_time": {
      "$gte": datetime.now()-timedelta(days=90),    # 3 months ago
      "$lte": datetime.now()                        # today
    }}
  ]
}
```

The query can be customized to load defects from different ECUs and phases using command-line parameters. For example, to load Head Unit defects from both Integration and System Test phases over the last 6 months, the filter adjusts accordingly. This flexibility allows the vector database to be rebuilt for different teams and testing contexts without code changes.

2.8.3 Field Projection

To minimize network transfer and processing time, the MongoDB query uses field projection to retrieve only the fields needed for embedding and metadata storage. This represents a 96% reduction in data transfer compared to retrieving all fields. For a batch of 10,000 defects, this optimization significantly reduces the time required to build the vector database and avoids loading unnecessary data such as attachments, comments, ownership histories, and internal workflow fields into memory.

2.9 Conclusion

The data understanding and data preparation phase revealed a vast, bilingual, and richly structured defect database with over 270 raw field columns. Through careful analysis of each field’s nature and potential contribution to the AI model, a precise selection was made.

This disciplined approach to variable selection — using only semantically meaningful text for embeddings while leveraging structured metadata for contextual boosting — is the foundation of the AI model’s effectiveness. By separating the “what happened?” (embeddings) from the “in what context?” (metadata), the system achieves better search accuracy than either pure semantic search or pure keyword/metadata filtering alone. The next chapter (Modeling) will detail how these selected fields flow through the multi-agent architecture and the 8-step query pipeline to produce ranked defect recommendations.

Chapter 3

Chatbot Modeling

3.1 Introduction

This chapter presents the complete AI architecture of the defect analysis chatbot, detailing the multi-agent system design, the rationale behind each model selection, the data ingestion pipeline, and the multi-stage user query pipeline. The modeling phase of the CRISP-DM methodology translates the data understanding from Chapter 2 into a working AI system capable of understanding natural language defect descriptions, performing semantic search across thousands of historical defect records, and ranking results by true relevance using large language model (LLM) reranking.

3.2 Capgemini Generative Engine API

All three AI agents communicate through the Capgemini Generative Engine API, a corporate-managed gateway that provides unified access to multiple LLM providers (AWS Bedrock and Azure OpenAI). This centralized API layer abstracts the differences between providers and handles authentication, rate limiting, and usage tracking. The API uses a unified request format where the model name, provider, prompt, system message, and model parameters (temperature, max tokens, topP) are specified in a structured JSON payload. The system prompt is tailored per agent: Agent 1 receives a BMW automotive expert persona, while Agent 3 receives task-specific instructions for keyword extraction or relevance scoring. A significant technical challenge was the corporate proxy configuration. BMW's network requires all outbound HTTPS traffic to pass through a proxy server with NTLM authentication. The system configures the proxy at the HTTP session level using the requests library's proxy settings, ensuring that API calls to the Capgemini endpoint are routed correctly through the corporate network. The API interface uses the LangChain framework [[LangChain, 2024](#)] for model invocation.

3.3 Multi-Agent Architecture Overview

The AI chatbot employs a multi-agent architecture where three specialized AI models collaborate to process each user interaction. Rather than relying on a single general-purpose model for all tasks, the system delegates specific responsibilities to the most appropriate model, achieving better performance, lower latency, and more cost-effective operation than a monolithic approach.

This architectural decision follows the principle of separation of concerns: each agent excels at its designated task.

Table 3.1 below presents the three agents and their respective roles.

Agent	Model	Provider	Role	Invocation
Agent 1 Conversational AI	GPT-4o (OpenAI)	Azure (via Capgemini)	Intent detection, language detection, defect analysis, conversation, Q&A	Every user message
Agent 2 Embedding Agent	Amazon Titan Embed Text v1	AWS Bedrock (via Capgemini)	Generate 1,536-dim semantic vectors for defect text	Database ingestion + every search query
Agent 3 Task Agent	Amazon Nova Lite v1:0	AWS Bedrock (via Capgemini)	Keyword extraction, LLM reranking, intent confirmation	Defect analysis pipeline only

Table 3.1: Multi-Agent Architecture — Model Roles and Responsibilities

The following diagram 3.1 illustrates how the three agents interact within the system:

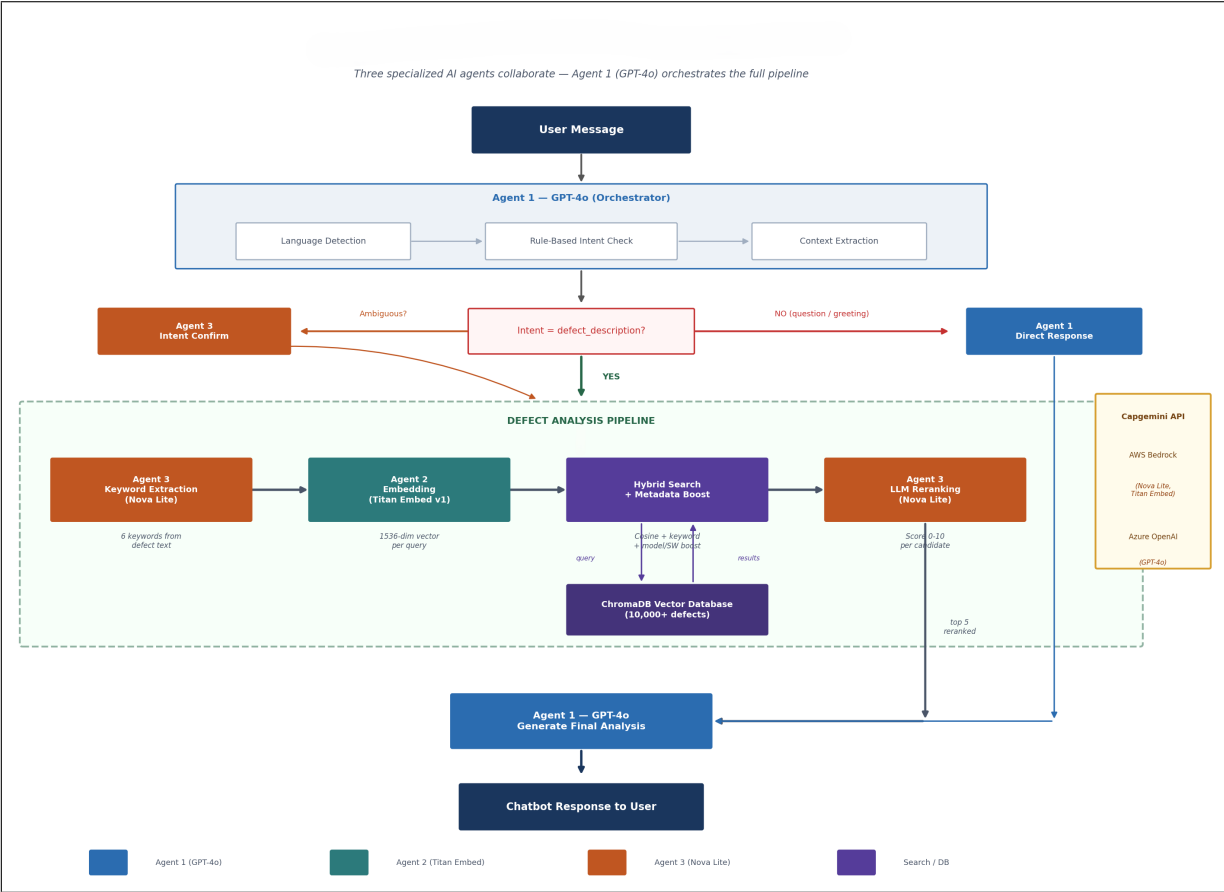


Figure 3.1: Multi-Agent Architecture - BMW Defect Analysis Chatbot

3.4 Agent 1: Conversational AI — GPT-4o

3.4.1 Agent Overview and Model Selection

Agent 1 is the primary conversational interface of the chatbot. It uses OpenAI’s GPT-4o model [OpenAI, 2024], accessed through Azure via the Capgemini API. It is responsible for:

- Detecting the user language (German or English) based on keyword analysis.
- Classifying the user intent (defect description, question, or greeting).
- Generating a structured analysis of defects (what happened, severity, affected area, next steps).
- Answering general questions about BMW systems, testing procedures, and the platform.
- Responding to greetings and casual conversation in the detected language.
- Maintaining conversation context through session-stored history (last 20 messages).

The selection of GPT-4o as the primary conversational agent was the result of a comparative evaluation against several alternative models. The key selection criteria were as follows:

- Multilingual quality (German + English).
- Reasoning capability for defect analysis.
- Instruction following for structured output.
- Availability through the Capgemini Generative Engine API.

Table 3.2 below compares the evaluated models across these criteria.

Model	Provider	Multilingual (DE/EN)	Reasoning	Instruction Following	Latency	Cost	Selected?
GPT-4o	OpenAI (Azure)	Excellent (native DE)	Excellent (SOTA)	Excellent (structured)	Medium (~2–3s)	Medium	✓ YES
GPT-4 Turbo	OpenAI (Azure)	Excellent	Excellent	Excellent	Slow (~4–6s)	High	× Too slow
Claude 3.5 Sonnet	Anthropic [Anthropic, 2024]	Very Good	Excellent	Excellent	Medium (~2–4s)	Medium	× Not on Capgemini API
Amazon Nova Pro	AWS Bedrock	Good	Good	Good	Fast (~1–2s)	Low	× Weaker in German
Amazon Nova Lite	AWS Bedrock	Adequate	Adequate	Good	Very Fast (<1s)	Very Low	× Used for tasks only
Llama 70B 3.1	Meta [AI, 2024]	Adequate	Good	Adequate	Slow (~3–5s)	Medium	× Weaker instruction following

Table 3.2: Conversational Model Selection Comparison

GPT-4o was selected as the conversational agent due to its strong overall performance across key requirements. It provides high-quality multilingual support, particularly for German, which is essential given the dataset composition. Its advanced reasoning capabilities enable accurate analysis

of complex defect reports, including identifying components, assessing severity, and suggesting actions. Additionally, GPT-4o reliably generates well-structured outputs that follow strict formatting instructions, minimizing the need for post-processing. Its response time of around 2–3 seconds ensures a smooth interactive experience, balancing speed and depth of analysis. Finally, its availability through the Capgemini Generative Engine API via Azure made it a compliant and practical choice within the enterprise environment.

3.4.2 Core Conversational Use Cases

Language Detection Algorithm

Before any AI model is invoked, the system performs rule-based language detection using keyword matching and regex analysis. This avoids an LLM call for a task that can be solved deterministically, reducing latency and API costs. The detection algorithm checks for 36 German indicators and 32 English indicators, including common words, articles, and domain-specific terms. Additionally, it applies regex patterns to detect German grammatical articles (der, die, das, dem, den, des) and umlauts (ä, ö, ü, ß). The decision rule is:

```
if has_umlauts OR has_german_articles OR german_count > english_count:
    language = "de"    # German
else:
    language = "en"    # English (default)
```

English is used as the default language when no clear language signal is detected, as it is the primary working language of the international EVV team.

Intent Classification

Agent 1 classifies each user message into one of three intent categories using a two-stage process: first a fast rule-based check, then an LLM confirmation if needed. Table 3.3 below describes the three intent categories, their detection criteria, and the corresponding system response.

Intent	Description	Detection Criteria	System Response
defect_description	User describes a bug or issue	Defect keywords (crash, freeze, Fehler, Absturz, etc.) and technical context	Trigger full defect analysis pipeline
question	User asks for information/help	Question indicators or no defect/greeting match	GPT-4o answers directly
greeting	Greeting, thanks, casual message	Short message with greeting keywords	Friendly response in detected language

Table 3.3: Intent Categories and Response Strategy

The rule-based intent classification uses separate keyword lists for German and English, containing 15+ defect indicators per language (e.g., crash, freeze, not working, broken, Absturz, funktioniert nicht, kaputt, fehlerhaft) and BMW-specific terms (e.g., iDrive, ConnectedDrive, infotainment, navigation, Spotify, Klimaanlage). When the rule-based check is ambiguous, a lightweight LLM call is made to Amazon Nova Lite (temperature: 0.001, max tokens: 10) for intent confirmation, minimizing cost for this auxiliary task.

Context Extraction

When a defect description is detected, the system uses regex-based context extraction to identify the user’s vehicle model and software version from the message text. These values are used later for metadata boosting in the search pipeline.

Table 3.4 below lists the regex patterns used for each context field.

Context Field	Regex Pattern	Examples Matched
Car Model	<code>\b(NA\d G\d{2} I\d{2} U\d{2})\b</code>	NA5, NA6, G20, G45, G70, I20, U10
SW Version (1)	<code>BMWIDCEVO[;\s] [\w-;\.]+</code>	BMWIDCEVO;2511_i510-25w32.7-1;idcevo
SW Version (2)	<code>BMW\s*IDCEVO[;\s] [\w-;\.]+</code>	BMW IDCEVO;2607_i440-25w35.7-1;idcevo
SW Version (3)	<code>\b\d{4}_i\d{3}\b</code>	2607_i460
SW Version (4)	<code>\b\d{2}w\d{2}[\.\-]\d[\.\-]?d*\b</code>	25w32.7, 25w32.7-1
SW Version (5)	<code>\b\d{2}[\-\.]\d{2}\b</code>	23-11, 25-05, 26-03
SW Version (6)	<code>(?<!\d)\d{4}(?!\\d)</code>	2607, 2603

Table 3.4: Regex Patterns for Context Extraction

The six software version patterns are applied in order of specificity (most specific first), and the first match is used. This ensures that a full build identifier like "BMWIDCEVO;2511_i510-25w32.7-1;idcevo" is captured as a single entity rather than being partially matched by a more generic pattern.

3.5 Agent 2: Embedding Agent — Amazon Titan Embed Text v1

3.5.1 Agent Overview and Model Selection

Agent 2 is the semantic backbone of the system. It uses Amazon Titan Embed Text v1 [Amazon, 2024b], a multilingual embedding model from AWS Bedrock, to convert free-form text (defect names and descriptions) into 1,536-dimensional numerical vectors that capture the semantic meaning of the text. These vectors are stored in ChromaDB and used for similarity search: defects with similar descriptions will have vectors that are close together in the 1,536-dimensional space, enabling the system to find related problems even when they use completely different words.

The embedding model is arguably the most critical model choice in the system, as it determines the quality of the semantic search — the foundation upon which all subsequent stages (keyword boost, metadata boost, LLM reranking) are built.

Table 3.5 below presents the comparative evaluation of available embedding models.

Model	Provider	Dimensions	Multilingual	Max Tokens	Batch	Cost	Selected?
Titan Embed Text v1	AWS Bedrock	1,536	Yes (100+ langs)	8,192	Yes	Low (\$0.0001 / 1K tokens)	✓ YES
text-embedding-3-large	OpenAI	3,072	Very Good	8,191	Yes	Medium	× Larger DB / slower search
text-embedding-3-small	OpenAI	1,536	Good	8,191	Yes	Low	× Not on Capgemini API
Cohere Embed v3	Cohere [Cohere, 2024]	1,024	Excellent	512	Yes	Low	× Not on Capgemini API
Voyage AI v2	Voyage AI	1,024	Good	4,096	Yes	Medium	× Not on Capgemini API
all-MiniLM-L6-v2	Sentence Transformers [Reimers, 2019]	384	Limited	256	Local	Free	× Too weak for use case

Table 3.5: Embedding Model Evaluation and Selection

Amazon Titan Embed Text v1 was chosen because it offers a strong balance between performance, cost, and practicality for this project. It handles both German and English very well, which is important since many defect reports are written in German, and it can still match them correctly with similar issues written in English.

The model also provides a good vector size (1,536 dimensions), which is detailed enough to capture meaning accurately without making storage and search too heavy. Its large input limit allows it to process long defect descriptions without losing important information, unlike other models that require heavy truncation.

In addition, Titan supports batch processing, which makes it efficient when generating embeddings for thousands of defects. It is also very cost-effective, making it possible to update the database regularly without concern. Finally, it is available through the company’s approved API, which makes it easy to integrate and maintain within the existing system.

3.5.2 Embedding Generation Process

The embedding for each defect is created by combining the **name** and **description** fields into a single text. Before combining them, both fields are shortened to stay within the token limits of the Amazon Titan Embed API: the name is limited to 300 characters, and the description to 2,500 characters. The final embedding text is simply the two fields joined with a space.

```
# Embedding text preparation
name = defect.get("name", "")[:300] # Truncate name to 300 chars
description = defect.get("description", "")[:2500] # Truncate desc to 2500 chars
embedding_text = f"{name} {description}" # ~2,800 chars max

# API call
payload = {
    "model": "amazon.titan-embed-text-v1",
    "input": embedding_text
}
response = session.post(EMBEDDING_API_URL, json=payload)
```

Returns: [0.0234, -0.0156, 0.0089, ..., -0.0312] # 1,536 floats

This text is then sent to the Amazon Titan Embed model, which returns a vector of 1,536 numerical values. This vector represents the meaning of the defect in a mathematical form, allowing the system to compare defects using similarity measures such as cosine similarity.

The truncation limits were not chosen randomly. They were adjusted after encountering errors from the Titan API when the input text exceeded the allowed token size. With these limits, the input stays safely within the 8,192-token maximum while preserving the most important information. To make the system more robust, error handling was also added: if the API request fails due to temporary issues (such as HTTP errors 403, 407, 429, or server errors), the system retries the request using exponential backoff (0.5s, 1s, 2s). If all retries fail, a default zero vector of length 1,536 is returned as a fallback.

3.6 Agent 3: Task Agent — Amazon Nova Lite v1:0

3.6.1 Agent Overview and Model Selection

Agent 3 uses Amazon Nova Lite v1:0 [Amazon, 2024a], a lightweight, fast, and cost-effective model from the AWS Bedrock Nova family. It handles the lightweight, structured tasks in the pipeline that require language understanding but not the full reasoning power of GPT-4o:

- Extracting exactly N keywords/key phrases from defect descriptions for semantic search
- Confirming intent classification when the rule-based check is ambiguous
- Scoring candidate defects (0–10) during LLM reranking based on relevance to the user’s query

The task agent has different requirements than the conversational agent: it needs to be fast, cost-effective, and reliable at producing structured output (comma-separated keyword lists, single-word intent labels, JSON score arrays). Table 3.6 below compares the task agent candidates.

Model	Provider	Latency	Cost	Structured Output	Key Limitation	Selected?
Amazon Nova Lite v1:0	AWS Bedrock	<1s	Very Low	Excellent	Weaker at creative German text	✓ YES
GPT-4o	OpenAI (Azure)	~2–3s	Medium	Excellent	Overkill for simple tasks	× Too slow and costly for tasks
Amazon Nova Micro	AWS Bedrock	<0.5s	Extremely Low	Good	Too limited for reranking logic	× Insufficient reasoning
Amazon Nova Pro	AWS Bedrock	1–2s	Low	Very Good	Slightly slower, no major benefit	× more expensive, no benefit

Table 3.6: Task Agent Model Comparison

Amazon Nova Lite was chosen for task-related operations mainly because it is fast, low-cost, and reliable for simple tasks. It responds in under a second, which keeps the system smooth since it is called multiple times during a single defect analysis. It is also much cheaper than GPT-4o, making it more suitable for frequent use at scale. Even though it is a lightweight model, it still handles structured outputs well, such as keyword lists and scoring formats, especially with strict instructions. While it is not as strong in reasoning as GPT-4o, it is good enough to compare defect descriptions and assign relevance scores, making it a practical and efficient choice for these supporting tasks.

3.6.2 Core Task Use Cases

Keyword Extraction

When a defect description is detected, Agent 3 extracts exactly 6 keywords or key phrases from the text. These keywords serve as the input for Stage 2 of the search pipeline (keyword-only search) and help capture specific terms that the embedding model might not distinguish (e.g., specific car models, software versions, feature names). The extraction is guided by a detailed prompt that specifies a priority order for keyword selection, as shown in Table 3.7 below:

Priority	Category	Example Keywords
1 (Highest)	Problem Description	“location name in English when Mandarin ”, “map freezes on 3D switch”
2	Affected Application/Feature	Spotify, Navigation, Climate Control, iDrive
3	Specific Error Type	crash, freeze, black screen, Absturz, Einfrieren
4	Car Model/Variant	NA5, G70, I20
5	Software Version	23-11, 25w32.7, 2607
6 (Lowest)	Additional Context	after software update, during navigation

Table 3.7: Keyword Extraction Priority Order

The prompt explicitly excludes generic terms like “defect”, “BMW”, “model”, “software”, “problem”, “Fehler” to prevent diluting the search with words that would match virtually every defect in the database. The extraction uses separate bilingual prompts (German/English) to match the detected language, ensuring that German defect descriptions produce German keywords that align with the German text in the database.

Intent Confirmation

When a user sends a message, the system classifies it into one of three categories: defect_description, question, or greeting. This is first handled by a rule-based classifier using keywords and simple text rules (see Section 3.2.4). If the system cannot clearly decide—such as when the message is technical but lacks clear defect keywords—Agent 3 is used as a fallback. The rule-based logic starts by checking for greetings (e.g., “hello”, “hallo”, “danke”). If not a greeting, it looks for defect indicators (like “crash”, “freeze”, “Absturz”) combined with BMW-related terms, or checks if the message is long and technical. If none of these conditions are met, the message is considered ambiguous and passed to Agent 3 for confirmation.

Table 3.8 below describes the decision flow for intent classification, showing when the rule-based engine handles classification autonomously and when Agent 3 is invoked.

Condition	Rule-Based Result	Agent 3 Invoked?	Example
Short message with greeting keywords	greeting	No	“Hello, how are you?”
Defect keyword + BMW term present	defect_description	No	“Navigation crashes on NA5 with SW 25-11”
Defect keyword + long technical text	defect_description	No	“After update the media menu is empty and Spotify won’t load...”
No defect keyword, no greeting match	Ambiguous	Yes	“The screen shows a yellow tint sometimes”
Technical text without explicit defect terms	Ambiguous	Yes	“Wireless charging notification appears every time I start”

Table 3.8: Intent Classification Decision Flow

When Agent 3 is used, it receives a simple prompt in the user’s language and must return one of the three categories. The settings are tuned to be very precise (temperature = 0.001, max tokens = 10), so the output is always a single word. If the result is unexpected, the system defaults to question as a safe option. This approach keeps the system fast and cost-efficient: most messages (around 70–80%) are handled instantly by the rule-based system with no cost, while only unclear cases are sent to Agent 3.

LLM Reranking (Relevance Scoring)

The final and most important task of Agent 3 is reranking the retrieved defects by giving each one a score from 0 to 10 based on how relevant it really is to the user’s query. This is needed because earlier steps like embeddings, keyword search, and metadata can find similar-looking defects, but they cannot fully understand whether the problems are actually the same. For example, two defects may involve the same system but describe different issues, and only a language model can clearly detect that difference.

To do this, the system takes the top 15 candidate defects and sends them to the model along with key information such as the name, a shortened description, ECU, car model, software version, and the user’s original query. The model follows clear rules when scoring, especially ensuring that defects with different symptoms do not receive high scores, even if they share the same component. Small bonuses are added if the car model or software version matches the user’s context.

The model returns a list of scores in a structured format, which the system carefully parses even if the output is slightly inconsistent. These scores are then combined with the original similarity scores, with more weight given to the model’s judgment. Very low-scored results are removed, and the remaining defects are sorted, with the top 5 returned to the user as the most relevant results.

3.7 ChromaDB Vector Database

The vector database stores the semantic embeddings generated by Agent 2 and enables fast similarity search. ChromaDB was selected after evaluating several vector database solutions, as compared in

Table 3.9 below:

Database	Type	Setup	Persistence	Metadata Filtering	Cost	Selected?
ChromaDB	Embedded	Minimal	Local disk	Full support	Free	✓ YES
Pinecone	Cloud managed [Systems, 2024]	API setup	Cloud	Full support	Paid	× Cloud dependency, cost at scale
Weaviate	Self-hosted / cloud [Weaviate, 2024]	Docker/K8s	Configurable	GraphQL queries	Free (self-hosted) or paid (cloud)	× Heavy infra requirements
Milvus	Self-hosted [Milvus, 2024]	Docker/K8s	Configurable	Full support	Free	× Too complex for project scale
FAISS	Library [Johnson, 2021]	Minimal	Manual save/load	No native support	Free	× No metadata filtering, no persistence
Qdrant	Self-hosted / cloud	Docker	Configurable	Full support	Free (self-hosted) or paid (cloud)	× Docker dependency

Table 3.9: Vector Database Selection Comparison

ChromaDB was chosen because it is simple, practical, and well-suited to the project’s needs. It runs directly inside the application without requiring any external setup, which makes it easy to install and move across environments. It also allows storing useful metadata with each vector and querying it efficiently, which is important for filtering and ranking results. Since the system handles around 10,000 vectors, its performance is fast and reliable without needing more complex database solutions. Finally, being fully Python-based, it integrates smoothly with the backend, avoiding unnecessary complexity.

3.8 Data Ingestion Pipeline

The data ingestion pipeline transforms raw defect documents from MongoDB into searchable vector embeddings stored in ChromaDB.

Figure 3.2 below illustrates the complete data ingestion pipeline from MongoDB to ChromaDB.

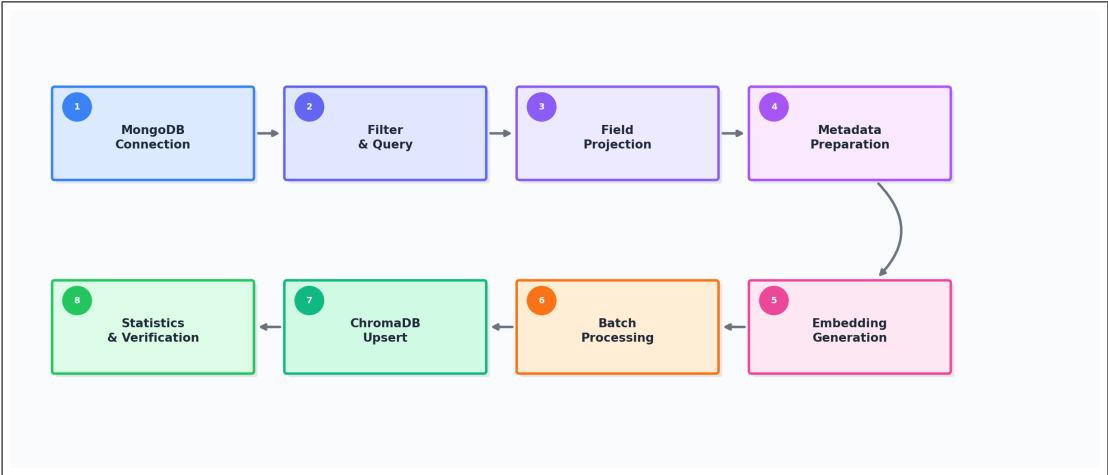


Figure 3.2: Data Ingestion Pipeline — From MongoDB to ChromaDB

- Step 1 — MongoDB Connection:** Establish connection to the octane-prod database on MongoDB Atlas using the SRV connection string with read-only credentials.
- Step 2 — Filter & Query:** Apply configurable filters (ECU, detection phase, time window) to the defect collection. Default filters: `ecu_to_modul.name = "CD"`, `detected_on.name = "02-Integration"`, `creation_time` within last 3 months.
- Step 3 — Field Projection:** Retrieve only the 11 required fields from each defect document, reducing data transfer by 96% compared to fetching all 270+ columns.
- Step 4 — Metadata Preparation:** Extract and transform fields into the ChromaDB metadata schema. Handle missing values with "Unknown" defaults. Convert `model_series` array to comma-separated string. Convert timestamps to string format.
- Step 5 — Embedding Generation:** Concatenate name (truncated to 300 chars) and description (truncated to 2,500 chars) into embedding text. Send to Amazon Titan Embed API for 1,536-dimensional vector generation.
- Step 6 — Batch Processing:** Process defects in batches of 50 to balance API throughput with memory usage. Each batch generates embeddings for up to 50 defects in a single API call.
- Step 7 — ChromaDB Upsert:** Insert vectors and metadata into the `bmw_defects` collection using ID-based upsert, which prevents duplicates when the pipeline is run multiple times.
- Step 8 — Statistics & Verification:** Track and report statistics: total defects processed, successful embeddings, failed embeddings, skipped duplicates. Verify the collection count matches expectations.

The pipeline supports four operational modes via command-line flags, as listed in Table 3.10:

Flag	Purpose	Use Case
<code>-initial-load</code>	First-time setup	New database creation or full rebuild
<code>-update</code>	Incremental update	Weekly or regular maintenance
<code>-rebuild</code>	Recreate from scratch	Schema changes or data corruption recovery
<code>-info</code>	Show DB statistics	Verification and debugging
<code>-limit N</code>	Process first N defects	Testing and development
<code>-ecu X</code>	ECU module filter	Loading defects for specific ECU
<code>-phase X</code>	Detection phase filter	Loading defects for specific test phase
<code>-months N</code>	Time-window filter	Recency adjustment

Table 3.10: Ingestion Pipeline Command-Line Flags

3.9 User Query Pipeline

When a user sends a message to the chatbot, it passes through an 8-step pipeline that transforms the raw text into a ranked list of similar defects with AI-generated analysis. This pipeline orchestrates all three agents and combines rule-based processing with semantic search and LLM reasoning. Figure 3.3 below presents the complete 8-step user query pipeline.

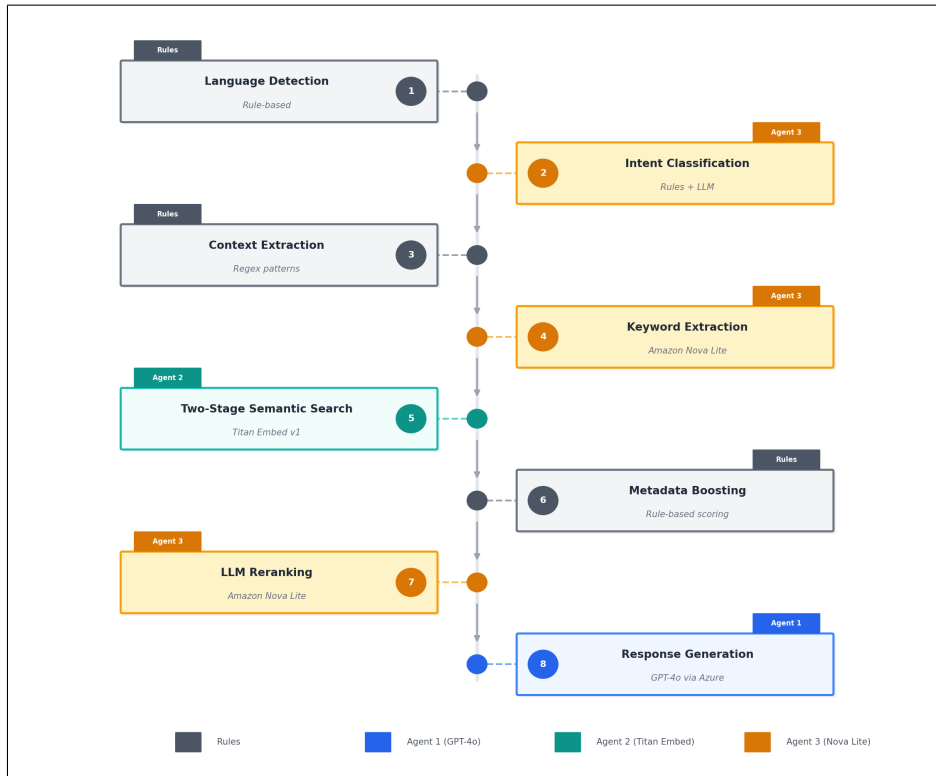


Figure 3.3: User Query Pipeline — 8-Step Processing Flow

3.9.1 Step 1: Language Detection

The pipeline begins with rule-based language detection (described in 3.4.2). This step determines whether all subsequent prompts and responses should be in German or English. The detection is performed without an LLM call, ensuring zero added latency. The detected language is stored in the session and propagated to all downstream steps.

3.9.2 Step 2: Intent Classification

The intent is classified using the two-stage approach described in 3.4.2. If the intent is "greeting" or "question", the pipeline branches to Agent 1 (GPT-4o) for a direct response. If the intent is "defect_description", the pipeline continues to Step 3. This early exit for non-defect messages avoids unnecessary search operations and LLM calls, significantly reducing latency and cost for casual interactions.

3.9.3 Step 3: Context Extraction

When a defect description is identified, the system uses regex-based context extraction (described in 3.4.2) to identify the user's car model and software version. If either is missing, the pending defect flow is triggered (Section 3.10), asking the user to provide the missing information before proceeding with the analysis. This ensures the metadata boosting step has context to work with, improving result relevance.

3.9.4 Step 4: Keyword Extraction

Agent 3 (Amazon Nova Lite) extracts exactly 6 keywords from the defect description (described in 3.6.2). These keywords are used in Stage 2 of the search and also displayed to the user as part of the analysis summary. The extraction uses language-specific prompts with a priority system that emphasizes the core problem/symptom over contextual details like car model or software version.

3.9.5 Step 5: Two-Stage Semantic Search

The search combines two complementary strategies to maximize recall:

Stage 1: Hybrid Full-Text Search (15 candidates)

The user’s full message is converted to a 1,536-dimensional embedding by Agent 2, and ChromaDB returns the 15 most similar defects based on L2 (Euclidean) distance. The raw L2 distances are converted to similarity percentages using the recalibrated mapping shown in Table 3.11 below:

L2 Distance Range	Similarity Range	Interpretation
< 50	95–100%	Near-identical text (likely duplicate)
50–100	90–95%	Very high similarity
100–150	85–90%	High similarity
150–200	75–85%	Moderate-high similarity
200–240	65–75%	Moderate similarity
240–280	50–65%	Low-moderate similarity
280–350	30–50%	Low similarity
> 350	0–30%	Very low similarity

Table 3.11: L2 Distance to Similarity Mapping

After the initial distance-to-similarity conversion, a keyword boost of +15% is applied for each important keyword found in the defect’s name or description.

Table 3.12 below lists the keyword categories that trigger the boost:

Keyword Category	Examples	Boost
Car Models	na5, na6, g20, g45, g70, g05, g30, g15, g21	+15% per match
Software Versions	Pattern: \b(\d{2}-\d{2})\b	+15% per match
Technical Terms	swo, cd, hud, navi, navigation, spotify, idrive, connectedrive	+15% per match
Symptoms	crash, freeze, black/white/yellow screen, not working, error	+15% per match

Table 3.12: Keyword-Based Similarity Boost Categories

The keyword boost compensates for cases where the semantic embedding captures the general topic

but misses specific technical identifiers. For example, a query about "NA5 navigation crash" should rank an NA5-specific defect higher than a G70 defect with similar symptoms, even if the embeddings are equidistant.

Stage 2: Keyword-Only Search (10 candidates)

The 6 keywords extracted by Agent 3 are concatenated into a single text string, embedded by Agent 2, and used for a second ChromaDB search with a higher similarity threshold (minimum 48%). This catches defects that use different phrasing but share specific keywords — for example, a query saying "Spotify app crashes" might match a defect titled "Music streaming application force-closes" in Stage 1 by embedding similarity, but Stage 2 would additionally find defects that literally contain "Spotify" and "crash" in their text.

Merge & Deduplication

Results from both stages are merged by defect ID. If the same defect appears in both stages, the highest similarity score is kept. The merged list is sorted by descending similarity, producing up to 25 unique candidate defects for the next step.

3.9.6 Step 6: Metadata Boosting

After semantic search, the similarity scores of all candidates are adjusted based on metadata matches with the user’s vehicle context (extracted in Step 3), as detailed in Table 3.13 below:

Condition	Boost	Rationale
User car model = defect lead_model	+7%	Exact hardware/software context match
User car model ∈ defect model_series	+5%	Platform match
User SW version ⊆ defect software_version	+5%	Version context match
No match	+0%	Pure semantic score

Table 3.13: Metadata Boosting Rules

The boosted similarity score is capped at 1.0 (100%) to prevent artificial inflation. A defect with a semantic similarity of 82% from the same car model (NA5) and software version (25w32.7) would receive: $82\% + 7\% + 5\% = 94\%$, ranking it above a semantically identical defect from a different car model ($82\% + 0\% = 82\%$).

3.9.7 Step 7: LLM Reranking

The top candidates (after metadata boosting) are sent to Agent 3 (Amazon Nova Lite) for LLM-based reranking. This is the most critical quality step: while semantic search finds defects with similar text, and metadata boosting adjusts for vehicle context, only an LLM can truly assess whether two defects describe the same underlying problem.

Agent 3 receives the user’s defect description and the candidate defects, and scores each candidate on a 0–10 scale using the rubric described in Table 3.14 below:

Score Range	Meaning	Example
10	Same problem, same component, same symptoms	Both describe “Navigation crashes when switching from 2D to 3D view”
7–9	Very similar problem	Both describe navigation crashes in different scenarios
4–6	Somewhat related	“Navigation crash” vs “Navigation wrong location”
1–3	Weakly related	“Navigation crash” vs “Media player crash”
0	Unrelated	“Navigation crash” vs “Seat heater malfunction”

Table 3.14: LLM Reranking Relevance Scale

Additionally, the LLM applies boosting rules within its scoring: +1 for matching car model, +1 for matching software version (capped at 10). The LLM returns a JSON array of scores, which is blended with the original retrieval similarity:

```
# Score blending formula
blended_similarity = 0.25 * original_similarity + 0.75 * (llm_score / 10.0)

# Weight rationale:
#   75% LLM judgment: LLM understands problem semantics and context
#   25% retrieval score: preserves embedding-based similarity as a baseline

# Filtering: candidates with llm_relevance < 2 are removed entirely
```

The 75/25 weight split was determined empirically. A 100% LLM weight would discard the embedding signal entirely, which could hurt performance when the LLM misinterprets a technical defect. A 50/50 split gave too much weight to the retrieval score, causing semantically distant but LLM-confirmed relevant defects to rank lower than expected. The 75/25 split provides the best balance: the LLM’s contextual judgment dominates, but the embedding similarity provides a safety net.

3.9.8 Step 8: Response Generation

Agent 1 (GPT-4o) generates a structured defect analysis based on the user’s description and the reranked results. The analysis includes:

- What: A clear summary of what the reported problem is
- Severity Assessment: Estimated impact level (Critical/High/Medium/Low) based on the defect description
- Affected Area: The specific component, feature, or system affected
- Recommended Next Steps: Suggested actions for the tester

The top 5 reranked defects are formatted with full metadata display (defect ID, similarity percentage, name, ECU, car model, software version, lifecycle phase, detection phase, creation and modification dates). The entire response is formatted in the user’s detected language (German or English) with emoji-enhanced visual markers for readability.

3.10 Pending Defect Flow

When the system detects a defect description but cannot extract either the car model or software version from the text, it enters the pending defect flow. This conversational mechanism pauses the analysis to ask the user for the missing information, improving the accuracy of metadata boosting in the subsequent search.

Figure 3.4 below illustrates the complete pending defect flow sequence.

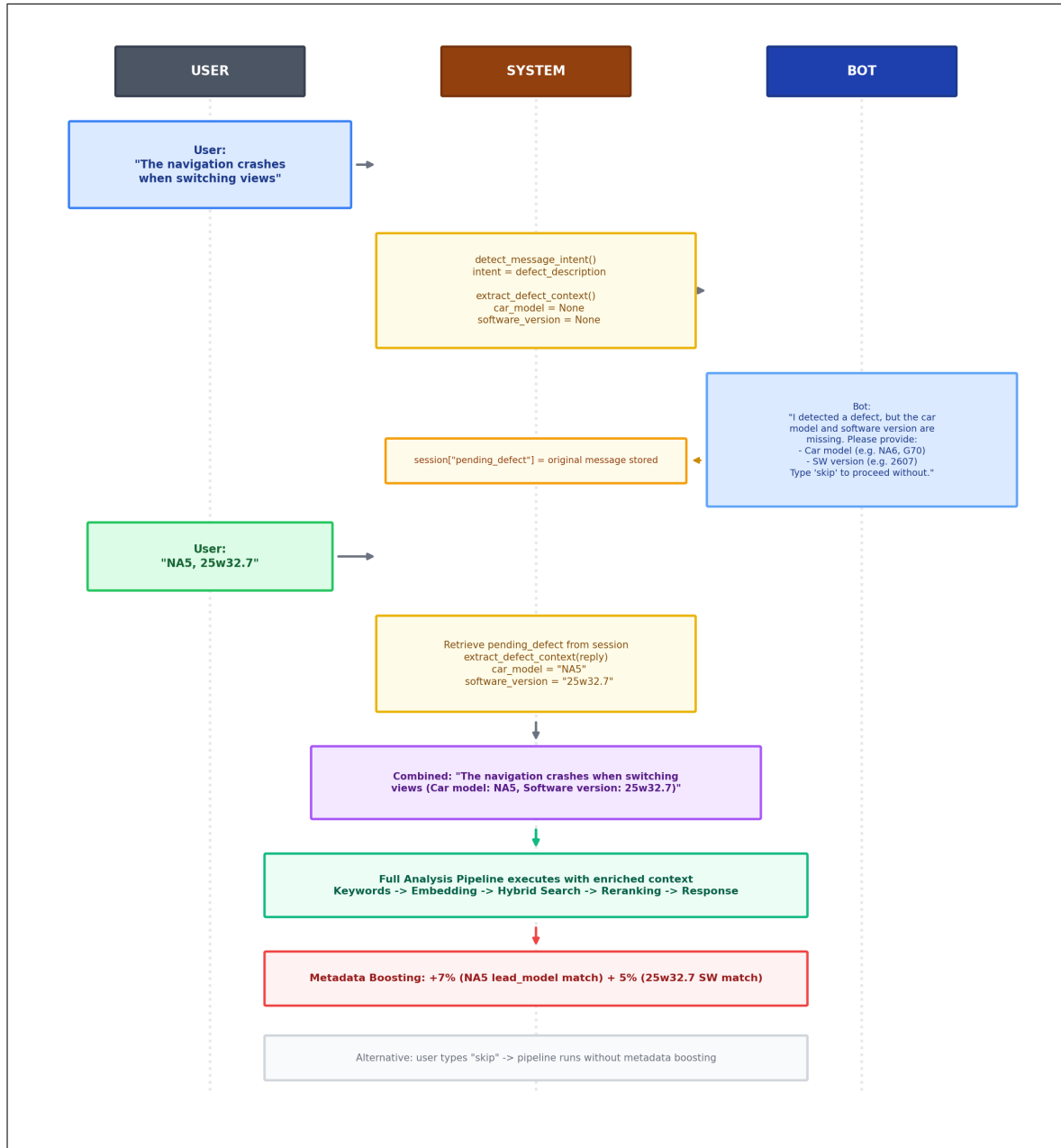


Figure 3.4: Pending Defect Flow — Context Collection Sequence

The pending defect is stored in the Django session (`request.session["pending_defect"]`) and retrieved when the user responds. The user can also type "skip" to proceed with the analysis without providing a car model or software version, in which case the search runs without metadata boosting. This design

respects the user’s workflow: sometimes a tester wants a quick search without specifying context, while other times they want to narrow results to their specific vehicle configuration.

3.11 Session and Conversation Management

The chatbot maintains conversation context through Django’s session framework. Each user’s conversation history is stored server-side and includes the last 20 messages (user + assistant pairs). This enables the chatbot to reference previous interactions, provide follow-up analysis, and maintain a coherent conversation flow.

Table 3.15 below lists the session variables used by the chatbot.

Session Variable	Type	Purpose
conversation_history	List of dicts	Last 20 messages {role, content}; passed to GPT-4o for context
pending_defect	String / None	Original defect description while awaiting missing model/SW version
detected_language	String (en/de)	Most recent detected language for response formatting

Table 3.15: Session Variables for Conversation Management

3.12 Conclusion

This chapter presented the complete modeling architecture of the AI defect analysis chatbot. The system employs a multi-agent design with three selected models: GPT-4o for multilingual conversation and defect analysis, Amazon Titan Embed for semantic embeddings, and Amazon Nova Lite for fast task operations such as keyword extraction and LLM reranking.

The 8-step query pipeline combines rule-based processing with AI-powered semantic search and reranking to produce highly relevant defect recommendations. The pending defect flow ensures that missing context is collected when necessary, while the bilingual architecture enables natural German and English interactions throughout the full pipeline.

The next chapter (Evaluation) will assess the performance and effectiveness of this modeling approach through quantitative testing and qualitative analysis.

Chapter 4

Evaluation

4.1 Introduction

This chapter presents the evaluation and validation of both the remote testing platform and the AI-powered defect analysis chatbot. Following the CRISP-DM methodology, the evaluation phase assesses whether the developed models and systems meet the objectives defined in Chapter 1, identifies the strengths and weaknesses of the approach, and documents the challenges encountered during development. The chapter covers remote testing validation (hardware and network tests), vector database build results, search quality assessment, LLM reranking effectiveness, chatbot functional testing, and a critical discussion of limitations and challenges.

4.2 Remote Testing Platform Validation

The remote testing platform was validated through a series of structured tests designed to verify the reliability, latency, and usability of the system under real-world conditions. Testing was performed in two distinct environments: the BMW office in Munich (where the test setups are physically located) and a home-office approximately 50 km away, connected to the BMW network via VPN.

4.2.1 Sound Cable and Voice Recognition Testing

The voice recognition testing capability required the construction of a custom sound cable to route audio from the tester’s workstation to the vehicle’s Head Unit microphone input. This hardware integration enables two voice testing modes, as described in Table 4.1 below:

Mode	Input Source	Signal Path	Use Case
Live Speech	Tester’s microphone (real-time)	Microphone → SSH tunnel → Sound cable → Head Unit	Testing natural voice commands with accent, pronunciation variations
Dynamic Speech	Text-to-speech (generated via website)	Website TTS → SSH tunnel → Sound cable → Head Unit	Automated/repeatable voice command testing with consistent input

Table 4.1: Speech Testing Modes

Testing confirmed that voice commands transmitted through the sound cable were recognized by the Head Unit’s voice recognition system with comparable accuracy to direct microphone input. Both Live Speech and Dynamic Speech modes functioned reliably over SSH connections, with the audio quality being sufficient for the voice recognition engine to process commands correctly. Minor latency was observed in Live Speech mode due to SSH tunneling overhead, but this did not affect the recognition accuracy.

4.2.2 WebScrcpy Latency Testing

WebScrcpy [NetMind, 2024] provides real-time screen mirroring and remote control of the Head Unit’s display through the web browser. Latency testing was performed in two scenarios to evaluate the system’s responsiveness across different network conditions ,as described in Table 4.2 below:

Test Scenario	Distance from Test Setup	Network Connec- tion	Observed La- tency	User Experience
Office (on-site)	< 1 km (same building)	BMW internal LAN (Ethernet)	< 100 ms	Excellent — real-time feel, no perceivable delay
Home-Office (re- mote)	~50 km (via VPN)	BMW VPN over home broadband	200–400 ms	Good — slight delay vis- ible but fully usable for testing tasks

Table 4.2: Network Latency Test Scenarios

The latency results demonstrate that the system is fully functional for remote testing at distances of at least 50 km from the physical test setup. While the home-office scenario introduces a perceptible delay (200–400 ms), this latency is acceptable for the testing tasks performed by the EVV team, which involve observing screen behavior, verifying UI elements, and confirming voice command responses. For comparison, standard video conferencing tools operate at similar latency levels and are widely considered acceptable for professional use.

4.2.3 User Acceptance Testing

User acceptance testing was conducted with Capgemini Engineering collaborators located across multiple countries and time zones. Testers were given access to the web platform and asked to perform typical testing tasks (logging in, reserving a test setup via the calendar, connecting to a Head Unit via WebScrcpy, executing voice commands, and using the AI chatbot for defect analysis). The feed- back collected from these sessions was consistently positive:

Ease of Use: Users found the web interface intuitive and easy to navigate. The sidebar navigation, calendar-based reservation system, and integrated WebScrcpy viewer were highlighted as particularly well-designed elements.

Reliability: SSH connections remained stable throughout extended testing sessions. No unexpected disconnections were reported during normal operation.

Added Value: Testers emphasized that the ability to test remotely from any location significantly reduced the need for travel to the BMW offices in Germany, saving time and operational costs.

AI Chatbot: The chatbot’s ability to find similar defects from the Octane database was recognized as a useful tool for avoiding duplicate defect reports. Testers found the similarity percentages and metadata display helpful for assessing whether their problem had already been reported.

The interest and enthusiasm demonstrated by BMW’s client representatives in demonstration sessions and follow-up meetings were clear indicators of the project’s added value. The regular feedback sessions ensured that the developed functionalities aligned with the client’s expectations and operational needs.

4.2.4 Current Status: Manual Team Testing

Following the successful completion of the initial evaluation phase, the system has been handed over to the BMW manual testing team for extended operational testing. This team is currently using the platform in their day-to-day testing workflows, providing real-world validation beyond the controlled evaluation environment. Their testing covers daily use scenarios, edge cases, and integration with existing BMW testing processes. Feedback from this operational testing phase is being collected and will inform future improvements and optimizations to the platform. This handover to the manual team represents a significant milestone, as it demonstrates that the system has reached a maturity level sufficient for production-like use by the target user group.

4.3 Vector Database Build Results

The vector database build process was evaluated through multiple iterations, starting with small test batches and scaling up to the full production dataset. This incremental approach allowed the team to identify and resolve issues before committing to the computationally expensive full build.

4.3.1 Initial Test Build (100 Defects)

The first test build was executed with the `--limit 100` parameter to validate the end-to-end ingestion pipeline with a manageable number of defects. Table 4.3 below shows the results, which revealed a significant issue with the embedding API:

Metric	Value	Percentage
Total defects loaded from MongoDB	100	100%
Successfully embedded and stored	72	72%
Failed (403 embedding API errors)	28	28%
Skipped	0	0%

Table 4.3: Embedding Process Metrics

The 28% failure rate was unacceptable for a production system. Investigation revealed that the 403 errors were caused by defect descriptions that exceeded the Amazon Titan Embed API’s input token limit. Some defect descriptions in the Octane database contain lengthy, multi-paragraph text with reproduction steps, log excerpts, and expected vs. actual behavior comparisons that, when combined with the defect name, exceeded the API’s maximum input size.

4.3.2 Root Cause Analysis: 403 Embedding Errors

The 403 errors from the Capgemini embedding API were traced to two primary causes:

Input Token Overflow: The Amazon Titan Embed Text v1 model has a maximum input size of 8,192 tokens. While most defect descriptions fit within this limit, some contained extremely long reproduction steps, stack traces, or log outputs that pushed the combined text (name + description) beyond the maximum. The Capgemini API returned a 403 Forbidden response for these oversized inputs rather than a more descriptive error code.

Rate Limiting: A secondary cause of 403 errors was API rate limiting. When processing defects in rapid succession (batch mode), the system occasionally exceeded the API's request rate threshold, triggering temporary 403 responses. This was exacerbated by the lack of backoff between batch requests.

4.3.3 Fix Implementation: Text Truncation and Retry Logic

Two complementary fixes were implemented to resolve the 403 errors:

Fix 1: Text Truncation

Strict character limits were applied to the name and description fields before they are sent to the embedding API.

Fix 2: Exponential Backoff Retry Logic

A retry mechanism with exponential backoff was added to handle transient API errors, including rate-limiting 403 responses and temporary server issues.

4.3.4 Production Build Results (10,000+ Defects)

After implementing both fixes, the vector database was rebuilt with the full production dataset. The results confirmed that the truncation and retry logic effectively resolved the 403 error problem.

Figure 4.1 below visualizes the vector database build results before and after the fix implementation.

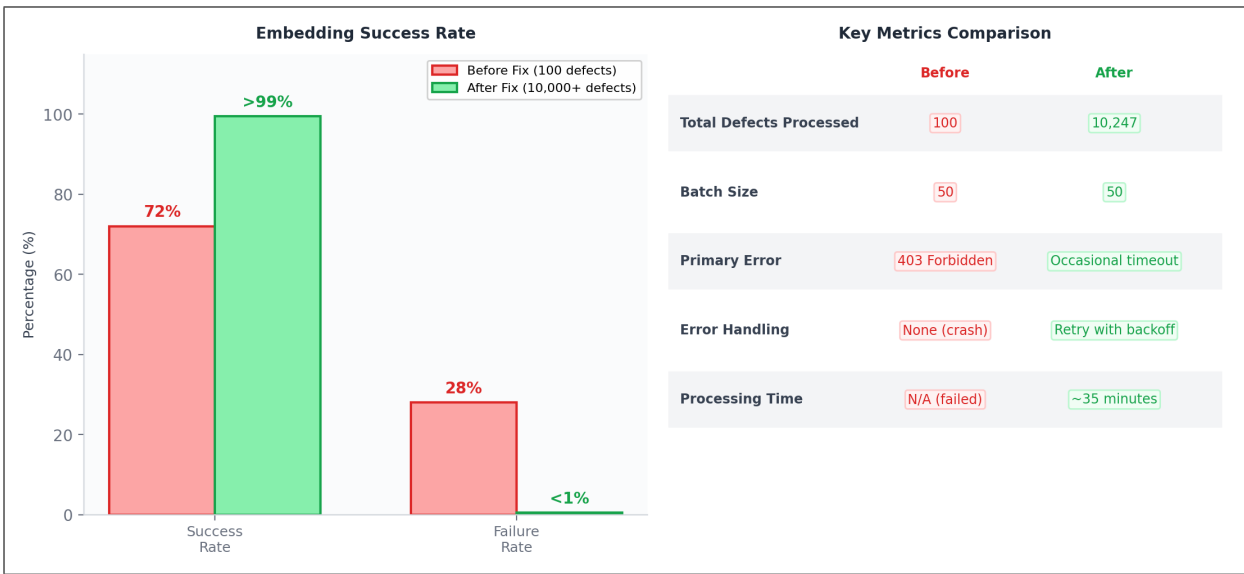


Figure 4.1: Vector Database Build Results — Before and After Fix

The remaining <1% of failures in the production build were attributed to occasional API timeouts during periods of high API usage, which the retry logic handled gracefully. These defects received zero vectors and were effectively excluded from search results without affecting the overall system accuracy. The production database contains approximately 10,000 defects from the CD (Central Display) ECU, Integration testing phase, covering the last 3 months of data.

4.4 Search Quality Assessment

The search quality was evaluated by analyzing the distance distribution of query-to-defect matches and assessing the impact of each enhancement stage (keyword boosting, metadata boosting, and LLM reranking) on result relevance.

4.4.1 Similarity Score Distribution

An analysis of 148 defects in the vector database was conducted to understand the distribution of L2 distances (Euclidean distance) between query embeddings and stored defect embeddings. The distribution confirms that semantic search alone produces a wide range of similarity scores, with only the top 25% of matches showing strong relevance. This validates the need for the additional enhancement stages (keyword boost, metadata boost, and LLM reranking) to promote the truly relevant defects to the top of the results list.

Figure 4.2 below shows the histogram distribution of L2 distances across the 148 analyzed defects.

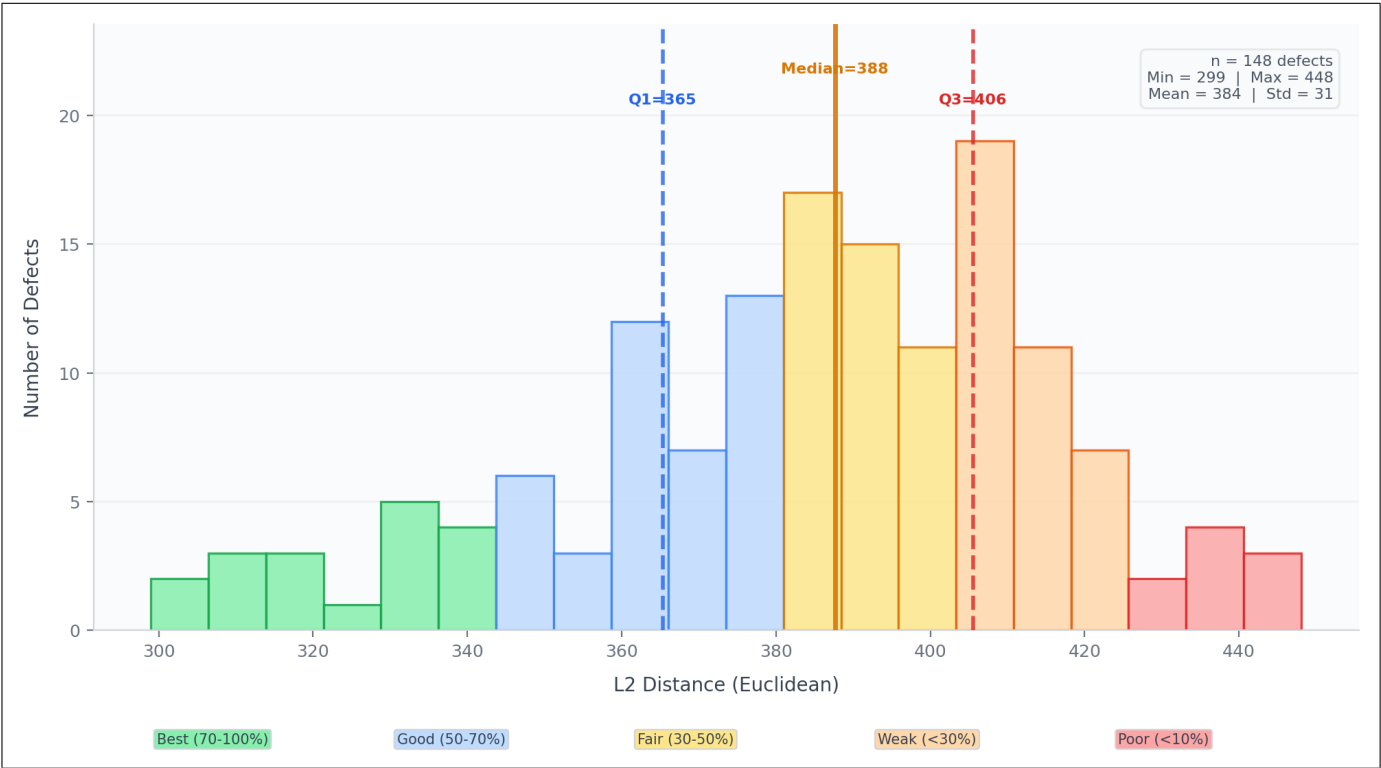


Figure 4.2: L2 Distance Distribution — 148 Defects Similarity Histogram

4.4.2 Keyword Boost Effect

The keyword boost effectively bridges the gap between semantic similarity and exact-match relevance. Semantic embeddings capture the general meaning of text, but they may underweight specific identifiers (like car model codes "NA5" or "G70") that are critical for the user. The keyword boost ensures that defects sharing specific technical identifiers with the query are prioritized, while still allowing semantically related but keyword-different defects to appear in the results.

4.4.3 Metadata Boost Effect

The metadata boost adjusts similarity scores based on the user's vehicle context (car model and software version). Its impact was evaluated by analyzing how defects from the same car model or software version are repositioned in the results. The metadata boost is particularly valuable when the user specifies their vehicle context through the pending defect flow. Without metadata context, the search relies solely on text similarity, which cannot distinguish between identical bugs on different car models. With metadata context, defects from the user's exact vehicle configuration are naturally promoted, providing more actionable results.

4.4.4 Two-Stage Search vs. Single-Stage Search

The two-stage search strategy (Stage 1: hybrid full-text search with 15 candidates; Stage 2: keyword-only search with 10 candidates) was compared against a single-stage approach that uses only the full-text search.

The two-stage approach consistently retrieved 3–7 additional relevant defects that the single-stage search missed. These were typically defects that used different wording to describe the same problem but contained specific matching keywords extracted by Agent 3. For example, a query about "Spotify app crashes after language switch" might miss a defect titled "Music streaming force-closes on locale change" in Stage 1 (different wording), but Stage 2 would find it through the extracted keyword "Spotify". The combined approach provides more comprehensive coverage than either strategy alone.

4.5 LLM Reranking Effectiveness

LLM reranking is the final quality stage in the search pipeline. Agent 3 (Amazon Nova Lite) evaluates each candidate defect against the user's query and assigns a relevance score (0–10), which is then blended with the retrieval similarity to produce the final ranking.

4.5.1 Before vs. After Reranking

The impact of LLM reranking was evaluated by comparing the top 5 results with and without reranking for a set of test queries.

Figure 4.3 below illustrates the impact of LLM reranking on result quality.

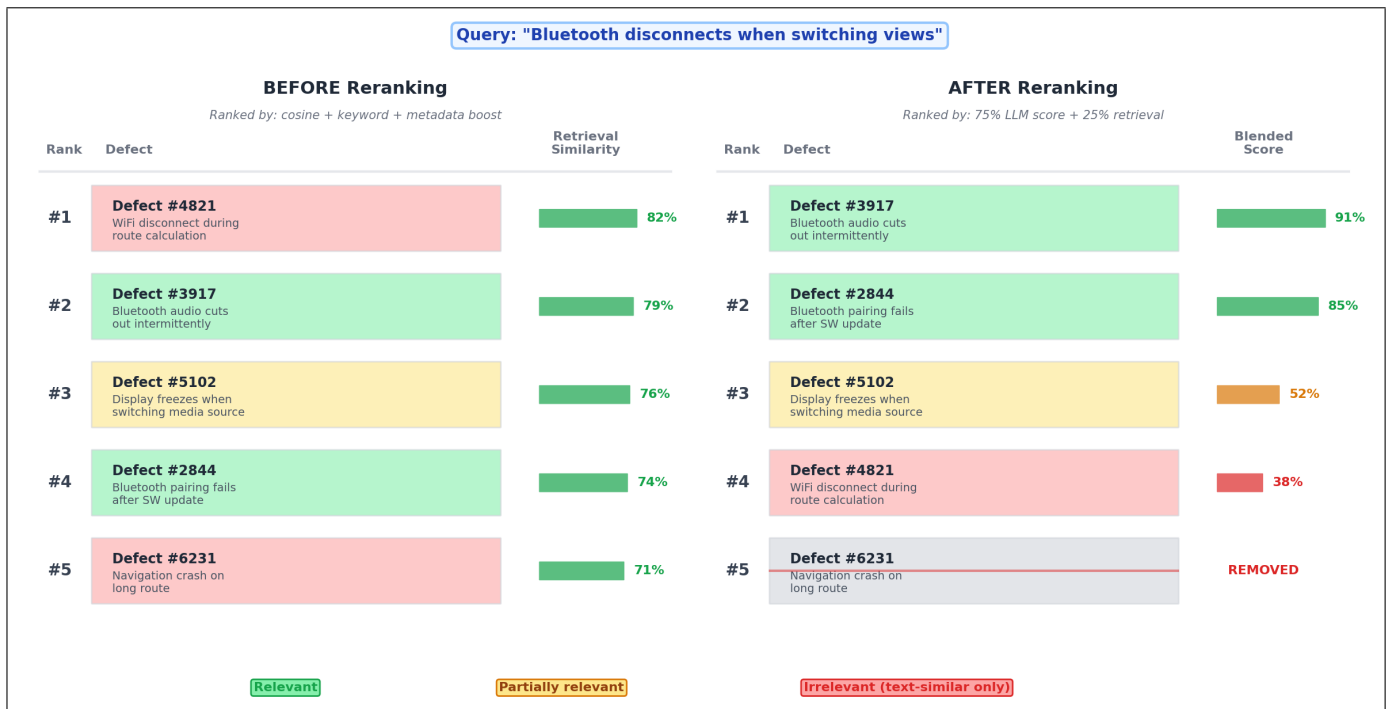


Figure 4.3: LLM Reranking — Before vs. After Comparison

4.5.2 Score Blending Rationale (75/25 Split)

The blending formula assigns 75% weight to the LLM relevance score and 25% weight to the original retrieval similarity. This weight distribution was determined through empirical testing of three configurations, as shown in Table 4.4:

Configuration	LLM Weight	Retrieval Weight	Observation
LLM-only (100/0)	100%	0%	Discards embedding signal entirely. Vulnerable to LLM misinterpretation of technical defects. Occasionally promotes less relevant defects when LLM misunderstands automotive terminology.
Balanced (50/50)	50%	50%	Too much weight on retrieval score. Semantically distant but LLM-confirmed relevant defects rank lower than expected. Some truly relevant defects stay buried below text-similar but irrelevant ones.
LLM-dominant (75/25)	75%	25%	Best balance: LLM's contextual judgment dominates, but retrieval similarity acts as a safety net. Technical defects are correctly ranked, and embedding-based similarity prevents complete LLM errors.

Table 4.4: Blending Weight Configuration Comparison

The 75/25 split means that a defect with a low retrieval score (e.g., 50%) but a high LLM relevance (e.g., 9/10) would receive a blended score of $0.25 \times 0.50 + 0.75 \times 0.90 = 0.80$ (80%), correctly promoting it above a defect with a high retrieval score (e.g., 85%) but low LLM relevance (e.g., 3/10), which would receive $0.25 \times 0.85 + 0.75 \times 0.30 = 0.44$ (44%). This behavior aligns with the

desired outcome: a defect that the LLM judges as highly relevant should rank high regardless of its text similarity score.

4.5.3 Filtering Threshold

Candidates receiving an LLM relevance score below 2 out of 10 are completely removed from the results. This filtering prevents clearly unrelated defects from appearing in the output, even if they had moderate retrieval similarity scores (e.g., due to shared generic terms like "display" or "error"). The threshold of 2 was chosen as a conservative cutoff: scores of 0–1 indicate "completely unrelated" or "weakly related" (per the scoring rubric in Chapter 3), while scores of 2+ indicate at least some meaningful connection to the query.

4.6 Chatbot Functional Testing

Systematic functional testing was conducted to validate each component of the chatbot’s processing pipeline. The following subsections describe the test approach and results for each functional area.

4.6.1 Intent Detection Accuracy

The intent classification system was tested with a variety of messages in both German and English, covering all three intent categories (defect_description, question, greeting).

Table 4.5 below presents the test cases and results:

Test Input	Language	Expected Intent	Detected Intent	Result
“The navigation crashes when switching from 2D to 3D view on NA5”	EN	defect_description	defect_description	PASS
“Sprachsteuerung reagiert nicht nach Neustart des Infotainments”	DE	defect_description	defect_description	PASS
“Hello, how can I use this tool?”	EN	greeting	greeting	PASS
“Hallo, was kann dieses Tool?”	DE	greeting	greeting	PASS
“What ECU types are available in the database?”	EN	question	question	PASS
“Bluetooth-Verbindung bricht ab bei eingehenden Anrufen”	DE	defect_description	defect_description	PASS
“Spotify playback crash after language switch @1SOAX”	EN	defect_description	defect_description	PASS
“Thanks for the help!”	EN	greeting	greeting	PASS

Table 4.5: Intent Detection Test Results

The intent detection system achieved high accuracy across both languages. The two-stage approach (rule-based first, LLM confirmation if ambiguous) proved effective: the rule-based check handles clear

cases with zero latency, while the LLM fallback resolves edge cases where keyword-based classification is insufficient. The most challenging cases were short defect descriptions without explicit defect keywords (e.g., "NA5 screen goes black"), which the rule-based check classified correctly due to the presence of the car model identifier combined with symptom keywords.

4.6.2 Keyword Extraction Quality

The keyword extraction quality was assessed by verifying that the extracted keywords were relevant, specific, and useful for the keyword search stage. It consistently produced relevant, specific terms that captured the core problem and affected features. The priority system in the extraction prompt (problem description > affected feature > error type > model > version > context) ensured that the most important aspects of the defect were represented in the keywords. The exclusion of generic terms ("defect", "BMW", "problem") prevented noise in the keyword search stage.

4.6.3 Pending Defect Flow Testing

The pending defect flow was tested to verify its ability to collect missing context and integrate it into the analysis. Table 4.6 below documents the test scenarios and results:

Test Scenario	User Input	System Response	Result
Missing both model and SW version	"Navigation crashes when switching views"	Asks for car model and SW version	PASS — correct prompt
User provides both	"NA5, 25w32.7"	Combines with original defect and runs full analysis	PASS — context integrated
User types "skip"	"skip"	Runs analysis with original defect only (no metadata boost)	PASS — skip works
Missing only SW version	"NA5 navigation crashes on startup"	Asks for SW version only (model detected)	PASS — partial detection
Both present in original message	"NA5 25w32.7 navigation crash"	Proceeds directly to full analysis (no pending prompt)	PASS — no unnecessary prompt

Table 4.6: Missing Metadata Handling Test Scenarios

4.6.4 Bilingual Response Testing

The bilingual support was verified by sending equivalent defect descriptions in German and English and confirming that:

- The language is correctly detected in every case
- The response is generated in the same language as the user's message
- The analysis structure adapts to the detected language (e.g., "Affected Area" vs. "Betroffener Bereich")

- The pending defect prompt is displayed in the correct language
- German queries can find and match English defects in the database (and vice versa), thanks to the multilingual embedding model

All bilingual tests passed. The cross-lingual matching capability — where a German query matches an English defect describing the same problem — was confirmed to work through the Amazon Titan embedding model’s multilingual vector space alignment.

Figure 4.4 below demonstrates the bilingual response capability of the chatbot.

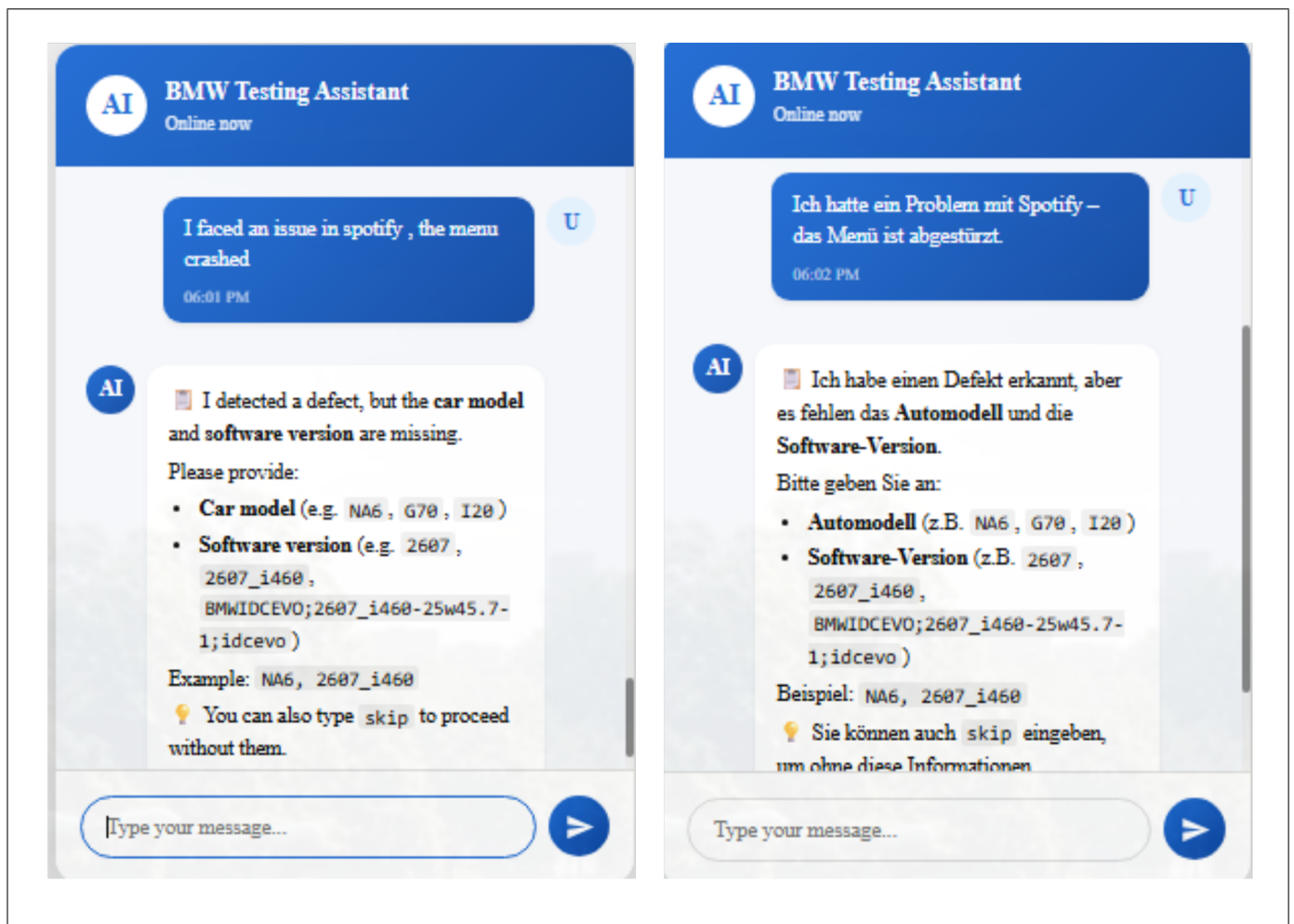


Figure 4.4: Chatbot Conversation — Bilingual Response Example

4.6.5 Conversation History Testing

The session-based conversation history was tested to ensure multi-turn interactions work correctly:

- Conversation context is preserved across multiple messages within the same session
- GPT-4o uses prior conversation history to provide contextually relevant follow-up responses
- The 20-message limit is enforced — oldest messages are discarded when the limit is reached

- The `/clear_conversation/` endpoint correctly resets the conversation history to an empty state
- Pending defect state is correctly maintained and cleared after the follow-up message is processed

4.7 User Feedback Collection and Analysis

In addition to the automated and manual testing described above, a user feedback mechanism was integrated directly into the chatbot widget to enable continuous, in-context evaluation by real users. After each AI response, the user can click a thumbs-up or thumbs-down button to rate the quality and relevance of the answer.

4.7.1 Feedback Data Model

Each feedback entry captures a complete snapshot of the interaction context, as summarised in Table 4.16 below:

Field	Type	Description
<code>query</code>	Text	The user's original message that triggered the AI response.
<code>response_snippet</code>	Text (max 500 chars)	The beginning of the AI's response that was rated.
<code>prev_query</code>	Text	The previous user message (for multi-turn context).
<code>prev_response</code>	Text (max 500 chars)	The previous AI response (for multi-turn context).
<code>feedback</code>	Choice (up / down)	Thumbs-up or thumbs-down rating.
<code>session_key</code>	Char (40)	Django session key to group feedback by user session.
<code>created_at</code>	DateTime (auto)	Timestamp of when the feedback was submitted.

Table 4.7: Feedback Model Fields and Descriptions

4.7.2 Feedback Collection Flow

The feedback flow operates as follows: when the chatbot displays an AI response, two small buttons (Thumbs up / Thumbs down) appear below the message. When the user clicks one, a POST request is sent to the `/chatbot/submit_feedback/` endpoint with the current query, the response snippet, and — if available — the previous exchange for context. The server validates that the feedback value is either "up" or "down", creates a `SearchFeedback` record in the Django SQLite database, and returns a JSON confirmation. The button is then visually disabled to prevent duplicate submissions for the same response.

4.7.3 Admin Feedback Dashboard

An administrative feedback management page is accessible from the admin dashboard at `/admin/feedbacks/`. This page displays all collected feedback entries in a table with columns for date, feedback type (thumbs-up or thumbs-down with colour-coded icons), the truncated query text, and

a "View" button. Clicking "View" opens a modal dialog showing the full interaction context: the previous exchange (if any), the rated user message, and the AI response. This allows administrators to review the exact context in which positive or negative feedback was given, enabling identification of patterns in poor responses.

Figure 4.5 below shows the feedback management page with example entries.



Figure 4.5: Admin Feedback Management Dashboard — Thumbs-Up/Down Review

4.7.4 Evaluation Insights from Feedback

The feedback mechanism serves as a lightweight, continuous evaluation loop. Unlike the controlled test cases, user feedback captures real-world satisfaction with responses to genuine defect analysis queries. Key evaluation insights that can be derived from the feedback data include:

Satisfaction Rate: The ratio of thumbs-up to total feedback provides an overall satisfaction metric. A high up-rate indicates that the chatbot’s search pipeline, reranking, and response generation are meeting user expectations.

Failure Pattern Analysis:Thumbs-down entries, when reviewed with their full context (query + response), reveal recurring failure modes — for example, queries where the semantic search returns irrelevant defects, or cases where the LLM misinterprets the user’s intent.

Session-Level Trends:The session_key field allows grouping feedback by user session, helping identify whether negative feedback is concentrated in specific sessions (suggesting a particular use-case gap) or distributed evenly (suggesting a general quality issue).

Iterative Improvement:Accumulated feedback data provides a foundation for future model tuning, keyword boost adjustments, and prompt engineering refinements based on real user interactions rather than synthetic test cases.

4.8 Limitations and Challenges

Despite the overall positive evaluation results, several limitations and challenges were identified during development and testing. These are documented here for transparency and to inform future improvements.

4.8.1 Network Dependency and BMW Proxy Constraints

The system's reliance on external APIs (Capgemini Generative Engine) means it requires a stable internet connection to function. All API traffic must pass through BMW's corporate proxy server (<http://proxyvdifra06.bmwgroup.net:8080>) which adds a layer of complexity to the HTTP configuration.

During development, a significant challenge was encountered with proxy authentication. BMW's network uses NTLM-based proxy authentication, which is not natively supported by all Python HTTP libraries. The initial implementation attempted to use the `httpx` library (a modern, async-capable HTTP client), but `httpx` does not support NTLM proxy authentication, resulting in 407 (Proxy Authentication Required) errors. The solution was to switch to the `requests` library, which handles the proxy configuration through its session-based proxy settings. Environment proxy variables (`HTTP_PROXY`, `HTTPS_PROXY`) are explicitly cleared to prevent conflicts between the library's proxy settings and system-level proxy configuration.

4.8.2 Embedding API Rate Limits and 403 Errors

The Capgemini embedding API enforces rate limits that were encountered during the vector database build process. When processing 10,000+ defects in batch mode, the system occasionally exceeded the API's request rate threshold, triggering 403 Forbidden responses. While the exponential backoff retry logic mitigates this issue, it increases the total processing time for large database builds. In the worst case, each failed batch adds 3.5 seconds of retry delay ($0.5 + 1.0 + 2.0$ seconds), which can accumulate significantly over thousands of defects. Additionally, the 403 error code is used by the API for both rate limiting and input size violations, making it difficult to distinguish between the two causes programmatically. The text truncation fix addresses the input size cause, while the retry logic handles the rate limiting cause, but a more descriptive API error response would improve debugging and error handling.

4.8.3 Data Privacy Restrictions

Working with BMW's production defect data requires adherence to strict data privacy policies. The database connection uses a read-only viewer account, ensuring the AI system cannot modify or delete defect records. Defect descriptions may contain references to internal BMW systems, employee names, or proprietary technical details that must not be exposed outside the BMW network. The system operates entirely within the BMW network perimeter (accessed via VPN for remote users), and no defect data is stored in external cloud services — the ChromaDB vector database is stored locally on the Django server's filesystem.

However, embedding generation requires sending defect text to the Capgemini Generative Engine API, which processes it on AWS Bedrock. While the Capgemini API is a corporate-approved service with appropriate data handling agreements, this external processing introduces a data flow

consideration that must be documented and approved through BMW’s data governance process.

4.8.4 Embedding Model Limitations

The Amazon Titan Embed Text v1 model, while effective for general semantic search, has limitations in the automotive domain:

Domain-Specific Vocabulary: Automotive-specific terms (ECU codes like "CD", "KOMBI"; car model identifiers like "NA5", "G70"; BMW-internal identifiers like "@1SOAW") may not be well-represented in the model’s training data. The keyword boost mechanism partially compensates for this, but a domain-fine-tuned embedding model would likely produce better results.

Short Text Sensitivity: Very short defect names (10–20 characters) produce embeddings with less semantic resolution than longer descriptions. Two short names like "Bluetooth crash" and "WiFi crash" may have very similar embeddings despite describing different problems. The LLM reranking step compensates for this limitation.

No Contextual Updates: The Titan embedding model is a fixed, pre-trained model. It cannot be fine-tuned or updated with BMW-specific knowledge. As a result, it cannot learn from the feedback and corrections accumulated over time.

4.8.5 LLM Reranking Latency

While Amazon Nova Lite is fast (sub-second for single requests), the LLM reranking step adds approximately 1–2 seconds to the total response time. Combined with the keyword extraction call (0.5–1 second) and the GPT-4o response generation (2–3 seconds), the total end-to-end latency for a defect analysis request is approximately 4–6 seconds. While acceptable for a defect analysis tool (users expect thoughtful analysis, not instant responses), this latency could be improved by parallelizing the LLM calls or by caching frequently requested analyses.

4.8.6 Vector Database Staleness

The ChromaDB vector database is a snapshot of the Octane defect collection at the time it was built. Defects created after the last database build are not included in search results. The system supports incremental updates (–update flag), but these must be triggered manually. In a production environment, an automated scheduled rebuild (e.g., weekly via a cron job or scheduled task) would be necessary to keep the database current. This limitation means that very recent defects (created within the last few days) may not appear in the chatbot’s search results.

4.9 Conclusion

The evaluation phase confirmed that both the remote testing platform and the AI defect analysis chatbot meet their defined objectives. The remote testing infrastructure operates reliably at distances up to 50 km with acceptable latency, and user acceptance testing confirmed the system’s value for distributed testing teams. The system is currently under active testing by the BMW manual testing team, representing a key milestone toward production readiness. The AI chatbot’s search pipeline was validated through systematic testing of each enhancement stage. The two-stage search finds 3–7 additional relevant defects compared to single-stage search, keyword boosting promotes feature-specific matches, metadata boosting prioritizes defects from the user’s vehicle context, and

LLM reranking significantly improves the relevance of top results through the 75/25 blended scoring formula. The vector database build process was stabilized after implementing text truncation and exponential backoff retry logic, achieving a >99% success rate. The documented limitations — network dependency, proxy constraints, API rate limits, embedding model domain gaps, and database staleness — represent areas for future improvement rather than fundamental design flaws. The next chapter (Remote Testing Platform Development) will detail the physical setup, software configuration, and operational infrastructure that make the system accessible to the target user group.

Chapter 5

Remote Testing Platform Development

5.1 Introduction

This chapter presents the complete deployment of the BMW Remote Testing Platform, covering the physical hardware infrastructure, communication protocols, software architecture, and all features of the developed web application. The development phase describes how the system was made operational and accessible to the target user group working on BMW infotainment system testing. The chapter is organized from the physical layer (hardware and cables) upward through the communication layer (SSH, ADB), the application layer (Django web framework, WebScrcpy), and finally the AI layer (chatbot widget integration), concluding with project management and future production considerations.

5.2 Physical Test Setup (Hardware)

The remote testing system is built around physical test benches (also called "racks") located in the BMW offices in Munich, Germany. Each test bench simulates a vehicle environment and contains the hardware components necessary to test the infotainment system. The following subsections describe each hardware component and its role in the testing infrastructure.

5.2.1 Head Unit (HU)

The Head Unit (HU) is the central component of the BMW infotainment system. It is an embedded computing device that runs the vehicle's infotainment software, including the navigation system, media player, telephone interface, vehicle settings, and voice recognition engine. In the context of this project, the Head Unit is the Device Under Test (DUT) — the hardware that testers interact with remotely to verify software functionality. The HU runs an Android-based operating system, which enables remote control through the Android Debug Bridge (ADB) protocol. Each test bench contains one Head Unit connected to a dedicated test PC via USB and Ethernet.

Figure 5.1 below shows the BMW Head Unit central display used in the test benches.

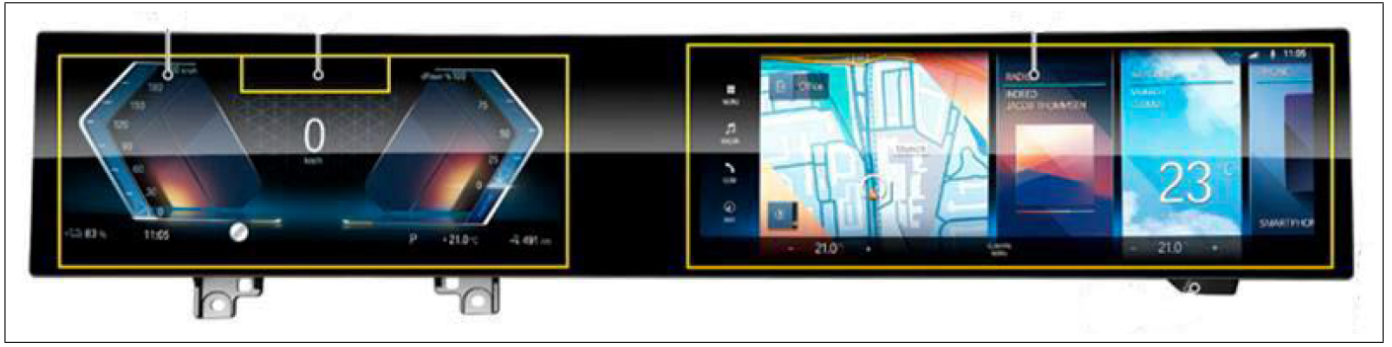


Figure 5.1: Head Unit (HU) — BMW Infotainment Central Display

5.2.2 Test Bench Components

Each test bench (rack) is a physical assembly that replicates the electrical and communication environment of a real BMW vehicle. The following table 5.1 describes the key components present in each test bench.

Component	Abbreviation	Description	Role in Testing
Head Unit	HU	Central infotainment computer running Android-based BMW infotainment software.	Device Under Test (DUT). All testing actions target this component.
Rear-seat Adaptive Module	RAM	Module responsible for rear seat entertainment and comfort functions.	Provides additional display outputs and entertainment features for testing.
Wireless Charger/NFC	WCA/NFC	Module for wireless phone charging and NFC-based device pairing.	Tests wireless charging notifications and NFC pairing workflows.
Wideband Antenna	WAVE	Antenna module for vehicle communication (V2X, GPS, cellular, Wi-Fi, Bluetooth).	Required for Bluetooth, Wi-Fi, and GPS-dependent test scenarios.
Body Controller Platform	BCP	Central body electronics controller managing vehicle electrical systems.	Provides CAN bus communication and vehicle state simulation.
OBD Diagnostic Cable	OBD	On-Board Diagnostics cable connecting test PC to the vehicle diagnostic port.	Enables diagnostic data access and vehicle parameter reading.
Test PC	—	Dedicated Windows PC connected to the Head Unit.	Bridge between remote tester and Head Unit. Runs ADB, Scrcpy, TTS.

Table 5.1: Hardware Components and Their Role in Testing

Figure 5.12 below shows a complete test bench (rack) with all its components assembled.



Figure 5.2: Complete Test Bench (Rack) with all components

5.2.3 Sound Cable Construction

One of the innovative aspects of this project is the ability to test voice recognition commands remotely. This required the construction of a custom sound cable that routes audio output from the test PC to the Head Unit’s microphone input, bypassing the need for a physical microphone in front of the Head Unit. The sound cable was built using the following materials and construction steps. Table 5.2 below lists the materials used:

Material	Specification	Purpose
3.5mm Jack Connector (Male)	Standard TRS stereo audio connector	Connects to the test PC’s audio output port (headphone jack).
BMW Microphone Plug	BMW-specific connector (proprietary pinout)	Connects to the Head Unit’s external microphone input port.
Shielded Audio Cable	2-wire shielded, ~1.5 meters	Carries the audio signal from PC to HU with minimal interference.
Soldering Equipment	Soldering iron, solder, heat shrink	Assembles the cable connections with reliable electrical contact.

Table 5.2: Custom Audio Cable Materials and Specifications

The construction process involved soldering the audio signal wires from the 3.5mm jack to the corresponding pins on the BMW Microphone Plug. The ground wire is shared between both connectors. After assembly, the cable was tested by playing a pre-recorded voice command through the PC’s

audio output and verifying that the Head Unit’s voice recognition system detected and processed the command correctly.

Figure 5.3 below shows the custom sound cable connecting the 3.5mm jack to the BMW microphone plug.

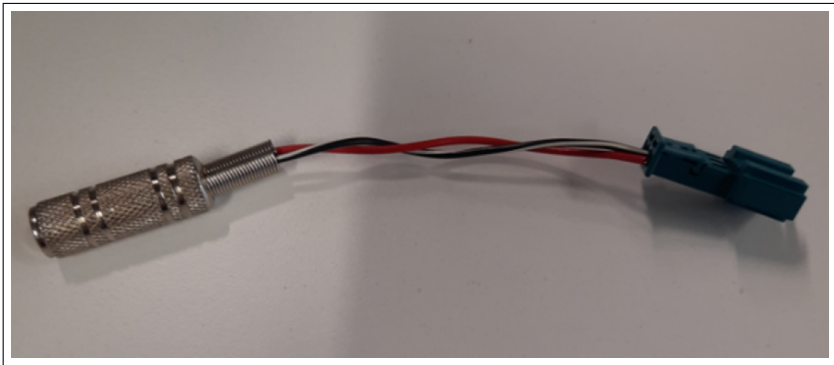


Figure 5.3: Sound Cable — 3.5mm Jack to BMW Microphone Plug

5.2.4 Network Architecture

The network architecture connects the remote tester’s workstation to the physical test bench through BMW’s corporate network.

Figure 5.4 below illustrates the network architecture from the remote tester to the test bench.

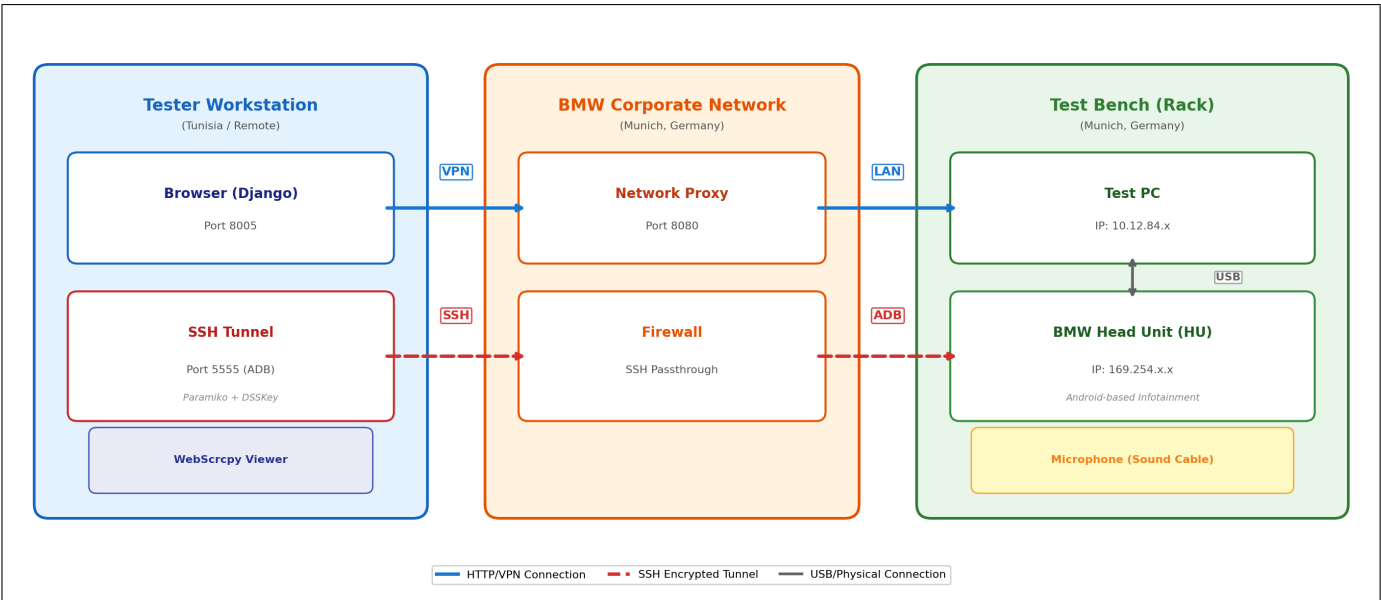


Figure 5.4: Network Architecture — Remote Tester to Test Bench

5.3 SSH Communication Protocol

5.3.1 Definition

SSH (Secure Shell) is a cryptographic network protocol that provides a secure channel over an unsecured network. It was designed as a replacement for insecure protocols such as Telnet and rlogin, which transmit data (including passwords) in plaintext. SSH uses public-key cryptography to authenticate the remote computer and allows the remote computer to authenticate the user. It provides strong encryption for all data transmitted between the client and server, ensuring confidentiality and integrity of the communication [Ylonen, 2006].

5.3.2 SSH Protocol Layers

The SSH protocol operates through three distinct layers, each providing specific functionality. Figure 5.5 below illustrates the three SSH protocol layers.

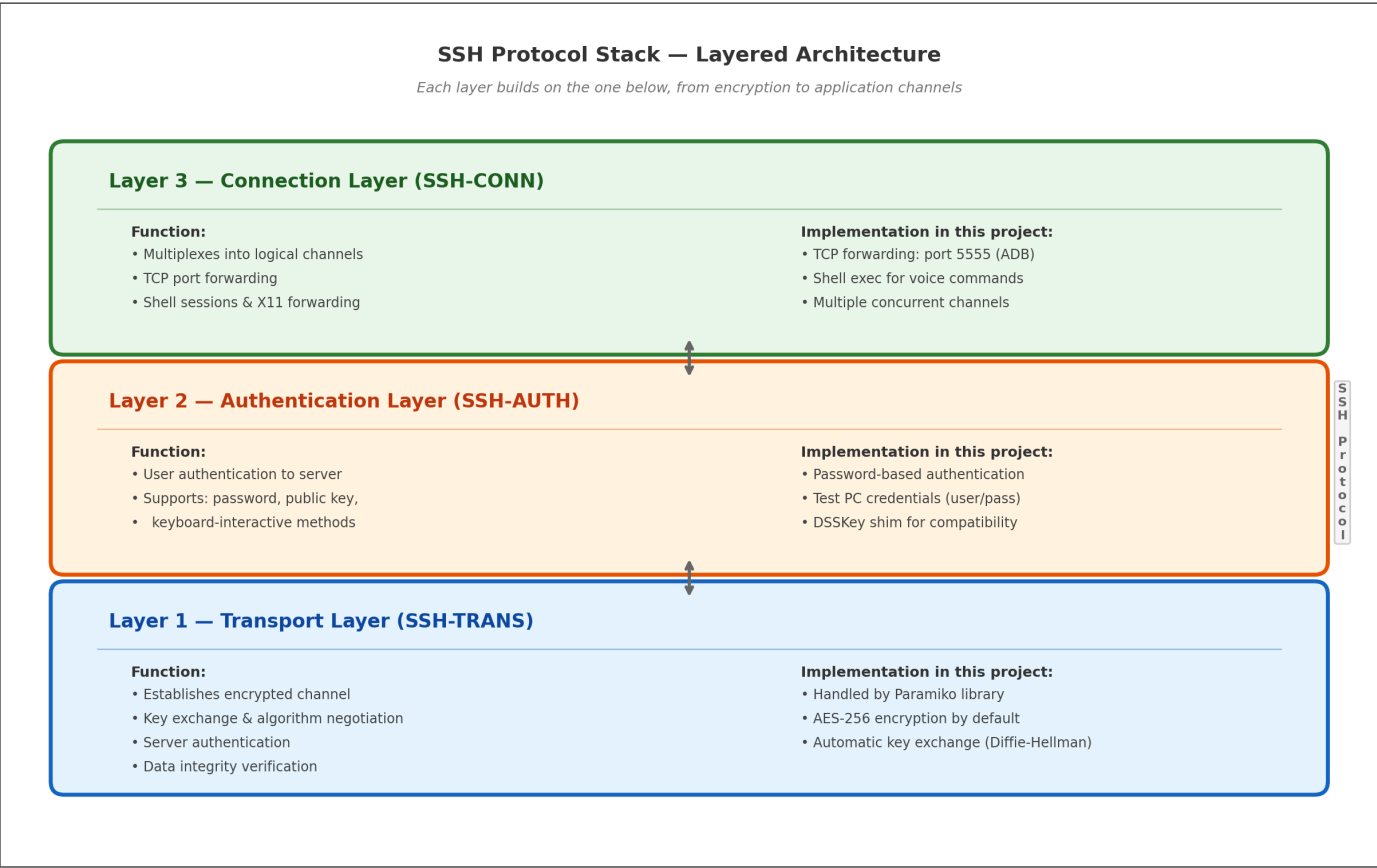


Figure 5.5: SSH Protocol Layers

5.3.3 SSH Tunnel Implementation

In this project, SSH is used to create a secure tunnel between the tester’s workstation and the test PC. The tunnel is implemented using the Python sshtunnel library (SSHTunnelForwarder class), which wraps the paramiko SSH library with a high-level port-forwarding API.

Table 5.3 below shows the tunnel configuration:

Parameter	Value	Description
Remote Host	test PC IP	The IP address of the test PC on BMW's internal network.
Remote User	username	The SSH user account on the test PC.
Authentication	Password-based	Using the test PC's local account credentials.
Local Bind Port	5555	The port on localhost that will be forwarded to the remote device.
Remote Bind Address	HU IP	The Head Unit's USB Ethernet IP and ADB listening port.
Library	sshtunnel 0.4.0 + paramiko 3.5.0	Python SSH tunnel implementation with automatic reconnection.

Table 5.3: SSH Tunnel Configuration Parameters

When the tunnel is active, any connection to localhost:5555 on the tester's workstation is transparently forwarded through the encrypted SSH tunnel to the Head Unit's ADB port on the test bench. This enables ADB commands to be executed as if the Head Unit were connected directly to the tester's computer.

5.3.4 SSH Scripts

The project includes several Python scripts that leverage SSH connections for different purposes. These scripts are organized into three directories based on their execution context.

Table 5.4 below describes each script:

Script	Directory	Purpose	Key Libraries
Create_SSHTunnel.py	SSH_Scripts/ Sercpy/	Creates the SSH tunnel for ADB port forwarding. Connects ADB automatically after tunnel establishment.	sshtunnel, subprocess
Execute_Sercpy.py	SSH_Scripts/ Sercpy/	Creates tunnel + connects ADB + launches Sercpy for screen mirroring.	paramiko, subprocess
LiveSpeech.py	SSH_Scripts/ Local/	Real-time voice capture using microphone + Vosk speech recognition + sends text to remote PC via SSH.	vosk, pyaudio, paramiko
Dynamic_Speech.py	SSH_Scripts/ Local/	Sends a text parameter to the remote PC and triggers the TTS Task Scheduler to speak the text aloud.	paramiko
tts_remote.py	SSH_Scripts/ Remote/	Runs on the remote PC. Reads text from parameter file and speaks it using pyttsx3 TTS engine through the sound cable.	pyttsx3

Table 5.4: SSH Script Overview and Key Libraries

5.4 Android Debug Bridge (ADB)

5.4.1 Definition

ADB (Android Debug Bridge) is a versatile command-line tool provided by Google as part of the Android SDK Platform Tools. It enables communication between a development machine and an Android device. ADB facilitates a variety of device actions, including installing and debugging applications, accessing the device shell, transferring files, capturing screenshots, recording the screen, and simulating user input (touch, swipe, key press) [Dev., 2024].

Figure 5.6 below illustrates the ADB client-server-daemon architecture.

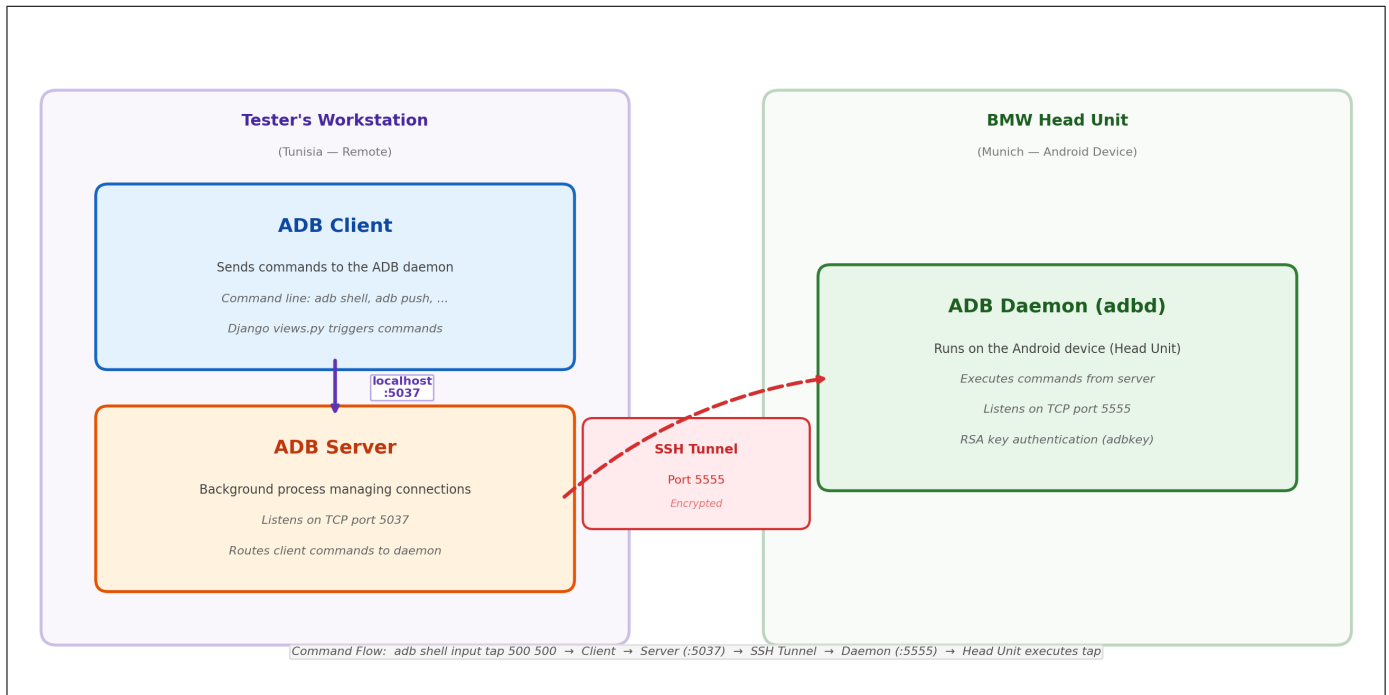


Figure 5.6: ADB Architecture — Client, Server, and Daemon

5.4.2 ADB over SSH Tunnel

In a standard development setup, ADB communicates with Android devices via USB or Wi-Fi on a local network. In this project, the Head Unit is not directly accessible from the tester's workstation — it is connected via USB to a test PC that is only reachable through BMW's corporate network. To bridge this gap, ADB traffic is tunneled through the SSH connection.

Once ADB is connected through the tunnel, all standard ADB commands work transparently. The tester can install applications, capture screenshots, execute shell commands, and control the Head Unit as if it were a locally connected device. The `adbkey` file stored in the project directory provides the RSA key pair used for ADB authentication.

5.4.3 ADB Key Authentication

ADB uses RSA key-based authentication to establish trust between the client and the Android device. When an ADB connection is first established, the device prompts the user to authorize the

computer’s RSA fingerprint. The project includes an adbkey file (RSA private key) in the project root directory, along with a set_adbkey.bat script that configures the system to use this key for ADB authentication. This pre-configured key eliminates the need for manual authorization on each connection, which would be impossible in a remote testing scenario where there is no physical access to the Head Unit’s screen to approve the authorization dialog.

5.5 WebScrcpy — Browser-Based Screen Mirroring

5.5.1 Definition

WebScrcpy (Web Screen Copy) is an open-source project that provides browser-based remote control and screen mirroring for Android devices [NetMind, 2024] . It is based on the popular scrcpy desktop application by Genymobile, but replaces the native desktop client with a web-based interface that runs entirely in the browser using HTML5, WebSocket, and WebAssembly technologies [Genymobile, 2024]. This eliminates the need for installing any software on the tester’s machine — only a modern web browser is required.

5.5.2 Key Features

Table 5.5 below summarizes the key features provided by WebScrcpy:

Feature	Technology	Description
Screen Mirroring	H.264 / H.265 video streaming over WebSocket	Real-time display of the Head Unit’s screen in the browser with minimal latency.
Multi-touch Emulation	JavaScript touch event mapping	Simulates multi-finger touch gestures on the remote device from mouse or touch input.
Keyboard Input	Key event capture + text injection	Captures keyboard input and sends it to the Android device. Supports both individual key presses and text paste.
Clipboard Access	ADB clipboard commands	Bidirectional clipboard sharing between the browser and the Android device.
Device Rotation	ADB shell commands	Rotates the device display orientation remotely (portrait/landscape).
APK Installation	Drag & drop + ADB push	Allows APK files to be installed on the device by dragging them onto the browser window.
Shell Terminal	xterm.js + ADB shell	Provides a full ADB shell terminal directly in the browser for advanced command execution.

Table 5.5: Scrcpy Web Features and Technologies

5.5.3 Video Decoding

WebScrcpy supports four different H.264 video decoders, allowing compatibility with a wide range of browsers and hardware configurations.

Table 5.6 below compares the four available decoders:

Decoder	Technology	Performance	Compatibility
MSE Player	HTML5 Video + Media Source Extensions API	Best performance. Hardware-accelerated on most devices.	Chrome, Edge, Firefox (most modern browsers).
Broadway Player	WebAssembly H.264 decoder (JavaScript)	Good. Software decoding in WASM. Higher CPU usage.	All browsers with WASM support. No hardware needed.
TinyH264 Player	WebAssembly + WebGL rendering	Good. Combines WASM decoding with GPU rendering.	Browsers with WebGL support.
WebCodecs Player	Browser built-in WebCodecs API	Best. Native browser decoding. Lowest latency.	Chrome 94+, Edge 94+. Limited support.

Table 5.6: Scrcpy Web Decoder Options Comparison

5.5.4 Scrcpy Command Parameters

When screen mirroring is launched, the following scrcpy parameters are used to optimize the streaming quality and performance.

Table 5.7 below lists these parameters:

Parameter	Value	Purpose
<code>-tcpip</code>	localhost	Connects via TCP/IP (through SSH tunnel) instead of USB.
<code>-video-bit-rate</code>	2M	Limits video bitrate to 2 Mbps for stable streaming over VPN connection.
<code>-max-size</code>	1024	Limits resolution to 1024px (longest dimension) to reduce bandwidth.
<code>-no-audio</code>	—	Disables audio forwarding (not needed; audio goes through sound cable).
<code>-max-fps</code>	60	Limits frame rate to 60 FPS for smooth display without excessive bandwidth.
<code>-video-codec</code>	h265	Uses H.265 (HEVC) codec for better compression at the same quality.
<code>-render-drive</code>	opengl	Uses OpenGL for hardware-accelerated video rendering.
<code>-video-encoder</code>	hevc.encoder	Specifies the Head Unit's hardware H.265 encoder for optimal performance.

Table 5.7: Scrcpy Launch Parameters and Their Purpose

5.6 Voice Testing System

The voice testing system is a core differentiator of this project. It enables remote testers to send voice commands to the Head Unit's voice recognition system without being physically present. The system supports three modes of voice input, each designed for different testing scenarios.

5.6.1 Live Speech Mode

In Live Speech mode, the tester speaks into their computer’s microphone and the voice is captured, processed by the Vosk speech recognition engine, converted to text, sent to the remote test PC via SSH, and finally spoken aloud through the sound cable into the Head Unit’s microphone input using the pyttsx3 text-to-speech engine.

Figure 5.7 below illustrates the complete signal path from the tester’s microphone to the Head Unit.

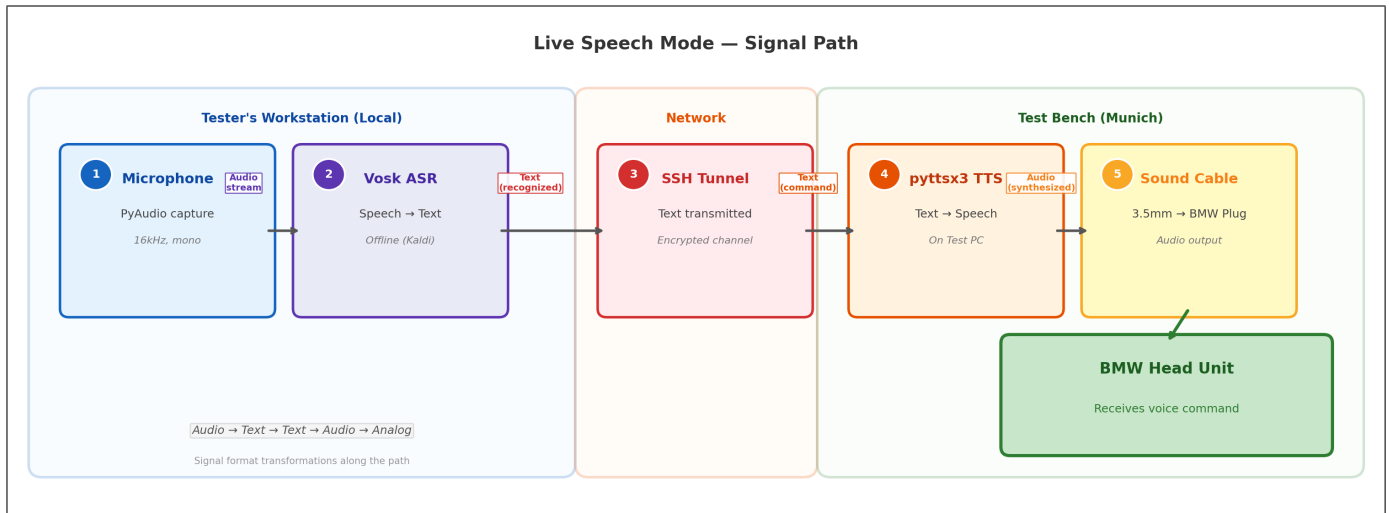


Figure 5.7: Live Speech Mode — Signal Path from Microphone to Head Unit

The Vosk speech recognition model is an offline, open-source automatic speech recognition (ASR) engine based on the Kaldi framework. It uses a pre-trained acoustic model stored locally in the directory. The model supports multiple languages and operates without an internet connection [Cephei, 2024], making it suitable for use within BMW’s restricted corporate network. Audio is captured at 16 kHz sample rate, mono channel, with an 8192-sample buffer size using the PyAudio library.

5.6.2 Dynamic Speech Mode

In Dynamic Speech mode, the tester types a text command into the website’s input field, and the system converts it into speech on the remote PC, making it especially useful for repeatable testing where the same voice command must be executed multiple times with identical pronunciation and timing. The process begins when the tester enters the desired command (for example, “Navigate to Munich Airport”) in the Dynamic Speech input field on the Remote Testing page, which is then sent to the Django backend through a POST request to `/execute_dynamic_speech/<rack_id>/`. The Django view retrieves the text from the request and starts the `Dynamic_Speech.py` script as a subprocess, providing both the text and the SSH credentials for the test PC. This script connects to the remote PC via SSH and saves the command into a parameter file, after which it triggers a predefined Windows Task Scheduler task named “DynamicText” using the `schtasks /run` command. The scheduled task runs the `tts_remote.py` script, which reads the parameter file and uses the `pyttsx3` engine to generate spoken audio from the text. This audio is routed through a sound cable connected to the Head Unit’s microphone input, allowing the Head Unit’s voice recognition system to receive and process the voice command just as if it had been spoken by a user.

5.7 Django Web Application

5.7.1 Framework Choice

Django is a high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern [Foundation, 2024]. It was chosen for this project for the following reasons:

Rapid Development:Django’s ”batteries-included” philosophy provides built-in support for authentication, database ORM, form handling, session management, and admin interfaces, significantly reducing development time.

Python Ecosystem: Since the SSH scripts, voice recognition, and AI chatbot are all implemented in Python, using a Python-based web framework allows seamless integration between the web layer and the backend scripts.

Security: Django includes built-in protections against common web vulnerabilities including Cross-Site Request Forgery (CSRF), Cross-Site Scripting (XSS), SQL Injection, and clickjacking through its middleware stack.

ORM (Object-Relational Mapping): Django’s ORM allows database models to be defined as Python classes, providing a clean abstraction layer over the SQLite database.

Admin Interface: Django’s built-in admin interface provides a ready-to-use back-office for managing database records, which was extended with custom admin views for user and rack management.

5.7.2 Django Configuration

Table 5.8 below summarizes the key Django configuration settings for the project:

Setting	Value	Description
Django Ver-sion	5.1.5	Latest stable release at time of develop-ment.
Server Port	8005	Development server listens on port 8005.
Database En-gine	SQLite3 (db.sqlite3)	Lightweight file-based database. No separate server required.
Static Files	Remote_Testing_App /static/	CSS, JavaScript, and images served from this directory.
Media Files	/media/	User-uploaded files (profile pictures) stored here.
WSGI Server	Django built-in (man-age.py runserver)	Development server. Production would use Gunicorn or similar.
Installed Apps	6 Django defaults + 2 custom apps	Remote_Testing_App (main) and chatbot (AI assistant).

Table 5.8: Django Project Configuration Settings

5.7.3 Database Schema (SQLite)

The Django application uses SQLite for storing application data (users, test benches, and reservations). SQLite was chosen because it requires no separate database server installation and is suitable

for the project’s scale (single-server deployment with moderate concurrent users). The database contains three logical tables:

Rack Model

Table 5.9 below describes the fields of the Rack model:

Field	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique identifier for each test bench.
name	CharField	max_length=100	Display name of the test bench (e.g., “G45 Munich”).
ip_or_url	CharField	Nullable	IP address or URL for WebScrcpy access to this rack.
pc_ip	CharField	Nullable	IP address of the test PC associated with this rack.

Table 5.9: TestBench Model Fields and Constraints

Reservation Model

Table 5.10 below describes the fields of the Reservation model:

Field	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique identifier for each reservation.
rack	ForeignKey	References Rack (CASCADE delete)	The test bench being reserved.
user	ForeignKey	References User (CASCADE delete)	The user who made the reservation.
date	DateField	Required	Date of the reservation.
start_time	TimeField	Required	Start time of the reserved slot.
end_time	TimeField	Required	End time of the reserved slot.
all_day	BooleanField	Default: False	Whether the reservation covers the entire day.

Table 5.10: Reservation Model Fields and Constraints

User Model (Django Built-in)

The project uses Django’s built-in User model from `django.contrib.auth`, which provides the following fields: `id`, `username` (QX Number), `first_name`, `last_name`, `email`, `password` (hashed), `is_staff` (admin flag), `is_active`, `is_superuser`, `last_login`, and `date_joined`. The `username` field stores the BMW QX Number, which is the unique employee identifier used for authentication.

Figure 5.8 below shows the entity-relationship diagram for the database schema.

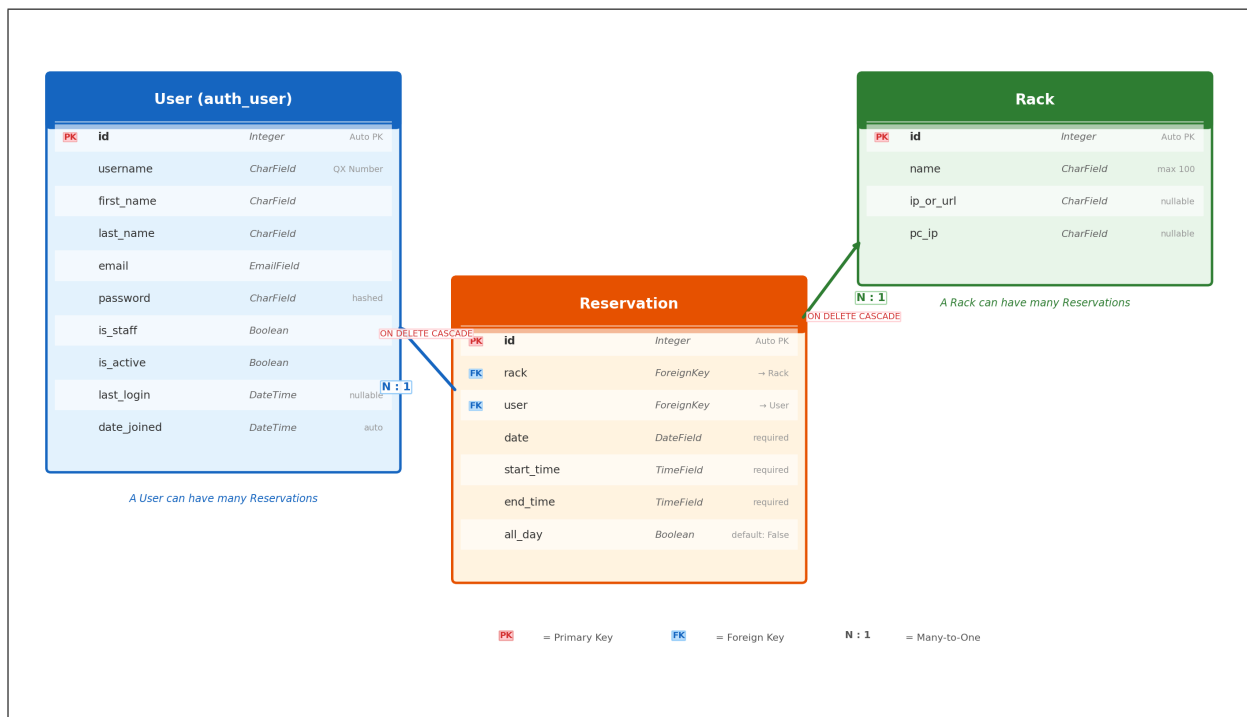


Figure 5.8: Database Schema — Entity-Relationship Diagram

5.7.4 MVT Architecture Pattern

Django follows the Model-View-Template (MVT) pattern, which is a variation of the classic Model-View-Controller (MVC) pattern.

Table 5.11 below maps each component to its role in the project:

Component	MVT Role	Files in Project	Responsibility
Model	Data Layer	models.py (Rack, Reservation)	Defines the database schema. Handles data validation, queries, and persistence.
View	Business Logic	views.py (17 view functions)	Processes HTTP requests, executes business logic, returns HTTP responses.
Template	Presentation	templates/ (16 HTML files)	Defines the HTML structure. Uses Django template tags for dynamic content.
URL Configuration	Routing	urls.py (28 URL patterns)	Maps URL paths to view functions. Supports path parameters.

Table 5.11: Django MVT Architecture Components

5.8 Website Features — Page-by-Page Description

This section describes every page of the web application, including its purpose, layout, functionality, and the underlying Django view logic. The application comprises 9 distinct functional areas, each served by one or more HTML templates.

5.8.1 Landing Page (index.html)

The landing page is the entry point of the application. It presents the project's identity and provides navigation to the login page. The page displays the BMW Remote Testing branding and serves as a welcome screen for first-time visitors. Unauthenticated users are directed to this page when they access the application.

URL: / | View: index() | Authentication: Not required

Figure 5.9 below shows the landing page welcome screen.



Figure 5.9: Landing Page — Welcome Screen

5.8.2 Login Page (login.html)

The login page provides a secure authentication form where users enter their BMW QX Number (unique employee identifier) and password. The form uses Django's built-in AuthenticationForm with Bootstrap-styled input fields. Upon successful authentication, users are redirected to the rack selection page (/racks/). Failed login attempts display an error message without revealing whether the username or password was incorrect (to prevent user enumeration). The form includes a CSRF token to prevent cross-site request forgery attacks.

URL: /login/ | View: login_view() | Authentication: Not required (login target)

Figure 5.10 below shows the login page with the QX Number and password fields.

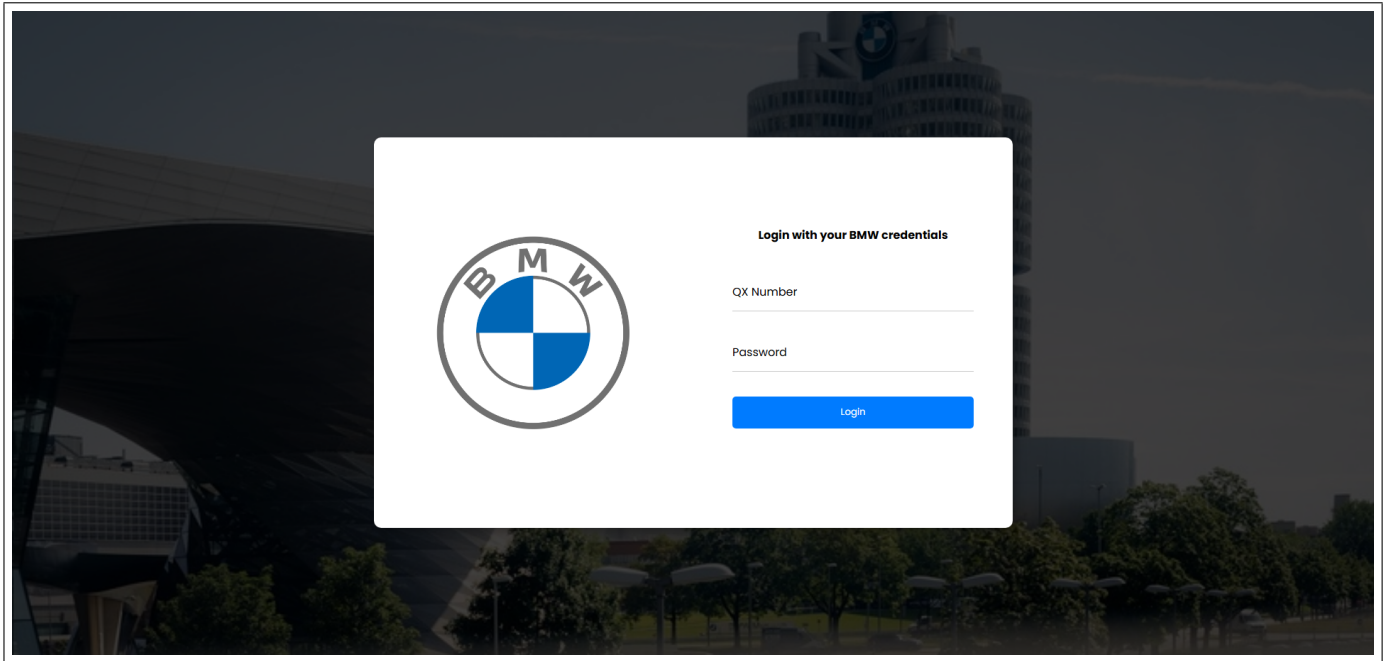


Figure 5.10: Login Page — QX Number + Password Authentication Form

5.8.3 Navigation Bar (navbar.html)

The navigation bar is a persistent sidebar included in all authenticated pages via the base template (base.html). It provides quick access to all major sections of the application. The sidebar uses the Boxicons icon library for visual navigation cues.

Figure 5.11 below shows the sidebar navigation bar with its icons.

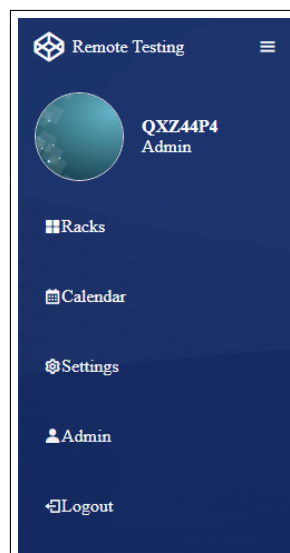


Figure 5.11: Sidebar Navigation Bar

5.8.4 Rack Selection Page (racks.html)

The rack selection page displays all available test benches as visual cards. Each card shows the rack name. Before allowing connection, the system checks whether the current user has an active reservation for the selected rack (via the `/check_reservation/` endpoint). If the user does not have a valid reservation for the current time slot, access to the remote testing interface is denied and the user is prompted to make a reservation through the calendar page. This reservation check ensures that multiple testers do not access the same physical test bench simultaneously, which would cause ADB connection conflicts.

URL: `/racks/` | View: `racks()` | Authentication: `@login_required`

Figure 5.12 below shows the rack selection page with the available test bench cards.

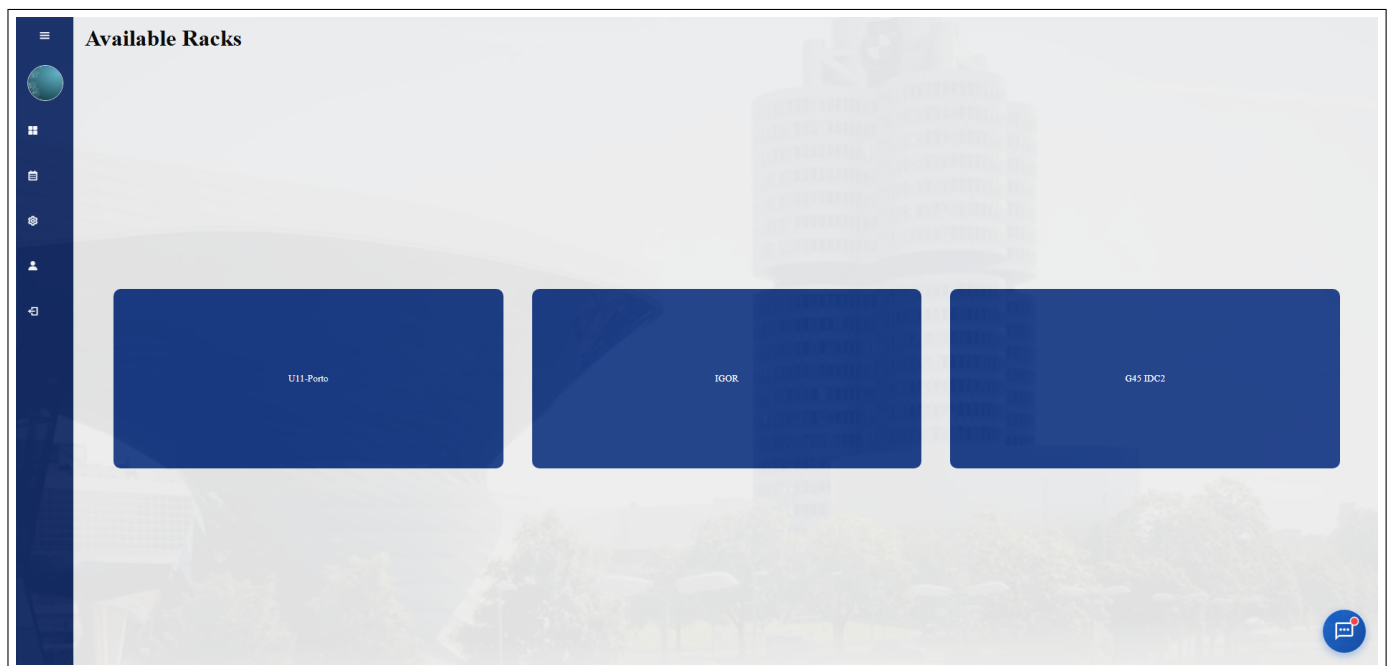


Figure 5.12: Rack Selection Page — Available Test Benches

5.8.5 Calendar Page (calendar.html)

The calendar page provides a reservation management system that allows testers to book time slots on specific test benches. The calendar is implemented using FullCalendar (version 5.10.1), a popular open-source JavaScript calendar library that provides an interactive, drag-and-drop interface [LLC, 2023].

The calendar supports the following features:

Time Slot Reservation: Users can create reservations by clicking on a date and specifying start/end times or selecting "All Day" to reserve the bench for the entire day.

Conflict Detection: When a new reservation is submitted, the Django backend checks for overlapping reservations on the same rack. If a time conflict is detected, the reservation is rejected and the user is informed. This prevents double-booking.

Event Visualization: Existing reservations are displayed as colored events on the calendar, showing the username of the person who booked each slot. This provides visibility into bench availability at a glance.

Reservation Deletion: Users can delete their own reservations by clicking on the event. Admin users (`is_staff=True`) can delete any user's reservation. A confirmation dialog prevents accidental deletions.

Rack-Specific View: Before accessing the calendar, users select which rack to view on the `select_rack_for_calendar` page. Each rack has its own calendar, ensuring that reservations are tracked per test bench.

URL: `/calendar/<rack_id>/` | View: `calendar_view()` | Data: `/calendar_data/<rack_id>/` (JSON)

Figure 5.13 below show the calendar interface.

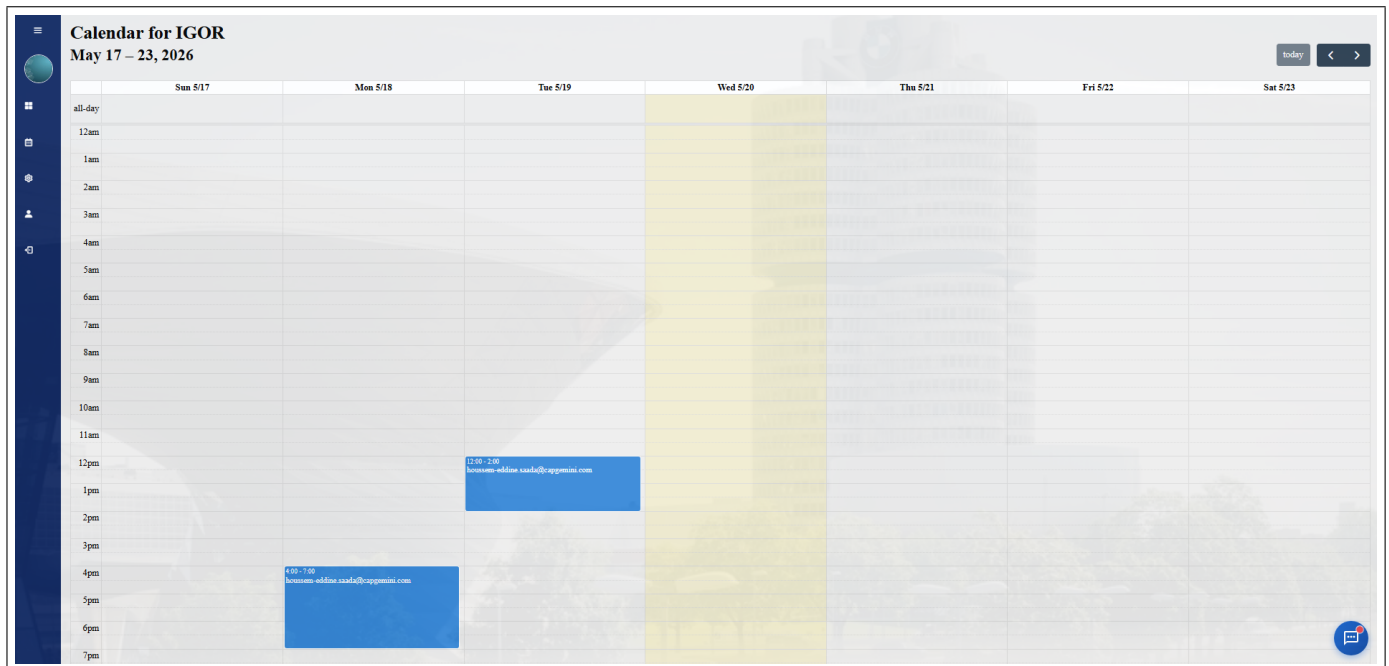


Figure 5.13: Calendar Page — Reservation System with FullCalendar

5.8.6 Remote Testing Page (`srcpy.html`)

The Remote Testing page is the central interface of the application. It is displayed when a user connects to a specific test bench. This page provides four main functionalities:

1. **Screen Mirroring (WebScrcpy)** An embedded iframe displays the WebScrcpy interface, showing the Head Unit's screen in real-time. The WebScrcpy URL is constructed from the rack's ip or url field in the database. The tester can interact with the Head Unit's display by clicking, dragging, and typing directly in the browser window.
2. **Live Speech** A "Live Speech" button activates the microphone-based voice testing mode. When clicked, the script captures the tester's voice through the local microphone, converts it to text using Vosk, and sends it to the remote PC for playback through the sound cable.
3. **Dynamic Speech** A text input field and "Send" button allow the tester to type voice commands that are sent to the remote PC for text-to-speech playback.
4. **Command Execution** A text area and "Execute" button provide a general-purpose command execution interface. The command output is displayed in a live output log area below the input.

This feature enables advanced testing tasks such as installing APKs via ADB, querying device properties, or running diagnostic commands.

URL: /scrcpy/<title>/<rack_id>/ | View: scrcpy() | Authentication: @login_required + active reservation

Figures 5.14 and 5.15 below illustrate the remote testing page .

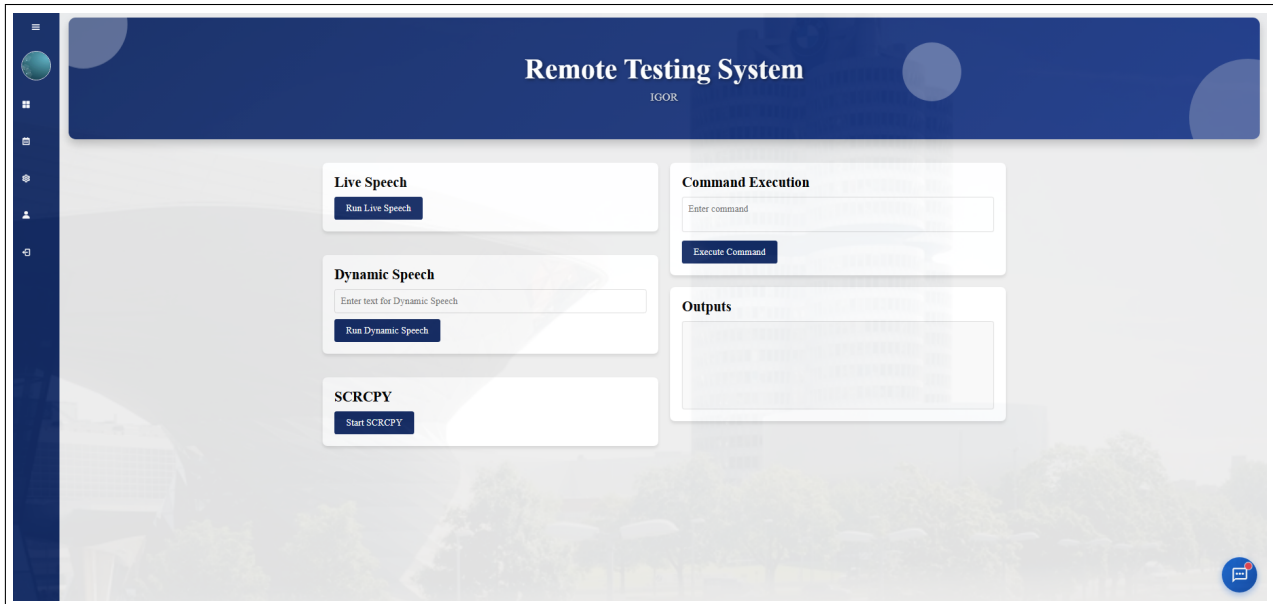


Figure 5.14: Remote Testing Page — Voice Testing + Command Execution

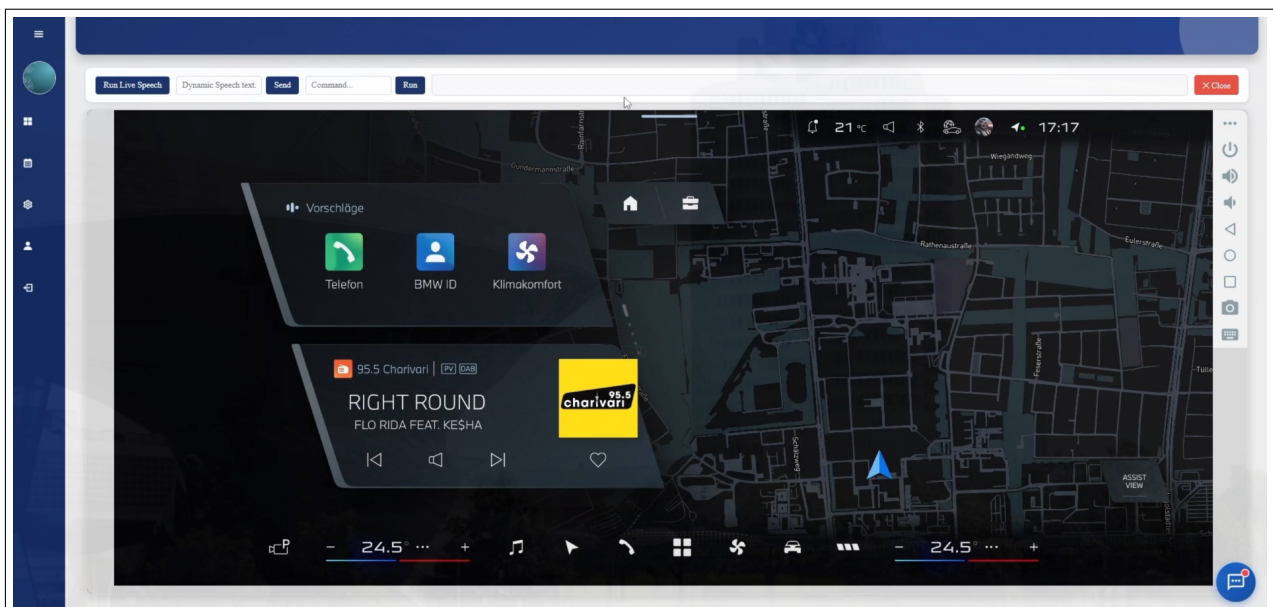


Figure 5.15: Remote Testing Page — WebScrcpy

5.8.7 User Settings Page (settings.html)

The settings page allows authenticated users to manage their profile. It provides two main features: **Profile Picture Upload:** Users can upload a profile picture that is stored in the /media/ directory and displayed in the navigation bar. The upload uses Django’s built-in file handling with MEDIA_ROOT configuration.

Password Change: Users can change their password using Django’s CustomPasswordChangeForm, which requires the current password and the new password (entered twice for confirmation). Password validation is handled by Django’s built-in validators.

URL: /settings/ | View: settings() | Authentication: @login_required

Figure 5.16 below shows the user settings page with profile picture upload and password change form.

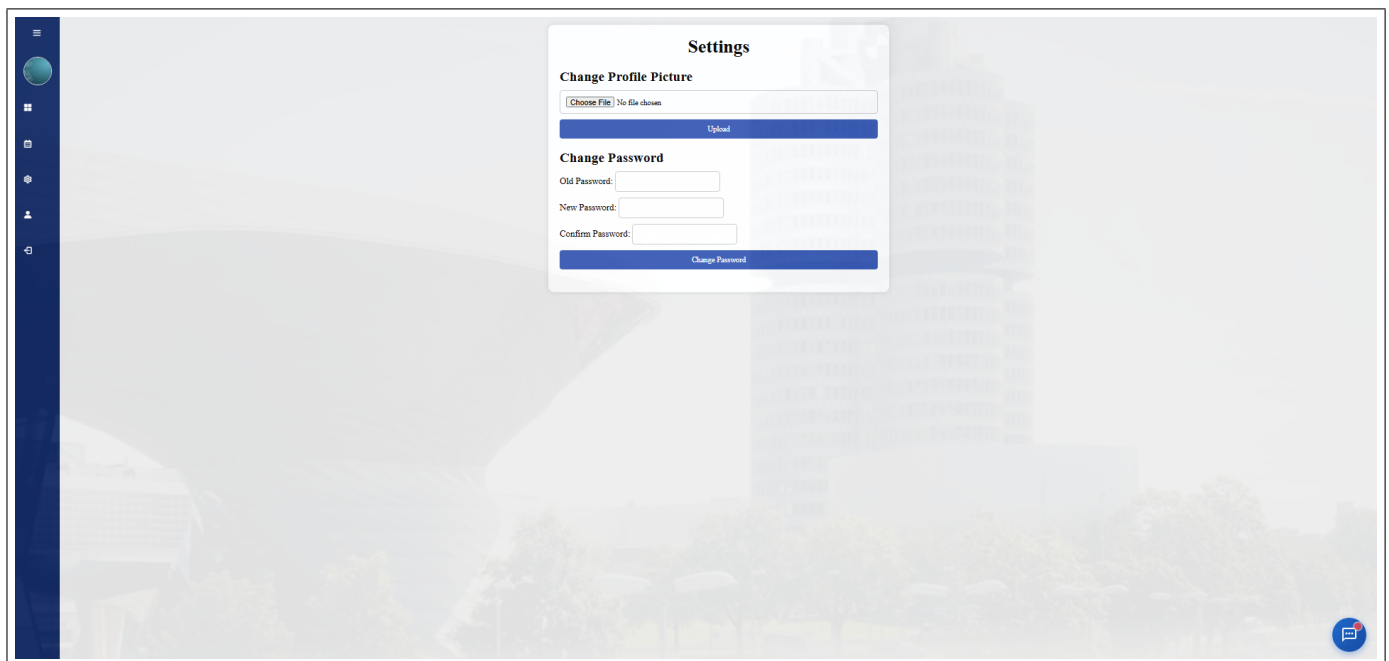


Figure 5.16: User Settings Page — Profile Picture and Password Change

5.8.8 User Management (user_management.html)

The user management page displays a table of all registered users with their username (QX Number), email, staff status, and action buttons (Edit, Delete). Admin users can perform the following actions from this page:

Add User: Navigate to /admin/users/add/ to create a new user account with the CustomUserCreationForm (username, email, password, staff status).

Edit User: Navigate to /admin/users/edit/<user_id>/ to modify an existing user’s information (username, email, staff status).

Delete User: Navigate to /admin/users/delete/<user_id>/ for a confirmation dialog before permanently removing the user account.

URL: /admin/users/ | View: user_management() | Authentication: @login_required + is_staff=True

Figures 5.17 and 5.18 below show the user management page and the add user form.

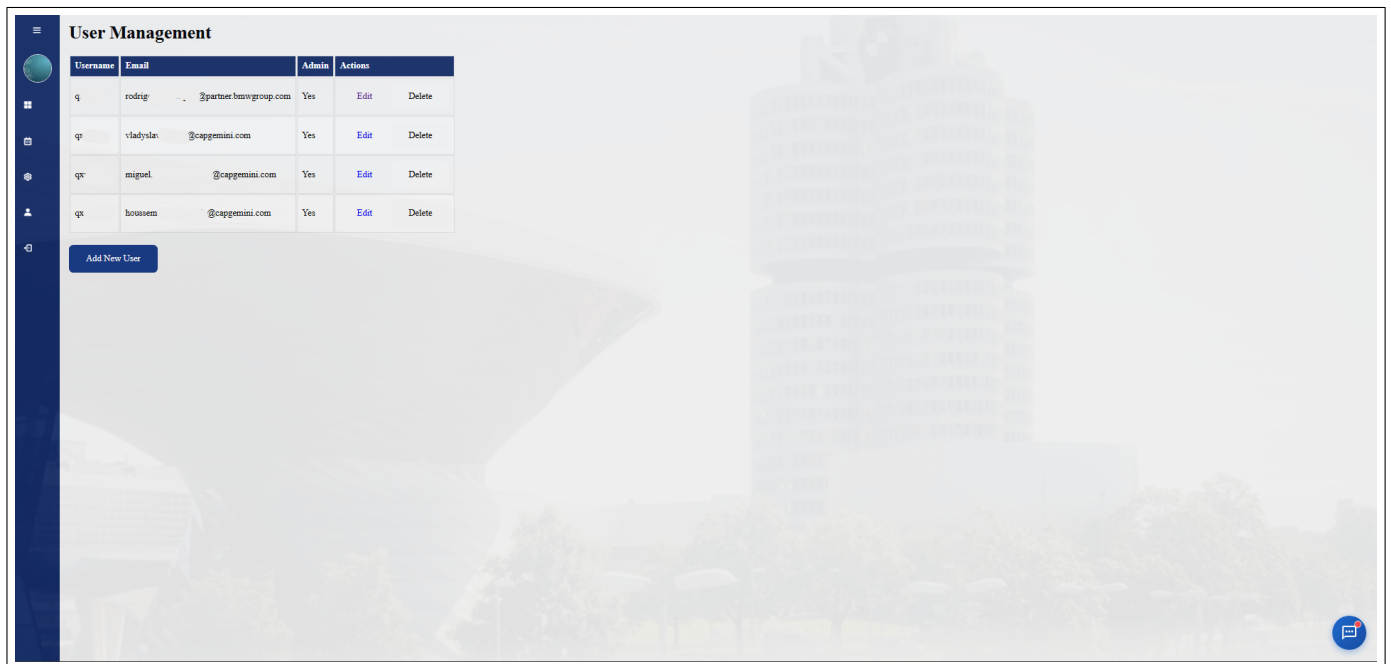


Figure 5.17: User Management Page — User List with Actions

The screenshot shows the 'Add User' form. It is a white modal box with a title 'Add User'. The form contains the following fields and text:

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address:

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

Staff status: ☐ Designates whether the user can log into this admin site.

At the bottom are two buttons: 'Add' and 'Cancel'.

Figure 5.18: Add User Form

5.8.9 Rack Management (list_racks.html + edit_rack.html)

The rack management system allows admin users to configure the test benches available in the system. The list page (/racks/) displays all racks in a table with their name, IP/URL, and PC IP. The edit page (/racks/edit/<id>/) provides a form to modify rack properties or delete a rack entirely. When adding a new rack, the admin specifies:

Name: A descriptive name for the test bench (e.g., "Rack 3 — Central Display").

IP or URL: The WebScrcpy server address for accessing this rack's screen mirroring interface.

PC IP: The IP address of the test PC associated with this rack, used for SSH tunnel creation.

URL: /racks/ (list) and /racks/edit/<id>/ (form) | Authentication: @login_required + is_staff=True
Figure 5.19 below shows the rack management page where admins configure test benches.



Figure 5.19: Rack Management Page — Test Bench Configuration

5.8.10 Chatbot Feedback Management (chatbot_feedback.html)

The Chatbot Feedback page enables admin users to review user interactions with the AI assistant, including queries, feedback indicators (thumbs up/down), and detailed responses. Each entry includes a View button that opens a modal displaying the full conversation, such as the previous user message, AI response, follow-up input, and the final analysis generated by the system. This feature helps administrators evaluate chatbot performance, monitor defect analysis quality, and identify areas for improvement.

URL: /chatbot/feedback/ | View: chatbot_feedback() | Authentication: @login_required + is_staff=True
Figure 5.20 below shows the chatbot feedback list page and the detailed interaction modal.

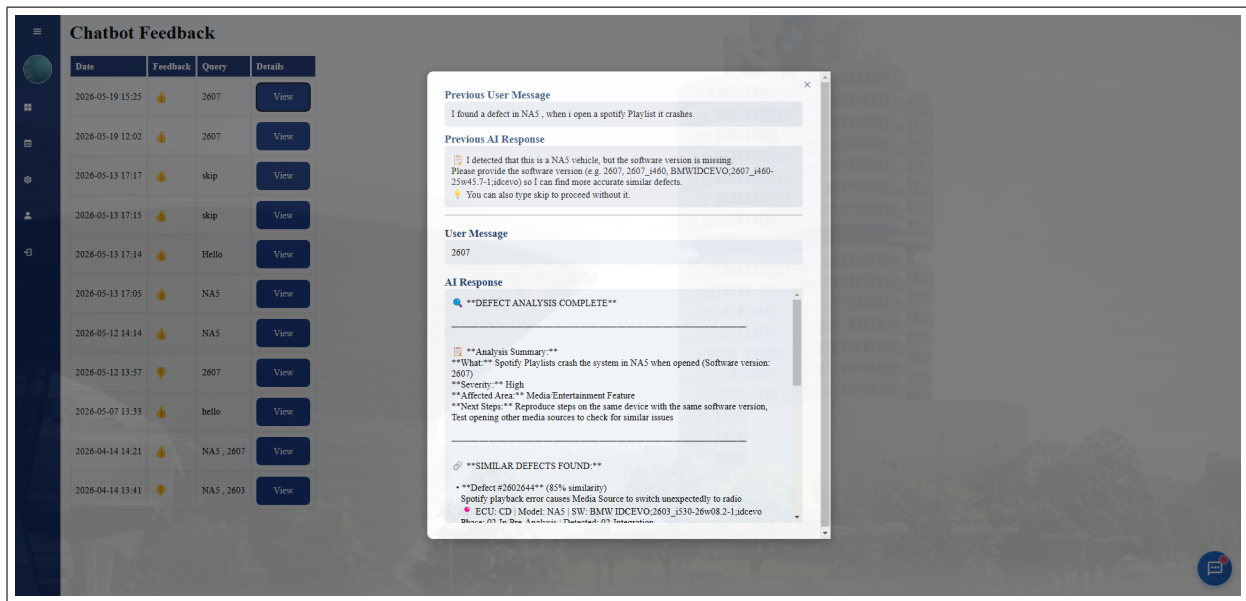


Figure 5.20: Chatbot Feedback Page — Interaction List and Detailed View

5.9 AI Chatbot Widget Integration

The AI-powered defect analysis chatbot is integrated into the web application as a floating widget that appears on all authenticated pages. The widget is included in the base.html template through Django's template inclusion system, making it accessible from any page without navigation.

5.9.1 Widget Design

The chatbot widget is implemented as a self-contained HTML/CSS/JavaScript component located in chatbot/templates/chatbot_widget.html. Its visual design follows modern chat interface conventions. Figure 5.21 below show the chatbot widget.

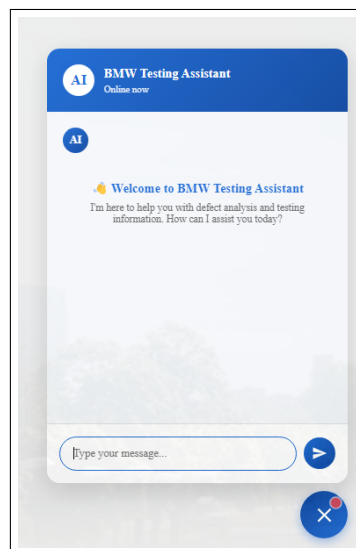


Figure 5.21: AI Chatbot Widget

5.9.2 REST API Endpoints

The chatbot widget communicates with the Django backend through four REST API endpoints, all prefixed with `/chatbot/`.

Table 5.12 below documents each endpoint:

Endpoint	Method	Request Body	Response	Purpose
<code>/chatbot/message/</code>	POST	<code>{"message": "user text"}</code>	<code>{"success": true, "response": "AI text", "message_type": "...", "keywords": [...], "analysis": "...}"}</code>	Main conversation endpoint. Handles all message types (defect, question, greeting).
<code>/chatbot/analyze/</code>	POST	<code>{"description": "defect text"}</code>	<code>{"success": true, "keywords": [...], "analysis": "...}"}</code>	Direct defect analysis without conversation context.
<code>/chatbot/keywords/</code>	POST	<code>{"text": "...", "num_keywords": 6}</code>	<code>{"success": true, "keywords": [...]"}</code>	Extract keywords from text without full analysis.
<code>/chatbot/clear/</code>	POST	(empty)	<code>{"success": true, "message": "...}"}</code>	Clears the conversation history for the current session.

Table 5.12: Chatbot API Endpoints

5.9.3 Session Management

The chatbot uses Django’s built-in session framework to maintain conversation state across multiple messages.

Table 5.13 below describes the two session variables used:

Session Variable	Type	Purpose	Limit
<code>conversation_history</code>	List of dicts <code>[{"role": "...", "content": "..."}]</code>	Stores the conversation messages exchanged between the user and the AI.	Maximum 20 messages. Oldest messages are discarded.
<code>pending_defect</code>	Dict or None <code>{"defect": "...", "missing": [...]}"}</code>	Stores a defect description that is awaiting additional context (car model, SW version) from the user.	Cleared after the follow-up message is processed.

Table 5.13: Session Variables for Conversation Management

5.9.4 CSRF Protection

All chatbot API requests include a CSRF token in the `X-CSRFToken` header. The token is extracted from the Django template using the `{% csrf_token %}` tag and passed with every `fetch()` request from the chatbot JavaScript (`chatbot.js`). This prevents cross-site request forgery attacks, where a malicious website could trick a user’s browser into making unauthorized POST requests to the chatbot endpoints.

5.9.5 User Feedback System

To support continuous quality monitoring of the AI chatbot, a feedback system was integrated directly into the chat widget. After each AI response, users can rate the answer using **thumbs up** or **thumbs down** buttons.

Figure 5.22 below shows the feedback buttons as they appear in the chatbot widget after an AI response. The implementation is composed of three main parts:

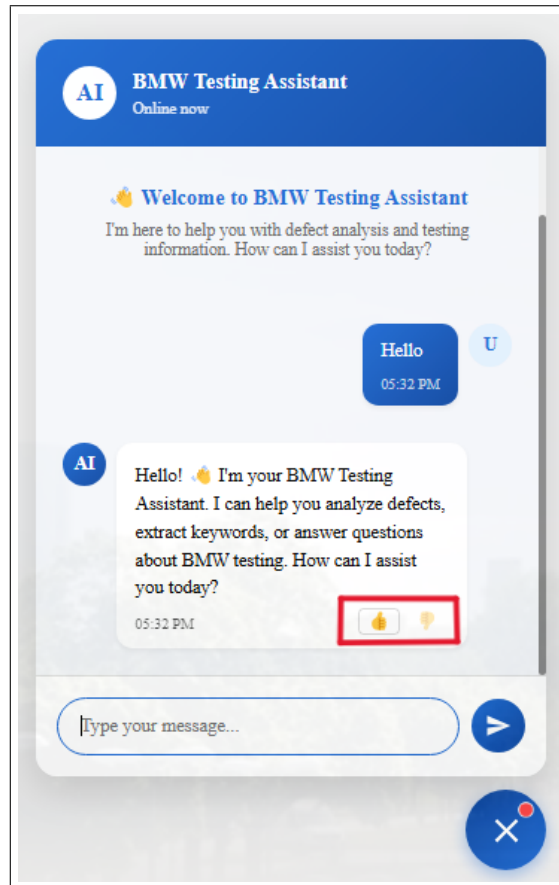


Figure 5.22: Chatbot Feedback Buttons — Thumbs-Up / Thumbs-Down in Widget

- **Backend Model (SearchFeedback):** A Django model stores each feedback entry, including the user query, AI response snippet, previous conversation context, feedback type (*"up"* or *"down"*), session key, and timestamp.
- **REST API Endpoint:** A POST endpoint (`/chatbot/submit_feedback/`) receives feedback in JSON format, validates the feedback value, stores the record securely using Django's CSRF protection, and returns a success response.
- **Frontend Integration:** In `chatbot.js`, clicking a feedback button sends the current and previous conversation context to the backend via a fetch POST request. Buttons are disabled after submission to prevent duplicates, with visual color feedback indicating the selected rating.

5.10 Security Considerations

Security is a critical concern for the BMW Remote Testing Platform, given that it operates within BMW's corporate network and handles access to vehicle test hardware. The following security measures are implemented across the system's layers.

5.10.1 Communication Security

Table 5.14 below summarizes the security mechanisms implemented at each communication layer:

Layer	Mechanism	Protection
SSH Tunnel	AES-256 encryption (paramiko/sshtunnel)	All traffic between tester and test PC is encrypted. Prevents eavesdropping on ADB commands and voice data.
ADB Connection	RSA key authentication (adbkey file)	Only authorized clients with the correct RSA key can connect to the Head Unit's ADB daemon.
Web Application	Django session cookies (HttpOnly, Secure)	Session tokens are stored in HttpOnly cookies to prevent JavaScript access (XSS mitigation).
API Requests	CSRF token validation (X-CSRFToken header)	All POST requests require a valid CSRF token to prevent cross-site request forgery.
BMW Network	VPN + corporate firewall	The application is only accessible within BMW's corporate network or via VPN.

Table 5.14: Security Layers and Their Protection Mechanisms

5.10.2 Django Security Middleware

Django's security middleware stack provides multiple layers of protection that are enabled by default in the project's configuration.

Table 5.15 below describes each middleware component:

Middleware	Protection
SecurityMiddleware	HTTP Strict Transport Security (HSTS) headers, XSS protection headers, content type sniffing prevention.
SessionMiddleware	Server-side session storage with encrypted session IDs. Prevents session hijacking through cookie security.
CsrfViewMiddleware	Validates CSRF tokens on all POST requests. Prevents cross-site request forgery attacks.
AuthenticationMiddleware	Associates authenticated users with requests. Enables <code>@login_required</code> and <code>is_staff</code> checks.
XFrameOptionsMiddleware	Sets X-Frame-Options: DENY header. Prevents the site from being embedded in iframes (clickjacking protection).

Table 5.15: Django Security Middleware and Their Protections

5.10.3 Authentication and Authorization

The application implements a two-tier authentication and authorization system:

Authentication: Users authenticate with their BMW QX Number (employee identifier) and password. Passwords are stored as hashed values using Django’s PBKDF2 algorithm with SHA256 (not in plaintext). The `@login_required` decorator protects all pages except the landing page and login page.

Authorization: Two access levels exist: Regular User (can view racks, make reservations, access remote testing, use the chatbot) and Admin (all regular permissions plus user management, rack management, and the ability to delete any reservation). Admin status is controlled by the `is_staff` flag on the User model.

5.10.4 Data Privacy

The project handles BMW-internal defect data that is subject to strict data privacy policies. The following measures ensure compliance:

- The MongoDB database connection uses a read-only viewer account, preventing any modification or deletion of defect records.
- The ChromaDB vector database is stored locally on the Django server’s filesystem, not in any external cloud storage.
- The application operates entirely within BMW’s corporate network perimeter, accessible only via VPN for remote users.
- No defect data is displayed to unauthorized users. All defect-related endpoints require authentication.
- The BMW TISAX (Trusted Information Security Assessment Exchange) framework governs all information security practices within the project.

5.11 Version Control

The development of the BMW Remote Testing Platform relied on **GitLab** as the central platform for version control, collaboration, and project management. GitLab was used to manage source code, track issues, handle merge requests, and support collaborative development throughout the project lifecycle [GitLab, 2022]. The development workflow followed a feature-branch strategy:

Main Branch: The main (or master) branch contains the stable, production-ready code. Only reviewed and tested code is merged into main.

Feature Branches: Each new feature or bug fix is developed on a dedicated branch (e.g., `feature/calendar-page`, `feature/chatbot-widget`, `fix/proxy-403-error`). This isolates changes and prevents conflicts.

Merge Requests: When a feature branch is complete, a Merge Request (MR) is created in GitLab. The MR is reviewed by a team member before being merged into main. This code review process ensures code quality and catches potential issues.

Commit History: All changes are tracked with descriptive commit messages, providing a complete audit trail of the project’s evolution.

Figure 5.23 below shows the GitLab repository.

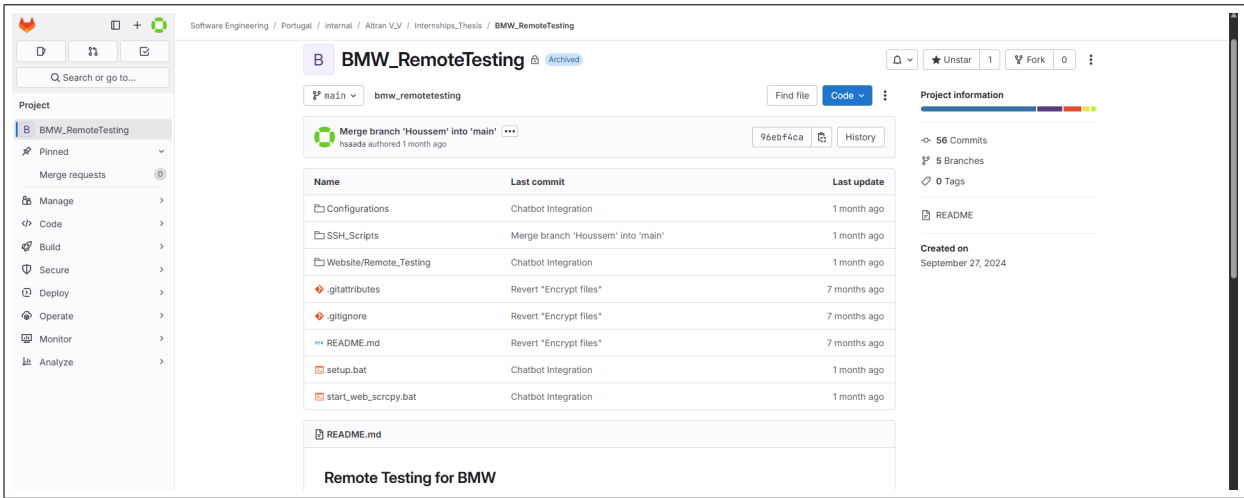


Figure 5.23: GitLab Repository

5.12 Hosting and Future Production Deployment

5.12.1 Current Deployment (Development Environment)

The current deployment uses Django’s built-in development server, accessible within BMW’s internal network. The WebScripy server runs on a dynamically assigned port on the same machine. This setup is appropriate for the current development and testing phase, where the user base is limited to the EVV team and the system is accessed primarily through BMW’s VPN.

5.12.2 Production Migration Path

For a production deployment, the following changes would be required. Table 5.16 below outlines the migration path:

Aspect	Current State	Production Change
Web Server	Django runserver (development server)	Gunicorn or uWSGI behind Nginx reverse proxy for performance and security.
Database	SQLite3 (file-based)	PostgreSQL or MySQL for concurrent access support and better performance.
DEBUG Mode	DEBUG = True	DEBUG = False with proper error logging and monitoring.
ALLOWED_HOSTS	Empty list (all hosts)	Restricted to specific hostnames (e.g., remote-testing.bmw.com).
Static Files	Django serves static files	Nginx serves static files directly (collect-static + Nginx static location).

Table 5.16: Development vs. Production Configuration Changes

The migration to a production environment would also involve containerization (Docker) for consistent deployment across environments, a CI/CD pipeline (GitLab CI) for automated testing and deployment, and monitoring (Prometheus + Grafana or similar) for system health tracking. These enhancements would transform the current development setup into a robust, scalable production system suitable for organization-wide deployment.

5.13 Conclusion

This chapter presented the complete development of the BMW Remote Testing Platform, from the physical hardware layer through the software application stack. The system integrates a diverse set of technologies — SSH for secure communication, ADB for Android device control, WebScrcpy for browser-based screen mirroring, Vosk and pyttsx3 for voice testing, Django for the web application, and a multi-agent AI chatbot for defect analysis — into a cohesive platform that enables remote automotive infotainment testing.

The web application provides 9 functional areas covering authentication, test bench reservation, remote control, voice testing, administration, and AI-assisted defect analysis. The system operates within BMW's secure corporate network, with SSH encryption, CSRF protection, and role-based access control ensuring security compliance.

The current deployment uses Django's development server for the EVV team's testing needs, with a clear migration path to a production environment identified for future scaling.

General conclusion

The development of the Remote Testing System and the AI-Powered Defect Analysis Chatbot marks a clear improvement compared to the client's previous way of working. By combining existing tools with modern technologies like semantic search, large language models, and vector databases, this project showed that it's not only possible to enhance current systems, but also to make them significantly more efficient. The work followed the CRISP-DM methodology, going step by step from understanding the business needs to building and validating the final solution. Working closely with Capgemini Engineering Tunisia helped validate the project and opened the door for wider adoption in a real enterprise environment.

From a technical point of view, this project brought together a variety of technologies, from web development tools like Python, Django, and JavaScript, to advanced AI components such as GPT-4o, Amazon Titan Embed, and ChromaDB. This combination resulted in a solid and fully functional platform. On the remote testing side, key features include voice recognition testing, a reservation system, screen mirroring through the browser, and an administration panel. On the AI side, the chatbot can understand defect descriptions in both German and English and perform intelligent searches across thousands of past records, making defect analysis much faster and more accurate. The platform is currently being tested by BMW's manual testing team, with strong potential to move into production. It already helps reduce the need for physical presence, saving time and costs, while making collaboration between distributed teams much easier. At the same time, the chatbot transforms a slow, manual process into a smarter and more intuitive one, capable of understanding meaning instead of just matching keywords.

Beyond the technical results, this project reflects the skills developed throughout the academic journey — especially problem-solving, adaptability, and the ability to work with complex systems. It also highlights how combining software engineering with artificial intelligence can create real value in an industrial context. Like any software project, continuous improvement and good documentation will be essential to ensure the platform evolves and keeps meeting user needs over time.

Future Work

The scalability of the system, as mentioned throughout this report, is a significant advantage, allowing easy integration of new functionalities and improvements. Despite the challenges encountered, the project successfully achieved its objectives. Reflecting on these aspects is essential to guide future enhancements and maximize its value for both the company and the client. The following objectives have been identified for future work:

- **Continuous frontend improvement:** Correction of minor visual bugs and evolution of the user interface based on technological advancements and user feedback.
- **Multi-panel simultaneous display:** Extending control and mirroring capabilities to multiple devices (e.g., Head-Up Display, Instrument Cluster) for a more complete testing experience.

- **New website functionalities:** Remote power control of test setups, automatic storage of test logs, and email reminders for scheduled reservations.
- **Additional voice recognition languages:** Expanding testing beyond English to include languages such as German and French to better support international teams.
- **AI chatbot enhancements:** Implementing automated database updates, exploring fine-tuned embedding models for automotive terminology, and adding support for attachment analysis (screenshots, logs).
- **Production deployment:** Transitioning to a production-ready infrastructure (Gunicorn/Nginx, PostgreSQL, Docker, CI/CD) to ensure scalability, monitoring, and organization-wide accessibility.

The solution developed in this project provides a strong and scalable foundation, with real potential for large-scale adoption and continuous evolution in remote testing environments.

Bibliography

- [AI, 2024] AI, M. (2024). "llama 3.1 — open-source large language model". <https://ai.meta.com/blog/meta-llama-3-1/>.
- [Amazon, 2024a] Amazon (2024a). "amazon nova models — amazon bedrock". <https://docs.aws.amazon.com/bedrock/latest/userguide/nova-models.html>.
- [Amazon, 2024b] Amazon (2024b). "amazon titan text embeddings — amazon bedrock". <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>.
- [Anthropic, 2024] Anthropic (2024). "claude 3.5 sonnet — ai assistant". <https://www.anthropic.com/claude>.
- [Bloom, 2012] Bloom, N. (2012). "*Does Working from Home Work? Evidence from a Chinese Experiment*". The Quarterly Journal of Economics.
- [Capgemini, 2024] Capgemini (2024). "capgemini annual report 2024 — universal registration document". <https://www.capgemini.com/about-us/investor-information/>.
- [Cephei, 2024] Cephei, A. (2024). "vosk — offline speech recognition api". <https://alphacephei.com/vosk/>.
- [Chapman, 2000] Chapman, P. (2000). "crisp-dm 1.0 — step-by-step data mining guide". <https://www.the-modeling-agency.com/crisp-dm.pdf/>.
- [Cohere, 2024] Cohere (2024). "cohere embed v3 — multilingual embeddings". <https://cohere.com/embed>.
- [Dev., 2024] Dev., A. (2024). "android debug bridge (adb)". <https://developer.android.com/tools/adb>.
- [Focus, 2024] Focus, M. (2024). "alm octane — application lifecycle management". <https://www.microfocus.com/en-us/products/alm-octane/overview>.
- [Foundation, 2024] Foundation, D. S. (2024). "django — the web framework for perfectionists with deadlines". <https://docs.djangoproject.com/>.
- [Gallup, 2023] Gallup, I. (2023). "state of the global workplace report". <https://www.gallup.com/workplace/349484/state-of-the-global-workplace.aspx>.
- [Genymobile, 2024] Genymobile (2024). "screpy — display and control your android device". <https://github.com/Genymobile/screpy>.

- [GitLab, 2022] GitLab (2022). "gitlab — the devsecops platform". <https://about.gitlab.com/>.
- [IOFS, 2015] IOFS (2015). "iso 9001:2015 — quality management systems — requirements.". <https://www.iso.org/standard/62085.html>.
- [Johnson, 2021] Johnson, J. (2021). "billion-scale similarity search with gpus.". <https://github.com/facebookresearch/faiss>.
- [KDnuggets, 2014] KDnuggets (2014). "what main methodology are you using for your analytics, data mining, or data science projects?". <https://www.kdnuggets.com/polls/2014/analytics-data-mining-data-science-methodology.html>.
- [LangChain, 2024] LangChain (2024). "langchain — build context-aware reasoning applications.". <https://docs.langchain.com/>.
- [LLC, 2023] LLC, F. (2023). "fullcalendar — full-sized javascript calendar". <https://fullcalendar.io/>.
- [Milvus, 2024] Milvus (2024). "milvus — open-source vector database built for ai.". <https://milvus.io/>.
- [MongoDB, 2024] MongoDB, I. (2024). "mongodb atlas — cloud database service.". <https://www.mongodb.com/atlas>.
- [NetMind, 2024] NetMind (2024). "ws-scrpcy — web-based scrpcy client using websocket.". <https://github.com/nicekid1/ws-scrpcy>.
- [OpenAI, 2024] OpenAI (2024). "gpt-4o — multimodal ai model.". <https://openai.com/index/hello-gpt-4o/>.
- [Reimers, 2019] Reimers, N. (2019). "sentence-bert: Sentence embeddings using siamese bert-networks". <https://arxiv.org/abs/1908.10084>.
- [Systems, 2024] Systems, P. (2024). "pinecone — vector database for machine learning". <https://www.pinecone.io/>.
- [Weaviate, 2024] Weaviate (2024). "weaviate — open source vector database.". <https://weaviate.io/>.
- [Ylonen, 2006] Ylonen, T & Lonvick, C. (2006). "the secure shell (ssh) protocol architecture". <https://www.rfc-editor.org/rfc/rfc4251>.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454